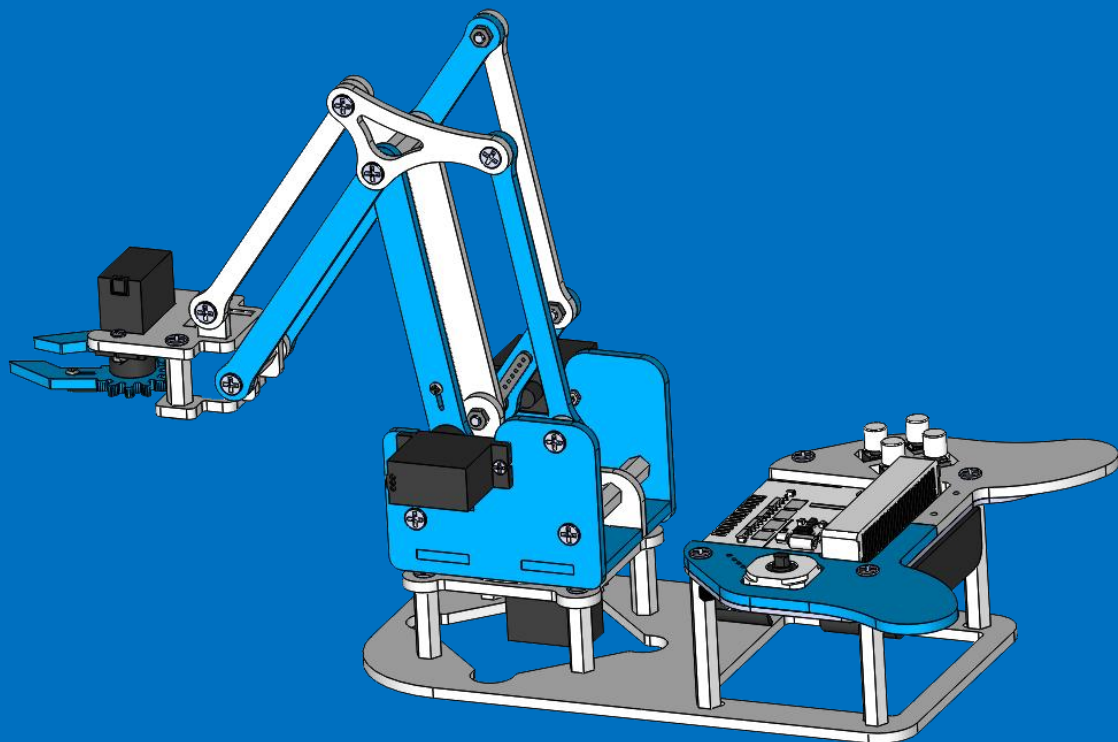


SIYEENOVE

ロボット アームキット micro:bit 用

MicroPython チュートリアル

2026 年 3 月 17 日



目次

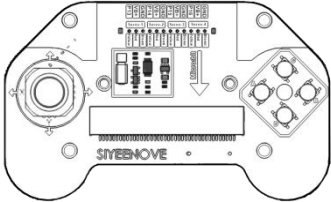
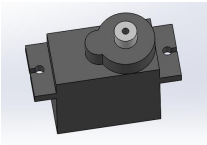
1. ロボットアームの紹介	4
1.1 パッケージ内容	4
1.2 仕様	6
1.3 製品寸法	6
1.4 SIYEENOVE ジョイスティックの紹介	7
1.4.1 ジョイスティックの仕様	7
1.4.2 インターフェース仕様	8
2. 組み立て	9
2.1 設置準備: サーボの角度を初期設定 (90 度に調整) します	9
2.2 ロボット組み立て前の注意事項	15
2.3 組み立て開始	15
ステップ 1 - サーボ 1 の取り付け	15
ステップ 2 - サーボ-2 の取り付け	16
ステップ 3 - サーボ-3 の取り付け	17
ステップ 4 - サーボ-4 の取り付け	19
ステップ 5 - ジョイスティックの電源を切り、サーボケーブルを抜く	20
ステップ 6 - ベースの取り付け (ナイロン製スタンドオフ	21
ステップ 7 - ベース (スチール製ボールベアリングプレート) の取り付け	22
ステップ 8 - ベースの取り付け (サーボ構造)	23
ステップ 9 - サーボホーンを回転ベースに取り付ける	24
ステップ 10 - アッパーアームリンケージの取り付け	25
ステップ 11 - 回転ベースの組み立て	26
ステップ 12 - 回転ベースをメインフレームに取り付ける	28
ステップ 13 - ターンテーブルに左サーボ構造を取り付ける	29
ステップ 14 - 上腕リンクの取り付け (左側)	30
ステップ 15 - ターンテーブルに右サーボ構造を取り付ける	31
ステップ 16 - プライマリアーム / エルボコネクタの取り付け (1)	32
ステップ 17 - プライマリアーム / エルボコネクタ (2) の取り付け	33
ステップ 18 - 前腕リンケージの取り付け (最初の位置)	34
ステップ 19 - 前腕リンケージの取り付け (第 2 位置)	35
ステップ 20 - 上腕と前腕のリンクを接続する	36
ステップ 21 - 左グリッパージョーを取り付ける	37
ステップ 22 - 右グリッパージョーを取り付ける	39
ステップ 23 - 手首の構造を取り付ける	40
ステップ 24 - グリッパを手首に取り付ける	42
ステップ 25 - 延長ケーブルでサーボ 4 を接続する	44
ステップ 26 - サーボをジョイスティックコントローラーに配線する	45
ステップ 27 - mJoystick をベースフレームに取り付ける	46
ステップ 28 - ケーブル管理:	48
3. ロボットアームの制御	50
3.1 制御コードのアップロード	50
3.2 ロボットアーム制御のクイックスタート	52

4. micro:bit Python エディタのクイックスタート	53
4.1 新しいプロジェクトを作成する	53
4.2 プロジェクトの保存	54
4.3 ファイルのインポート	56
4.4 コードのアップロード	57
4.5 micro:bit Python エディタの基本構文を学ぶ	62
5. 基本的なサンプルプロジェクト	63
5.1 ボタン	64
5.2 のジョイスティック	66
5.3 振動モーター	68
5.4 バッテリー電圧	69
5.5 サーボ	70
5.6 ロボットアームの制御	73
6.QA	78
7.お問い合わせ	80

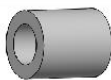
1. ロボットアームの紹介

SIYEENOVE Micro:bit ロボットアームは、学生や愛好家にロボット工学、コーディング、自動化の基礎を教えるために設計された、教育用かつプログラム可能なロボットアームです。BBC Micro:bit マイクロコントローラーを搭載したこのロボットアームは、STEM（科学、技術、工学、数学）分野における実践的な学習体験を提供します。

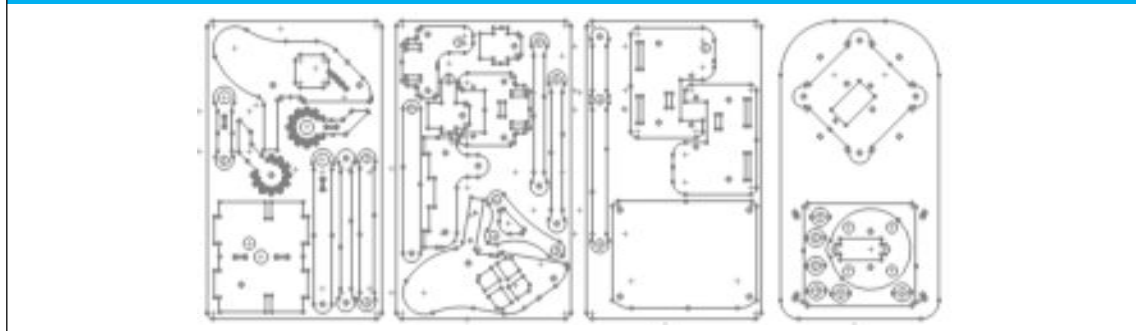
1.1 パッケージ内容

#パート 1: 電子部品		
ジョイスティック 1 個	サーボモーター 4 個	3 芯ケーブル 1 本
		

#パート 2: 締結部品 / 付属品		
以下に順にリストアップされた付属品は、すべてラベルとシリアル番号が貼られた小さな袋に梱包されています。袋に記載されたシリアル番号と品名を確認すれば、すぐに必要な部品が見つかります。		
① M3×12mm Screw 6Pcs 	② M3×9mm Screw 43Pcs 	③ M2×8mm Screw 10Pcs 
④ M1.4×5mm Screw 7Pcs 	⑤ M3 Nut 12Pcs 	⑥ M2 Nut 10Pcs 
⑦ M3×14mm Standoff 3Pcs 	⑧ M3×17+6mm Standoff 2Pcs 	⑨ M3×26mm Standoff 2Pcs 
⑩ M3×18mm Standoff 4Pcs 	⑪ M3×20mm Standoff 5Pcs 	⑫ M3×38mm Standoff 4Pcs 

13 5×3.2×3mm Spacer 8Pcs 	14 5×3.2×6mm Spacer 6Pcs 	15 Spiral Conduit 
16 Steel Ball 4Pcs 	17 Button Cop 4Pcs 	

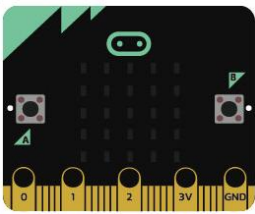
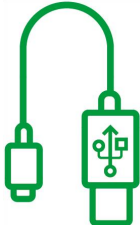
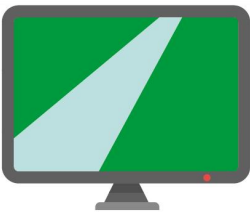
#第3部: アクリル板



#第4部: 工具

		
M3 ドライバー 1本	M1.5 ドライバー 1本	レンチ 1本

別途ご用意いただくもの

			
Micro:bit V2 1個	Micro USBケ ーブル	コンピュータ インターネット接続およ びUSBポート付き	1.5V AA電池 4個

1.2 仕様

制御方法: **SIYEENOVE** ジョイスティック (micro:bit は付属しません)

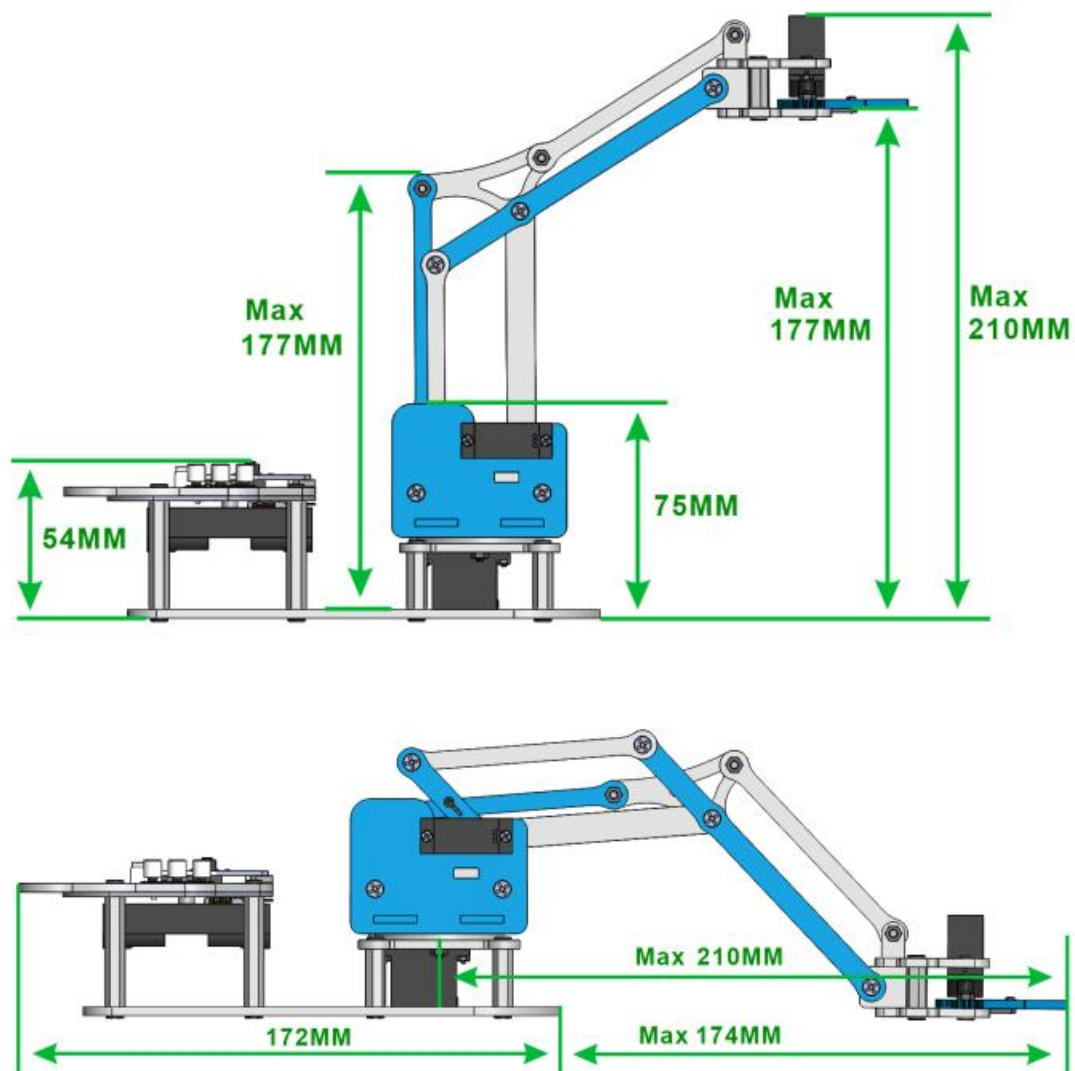
プログラミング方法: **MakeCode**

必要な電池の種類: 単 3 形電池 4 本

サーボタイプ: **MG90S 180°**

最大可搬重量: 60g 以下

1.3 製品寸法



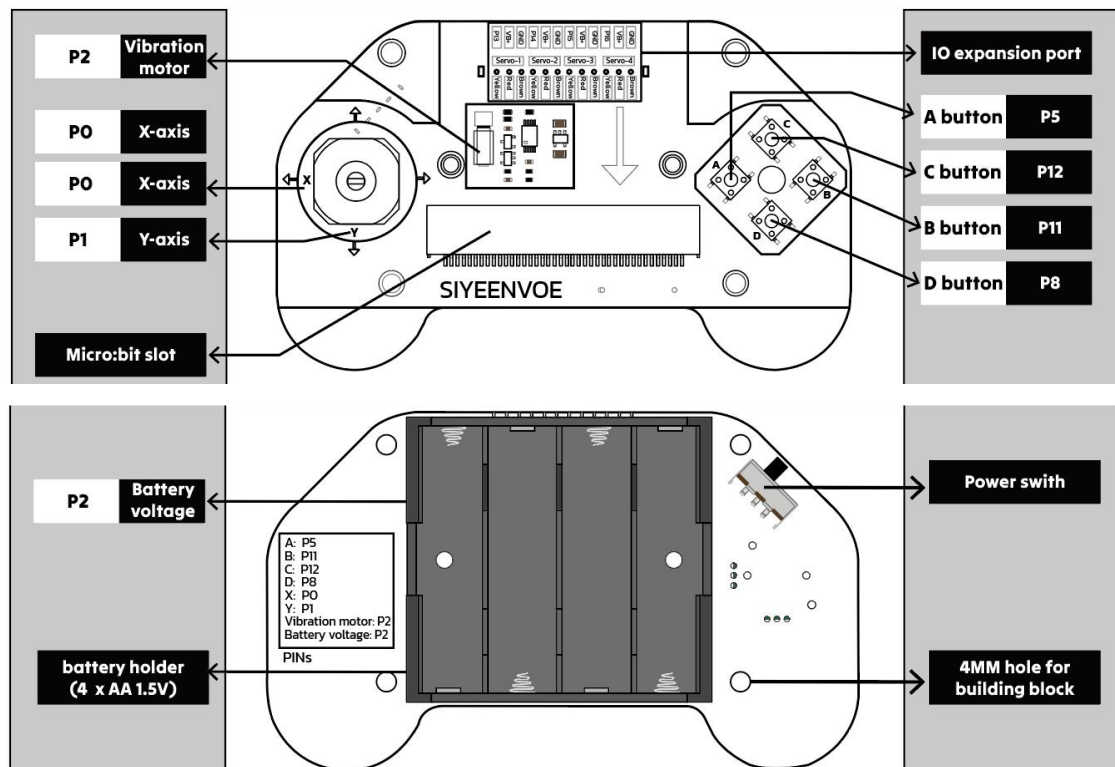
1.4 SIYEENOVE ジョイスティックの紹介

SIYEENOVE ジョイスティックコントローラは、ロボット工学、ゲーム、教育用途で使われる手動入力デバイスです。2 軸（X/Y 軸）の動きと統合されたボタンを備え、高精度な方向制御を可能にするため、ロボットアーム、ドローン、その他のモーションベースのシステムの操作に最適です。

1.4.1 ジョイスティックの仕様

シールド	
名称	mJoystick
SKU	M1E0001
USB コネクタ	
名称	Micro USB
対応マザーボード	
名称	Micro:bit
ピン	
デジタル I/O ピン	4 (P13、P14、P15、P16)
サーボピン	4 (P13, P14, P15, P16)
通信	
SPI	あり (P13, P14, P15, P16)
電源	
入力電圧（公称）	6V（単 3 形電池 4 本、1.5V/本）
VB	6V (単 3 形電池 4 本、1.5V/本)
振動モーター	
型番	3610 振動モーター
定格電圧/電流	2.7V/75mA
回転数	14000±2500RPM
外形寸法	
幅	72.5 mm
長さ	120 mm
高さ	29mm
重量	50 g

1.4.2 インターフェース仕様



名称	説明
振動モーター	Micro:bit の P2 ピンによって制御され、mJoystick にハプティックフ
X 軸	ジョイスティックの X 軸。そのアナログ値は Micro:bit の P0 ピンから読み取ることができます
Y 軸	ジョイスティックの Y 軸。そのアナログ値は Micro:bit の P1 ピンから読み取ることができます
Micro:bit スロット	Micro:bit のメイン制御ボードを挿入するために設計されています
単三電池 4 本 1.5V 電池ケース	この電池ケースには、標準的な 1.5V、直径約 14.5mm、高さ 50.5mm の単三電池を 4 本収納できます
電池電圧	単 3 形電池 4 本を装着すると、Micro:bit の P2 ピンから電圧を読み取ることができます
I/O 拡張	外部サーボやセンサーを接続するために使用します。VB ピンの電圧は、単 3 形電池 4 本の直列電圧 ($1.5V \times 4 = 6V$) に相当します
ボタン	Micro:bit の P5 ピンを介してその値を読み取ることができます
B ボタン	Micro:bit の P11 ピンを介してその状態を読み取ることができます
C ボタン	その値は Micro:bit の P12 ピンから読み取ることができます
D ボタン	その値は Micro:bit の P8 ピンから読み取ることができます
電源スイッチ	mJoystick の主電源を制御します。
4mm の穴	mJoystick には、拡張用に LEGO コネクタと互換性のある 4×4mm の

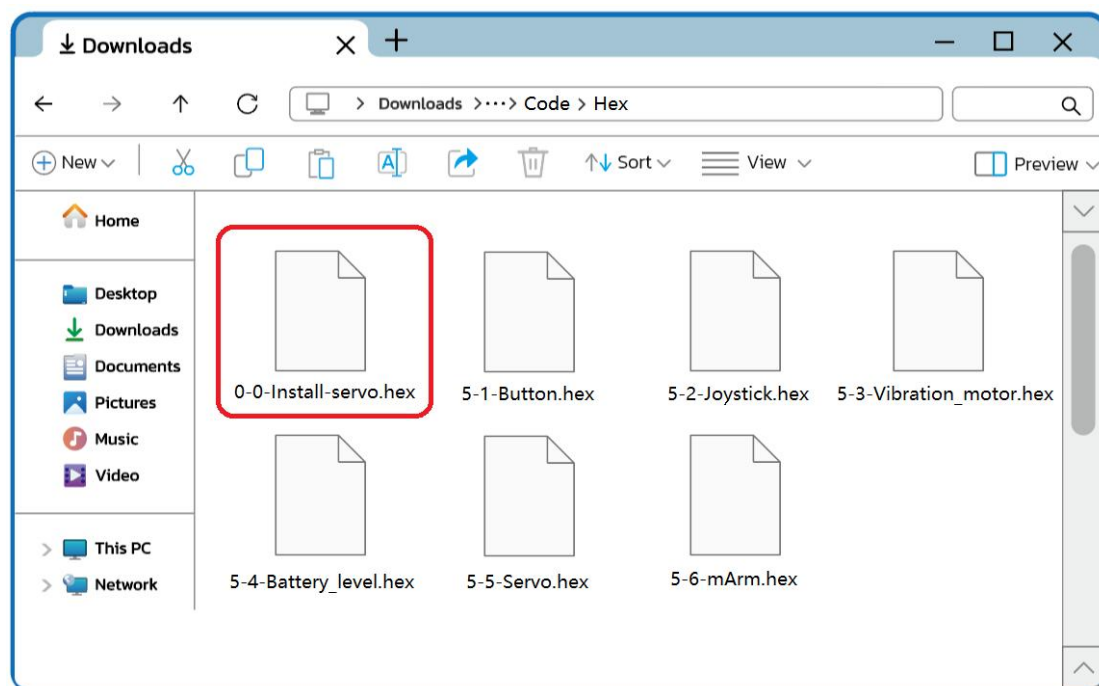
2. 組み立て

2.1 設置準備：サーボの角度を初期設定（90 度に調整）します

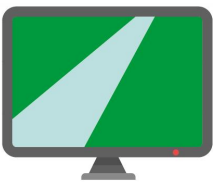
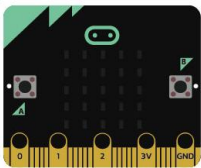
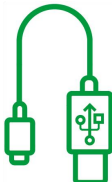
注意！

サーボには角度の初期化（90 度に設定）が必要なため、組み立て前に **microbit-0-0-Install-servo.hex** コードをアップロードする必要があります。適切なサーボの初期化を行わずに設置した場合、ロボットアームはあらかじめ設定されたプログラムやタスクを実行できなくなります。深刻な場合、電源投入時の起動によりサーボが詰まり、サーボが破損する可能性があります。

初期化コードは、以下の図に示すように、ダウンロードしたチュートリアルパッケージ内の「**Code>Hex**」フォルダに保存されています：

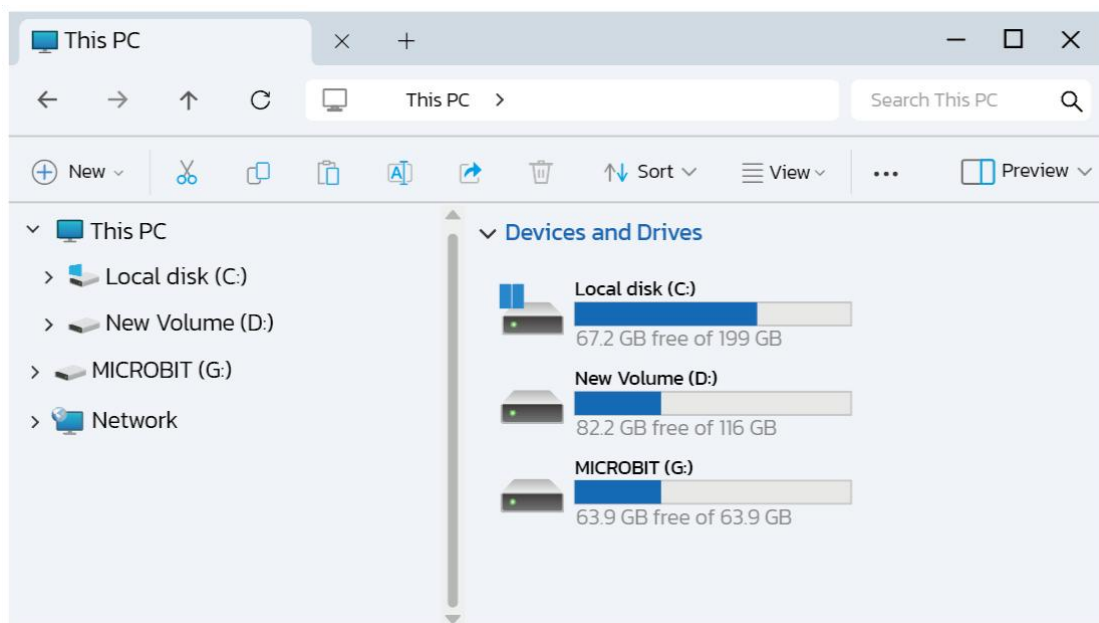


必要なツール：

PC	Micro:bit v2.x.x	Micro USB ケーブル
		

手順:

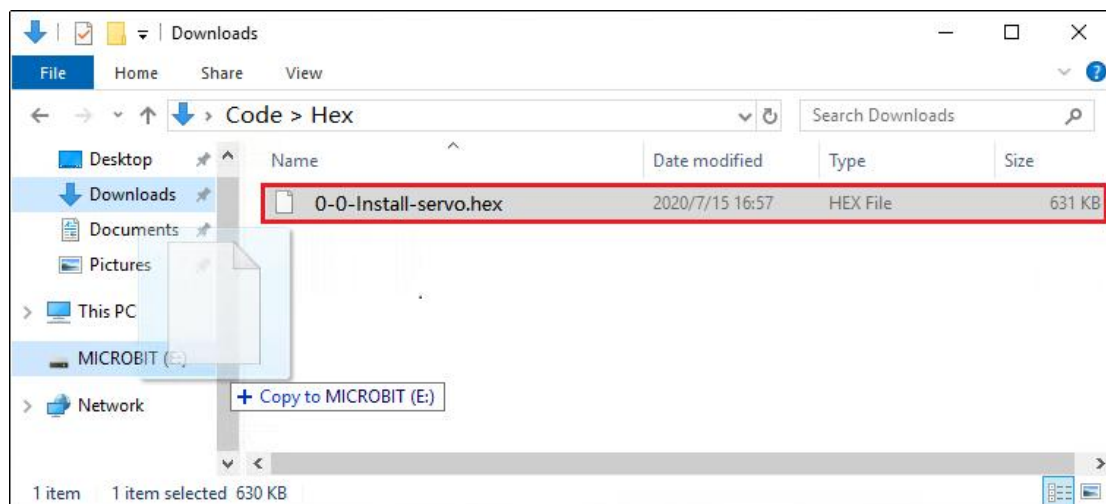
1. Micro USB ケーブルを使って micro:bit を PC に接続します。PC は micro:bit をリムーバブルドライブ（例：「MICROBIT」）として認識します。



2. HEX ファイル（microbit-0-0-Install-servo）を探します。以下の 2 つの方法のいずれかを使用して、micro:bit にコードを素早くアップロードします。

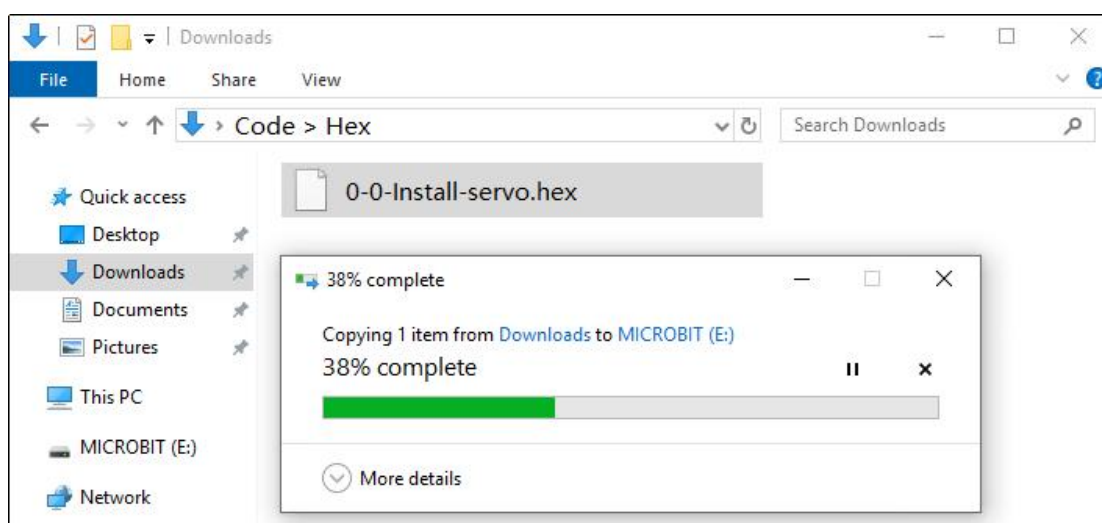
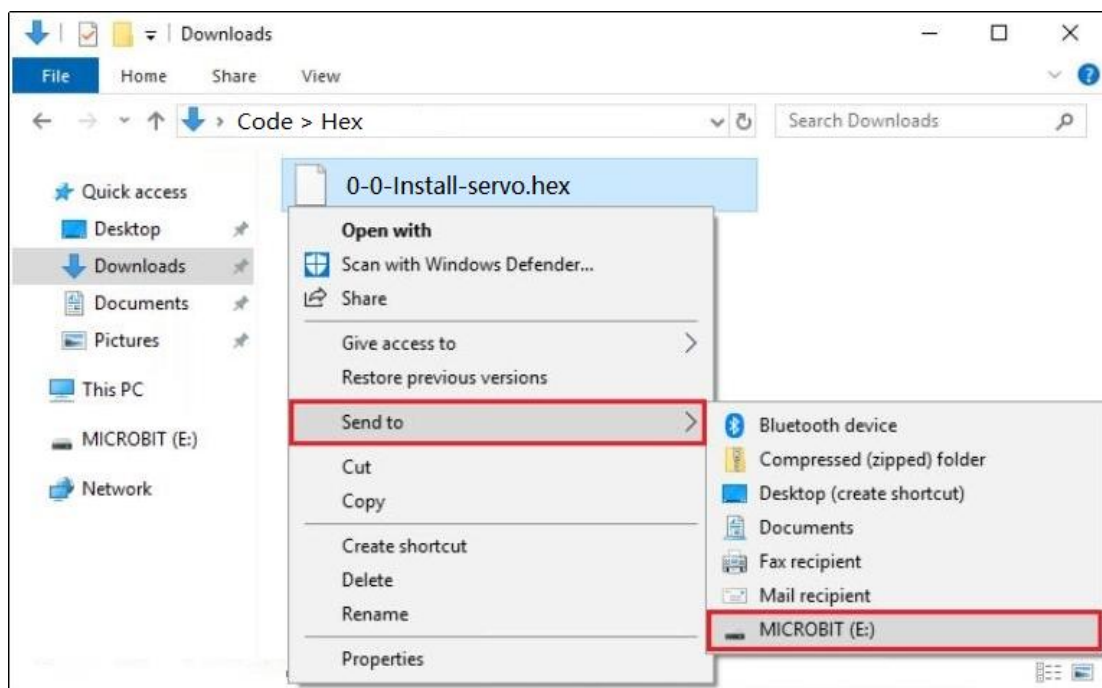
方法 1（ドラッグ&ドロップ）:

HEX ファイルを MICROBIT ドライブにドラッグ&ドロップするだけです。



方法 2 (「送る」):

HEX ファイルを右クリックし、「送る」→「**MICROBIT ドライブ**」を選択します。

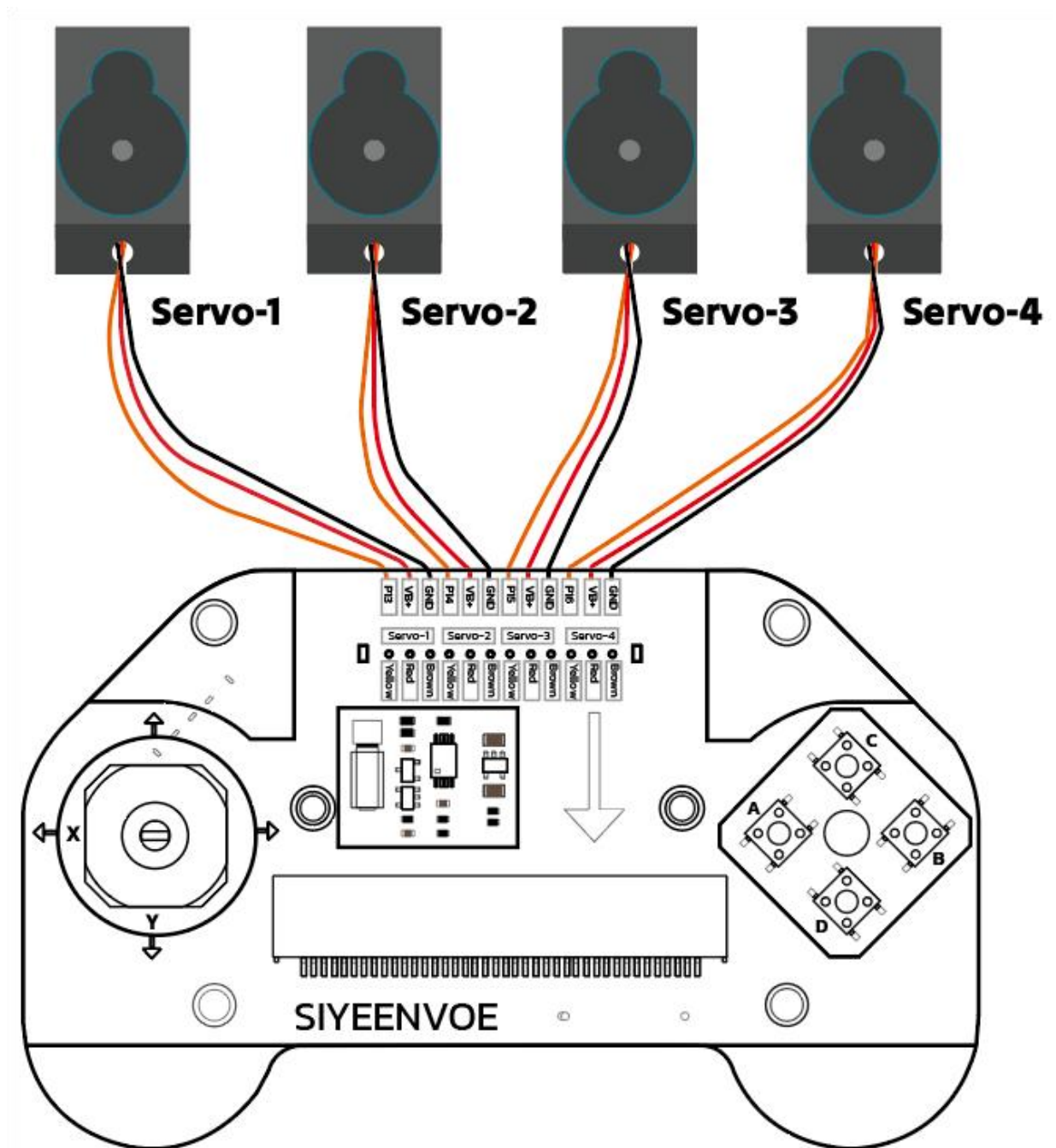


注意:

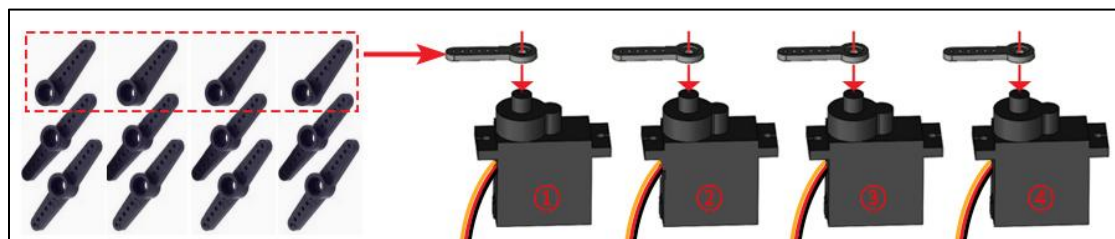
アップロードする前に、**Micro:bit** が正しく接続されていることを確認してください。
ファームウェアの書き込み中は、デバイスを切断しないでください。

3. サーボをジョイスティックに接続する:

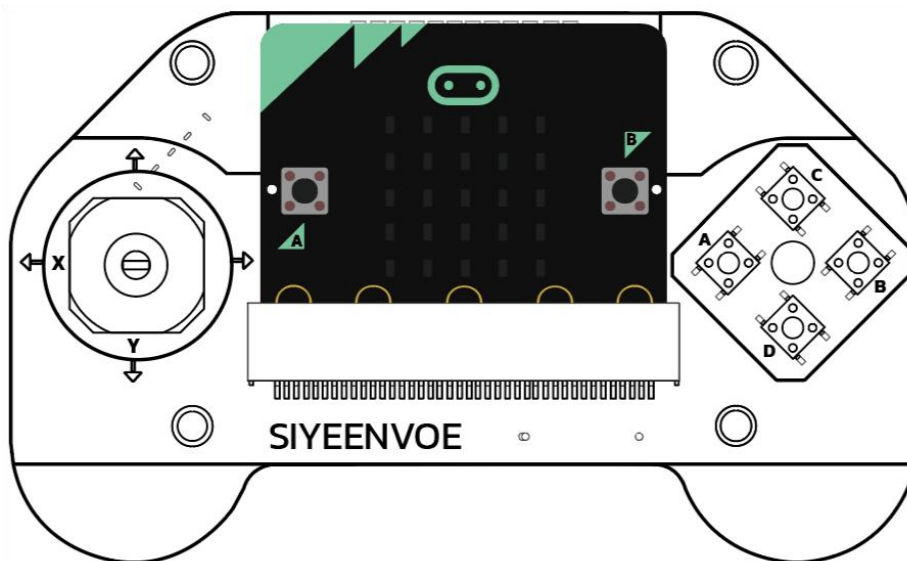
サーボ-1			サーボ-2			サーボ-3			サーボ-4		
P13	VB+	GND	P14	VB+	GND	P15	VB+	GND	P16	VB+	GND



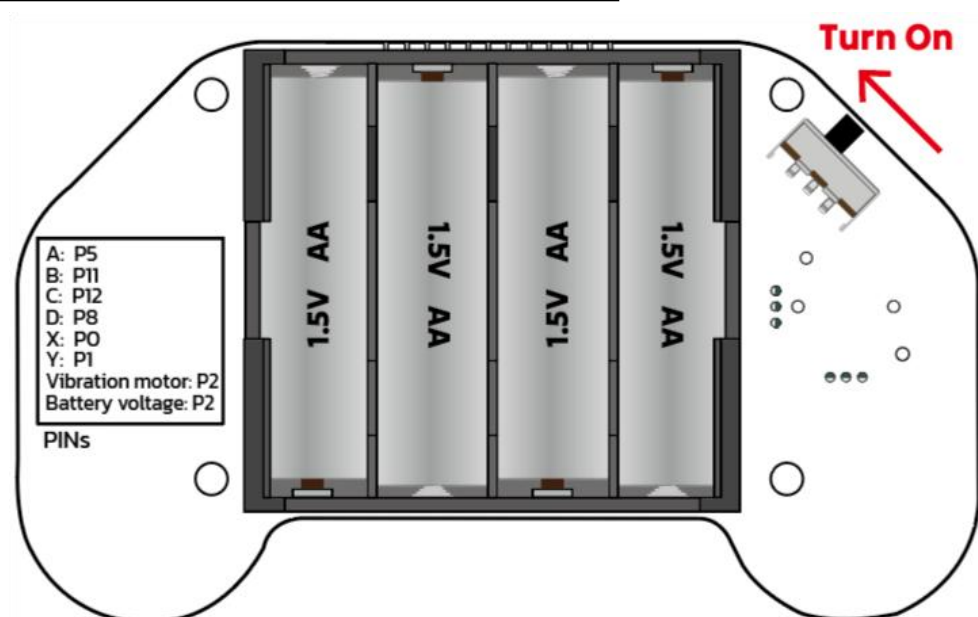
4. サーボシャフトの回転や初期位置での停止状態を容易に確認できるよう、この作業中はネジで固定せずに、一時的なサーボホーン（アーム）をサーボシャフトに取り付けることをお勧めします。



5. micro:bit をジョイスティックに挿入します



6. 単 3 形電池を 4 本挿入し、電源スイッチを入れます



7. 結果:

1) **バッテリー残量表示:**

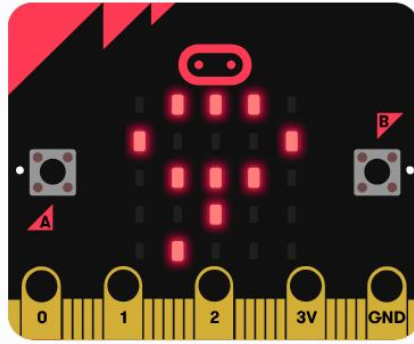
電源投入後、micro:bit の LED マトリックスはバッテリー残量 (0%~100%) を表示し続けます。

重要: バッテリー残量が **30%**を下回った場合は、直ちにすべての電池を交換してください。

バッテリー残量が表示されない場合:

Micro:bit が mJoystick に完全に差し込まれているか確認してください

電池が正しく挿入されているか確認してください



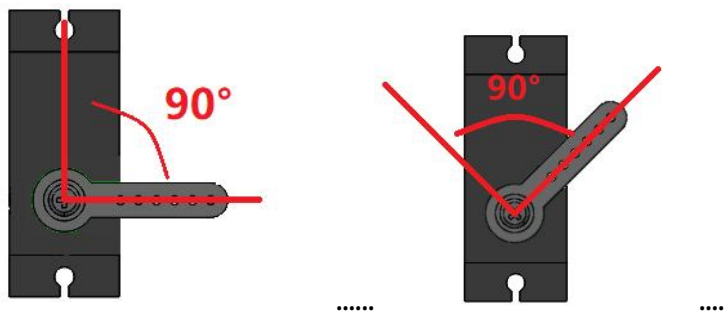
2) サーボの初期化手順:

初回電源投入時:

5 秒ほど待つと、4 つのサーボすべてがランダムな位置から→0°→ゆっくりと 90°まで回転し（90°で停止します）

それ以降の電源投入時:

5 秒間待つと、すべてのサーボが 90° → 0° → ゆっくりと 90°に戻ります（その位置で保持）



サーボが回転しない場合:

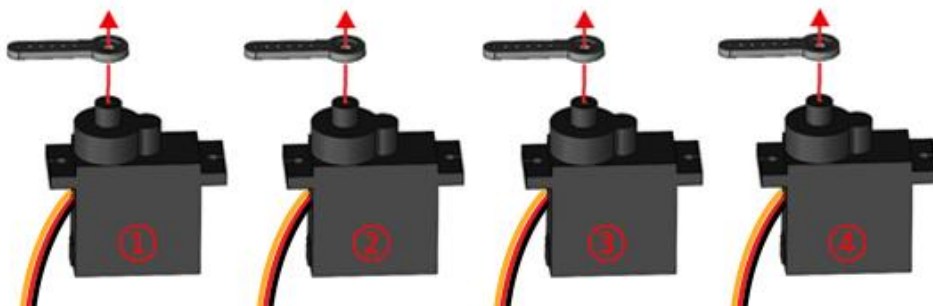
サーボの配線接続を確認してください

配線に問題がないにもかかわらずサーボが反応しない場合は、コードを再アップロードしてください

Micro:bit のエッジコネクタを清掃し、しっかりと差し直してください

問題が解決しない場合はサポートにお問い合わせください（サーボの故障の可能性あります）

8. サーボ角度の初期化が完了したら、後続の取り付け作業のために各サーボからサーボホーンを慎重に取り外してください。ギアトレインの破損やキャリブレーション済みの位置がずれるのを防ぐため、サーボシャフトを回転させないでください。

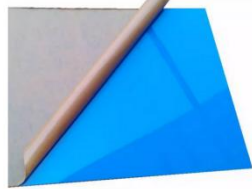


2.2 ロボット組み立て前の注意事項

1) 組み立て前に保護フィルムを剥がしてください！ 上面と下面の両方に保護フィルムが貼られています



また



2) 部品を間違えたり混同したりしないよう、一度にすべて袋から出さないでください。小さな部品が混ざらないよう、マニュアルの指示に従って部品袋を順番に開けてください。予備としてネジが含まれている場合があります。

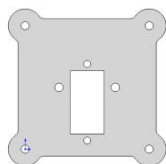
3) 図面と同じ向きで組み立ててください。部品の色は異なる場合があります。

4) 部品に欠品や破損がある場合は、組み立てや使用をしないでください。アフターサービスについてお問い合わせください。

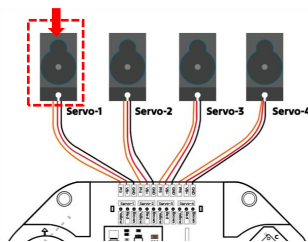
2.3 組み立て開始

ステップ 1 - サーボ 1 の取り付け

アクリル板 1
枚



サーボ-1



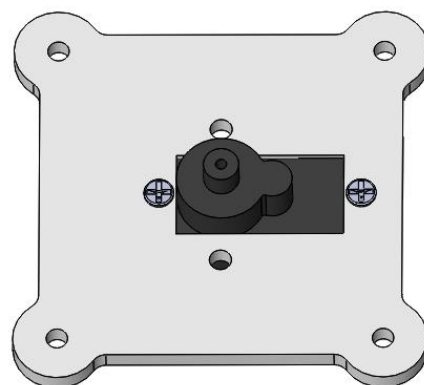
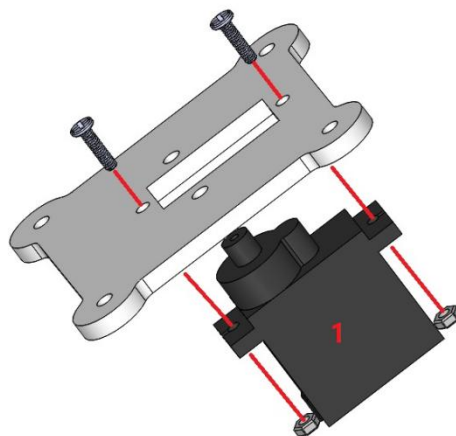
袋 No.③

M2x8mm ネジ 2 本

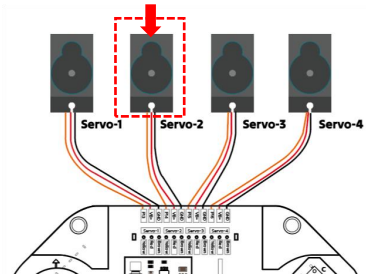
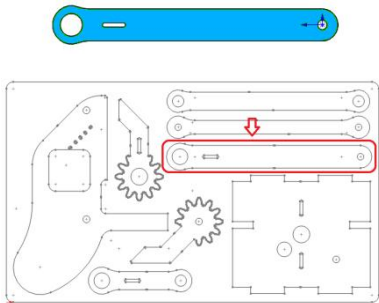
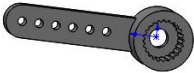
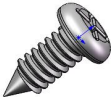


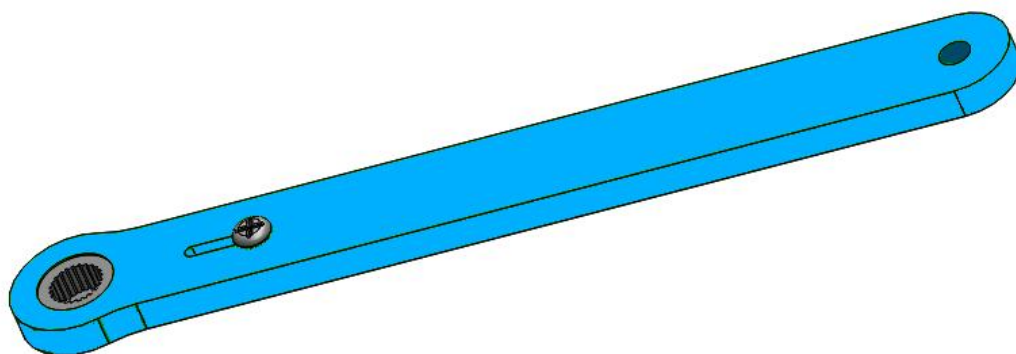
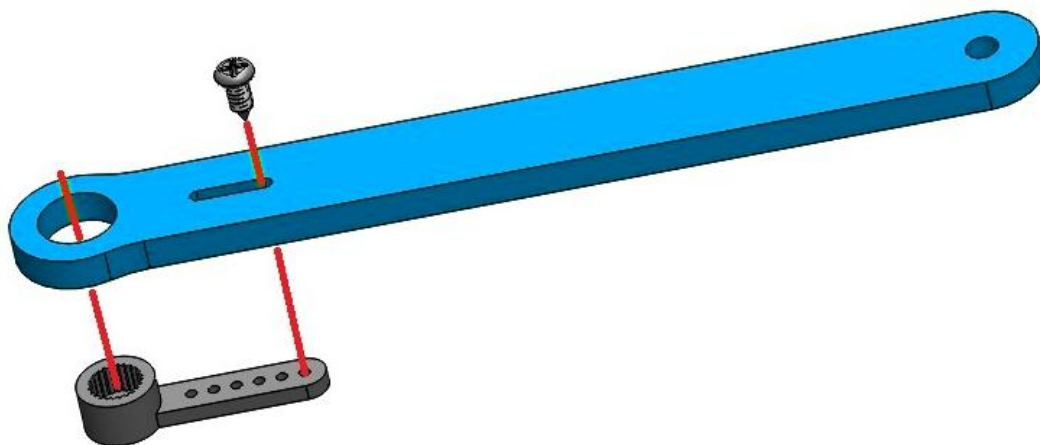
袋 No.⑥

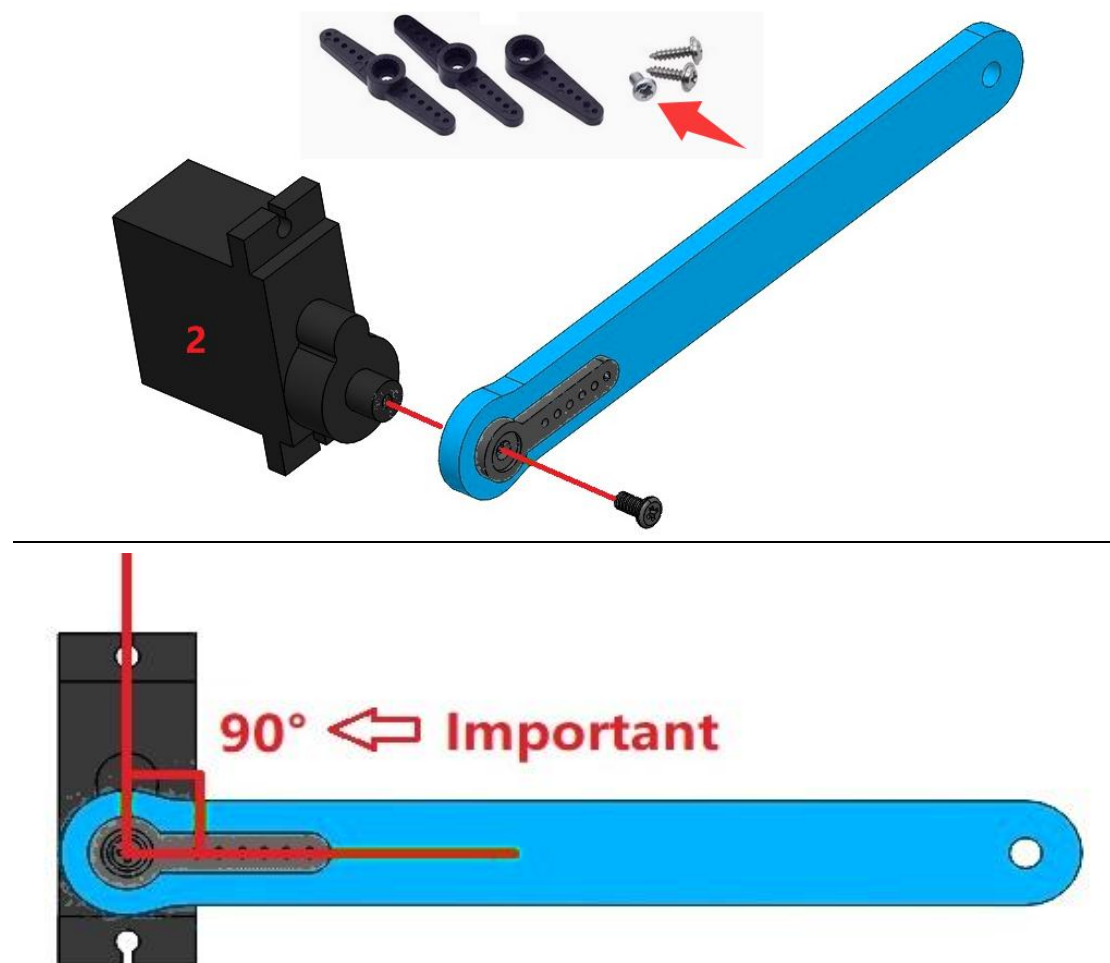
M2 ナット 2 個



ステップ 2 - サーボ-2 の取り付け

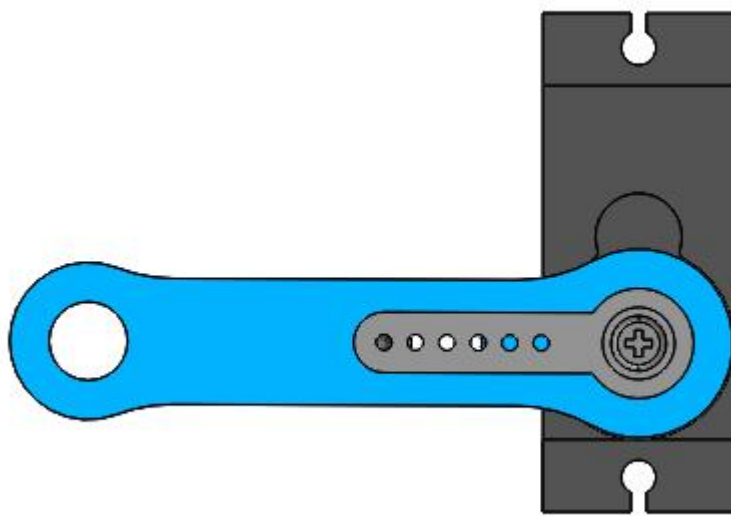
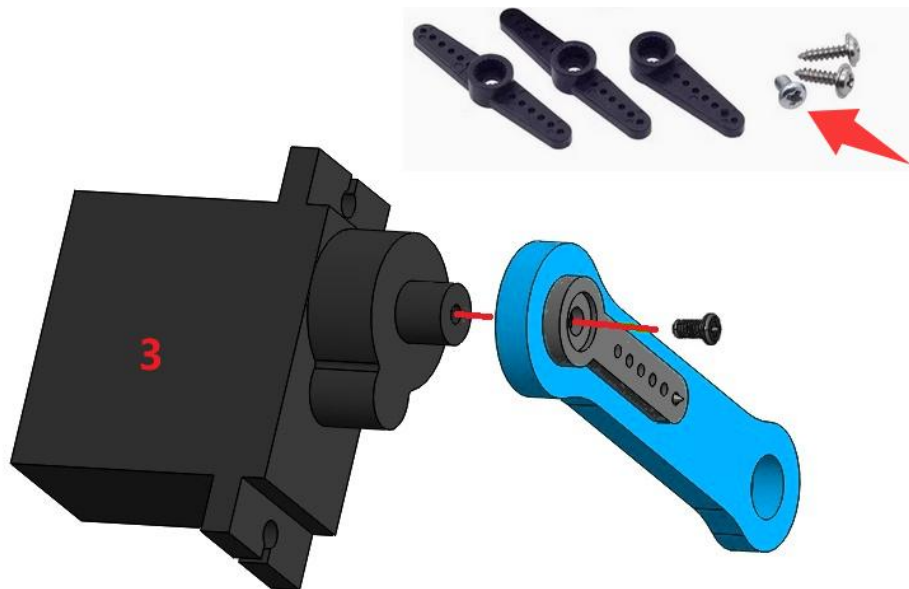
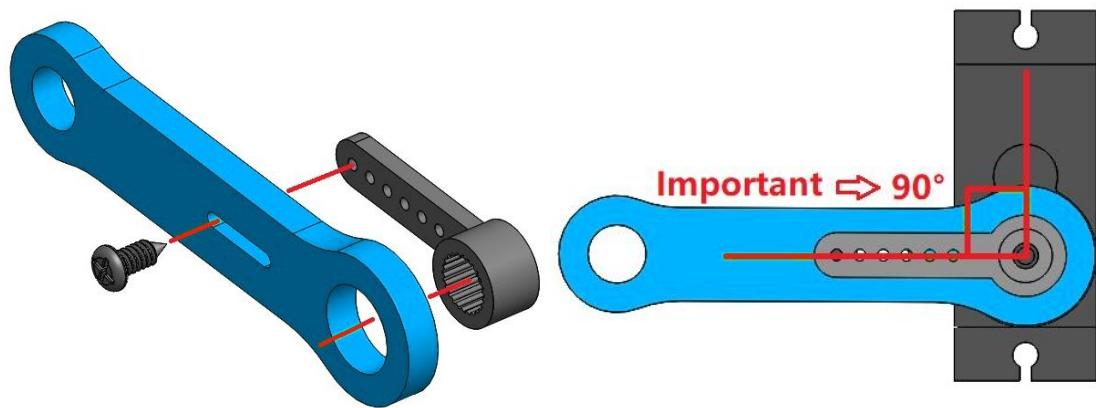
サーボ-2		アクリル板 1 枚	
			
サーボホーン 1 個	袋 No.④ M1.4x5 セルフタッピングネジ 1 本		M2.5x4mm ネジ 1 本
			



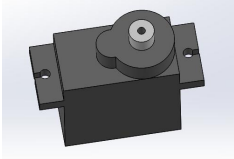
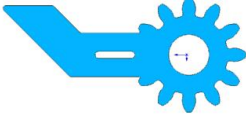
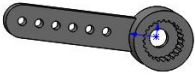
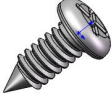


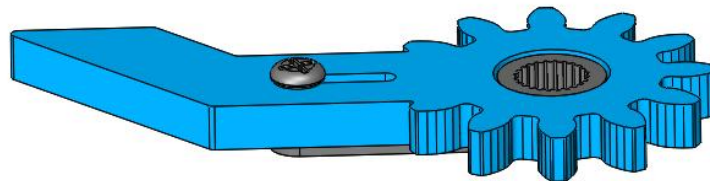
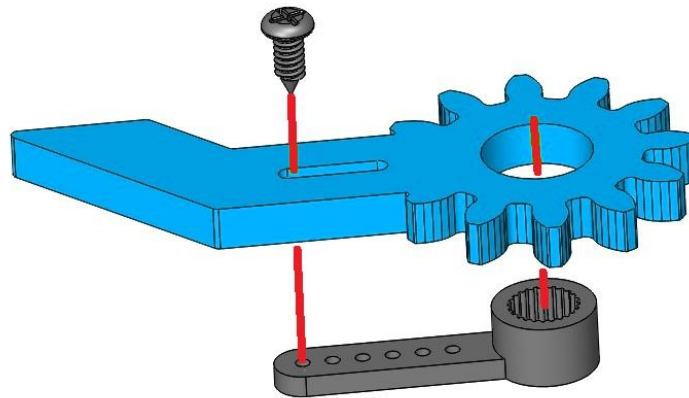
ステップ 3 - サーボ-3 の取り付け

サーボ-3		アクリル板 1 枚	
サーボホーン 1 個	袋 No.④ M1.4×5 セルフタッピングネジ 1 本	M2.5×4mm ネジ 1 本	



ステップ 4 - サーボ-4 の取り付け

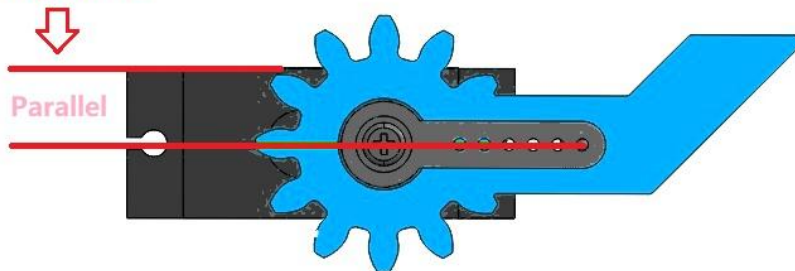
Servo-4		アクリル板 1 枚	
			
サーボホーン 1 個	袋 No.④ M1.4×5 セルフタッピングネジ 1 本		M2.5×4mm ネジ 1 本
			

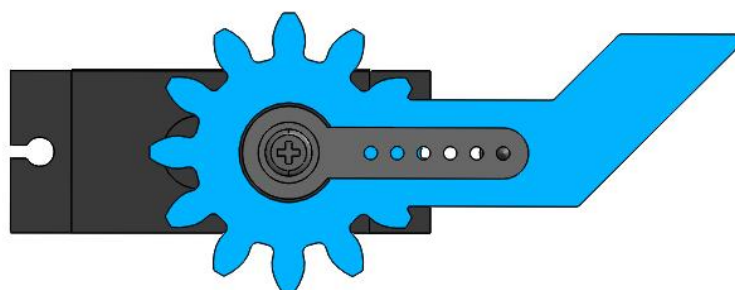


Important



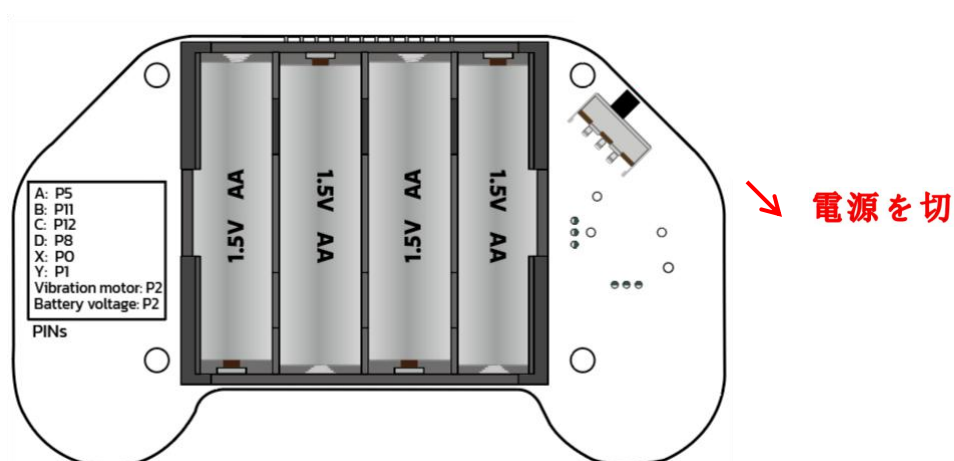
Parallel



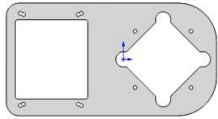


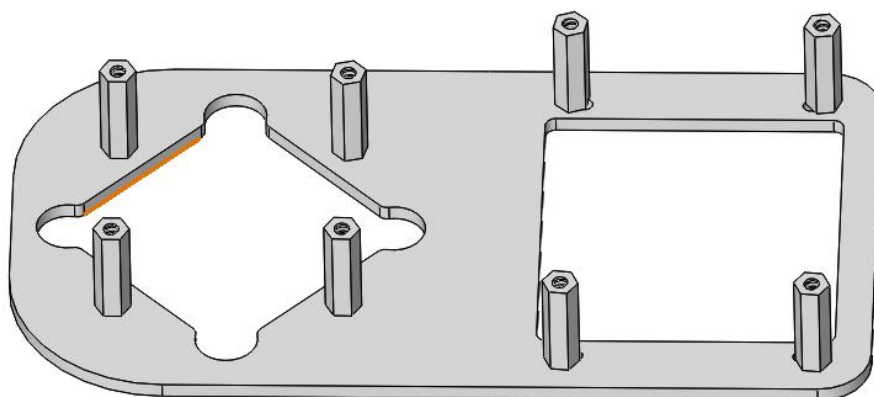
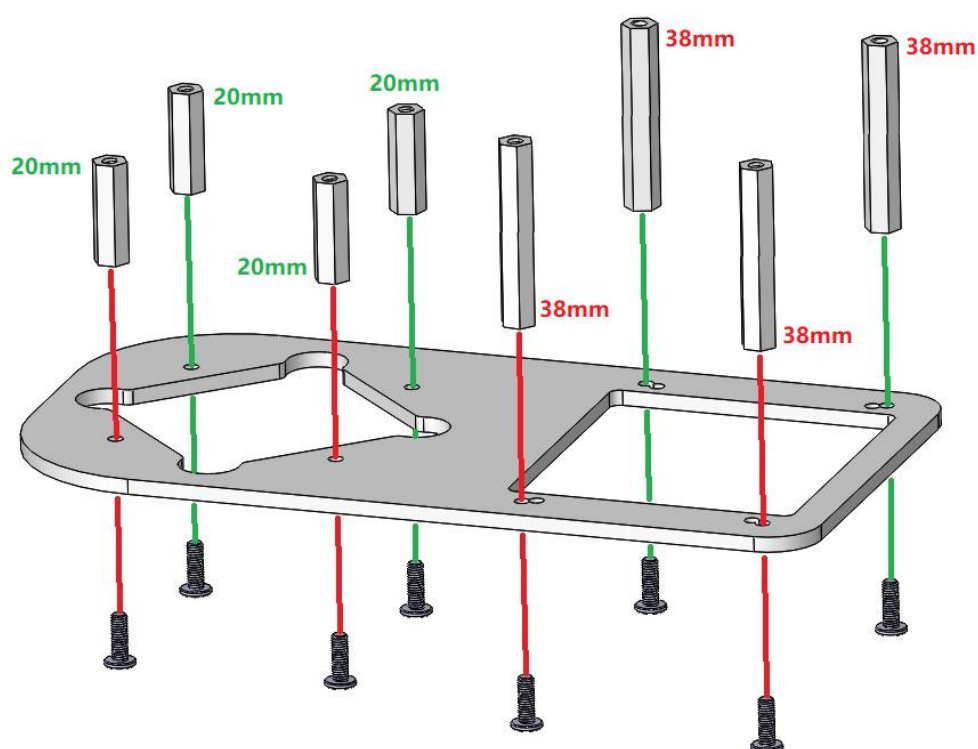
ステップ 5 -ジョイスティックの電源を切り、サーボケーブルを抜く

次の取り付け手順のため、ジョイスティックの電源を切り、すべてのサーボケーブルをポートから抜いてください。

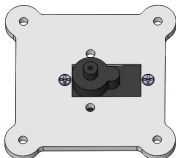
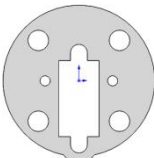
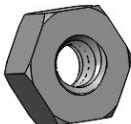


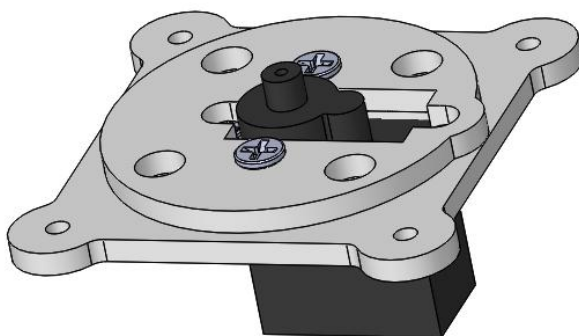
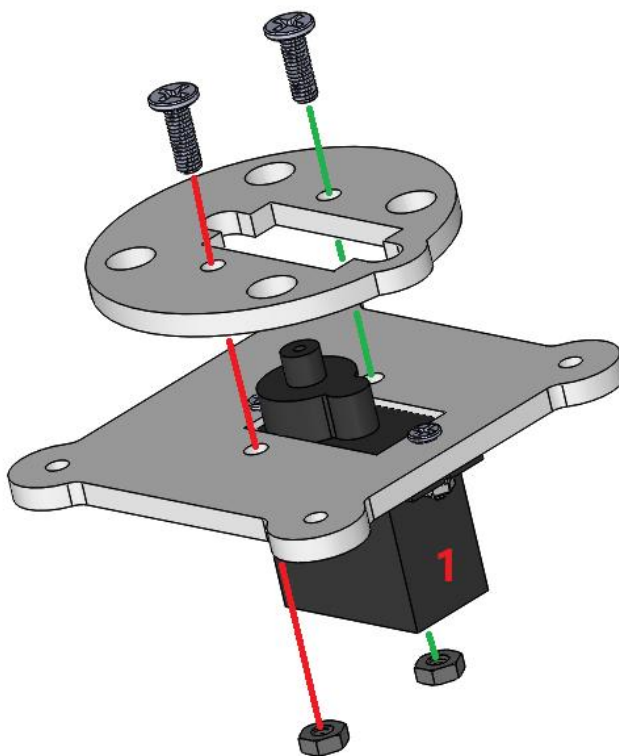
ステップ 6 - ベースの取り付け（ナイロン製スタンドオフ

アクリル板	袋 No.⑪ M3x20mm ナイロン スタンドオフ 4 個	袋 No.⑫ M3x38mm ナイロ ンスタンドオフ 4 個	袋 No.② M3x9mm ネジ 8 本
			

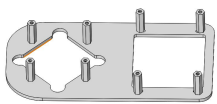
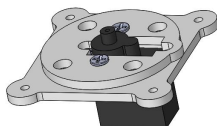


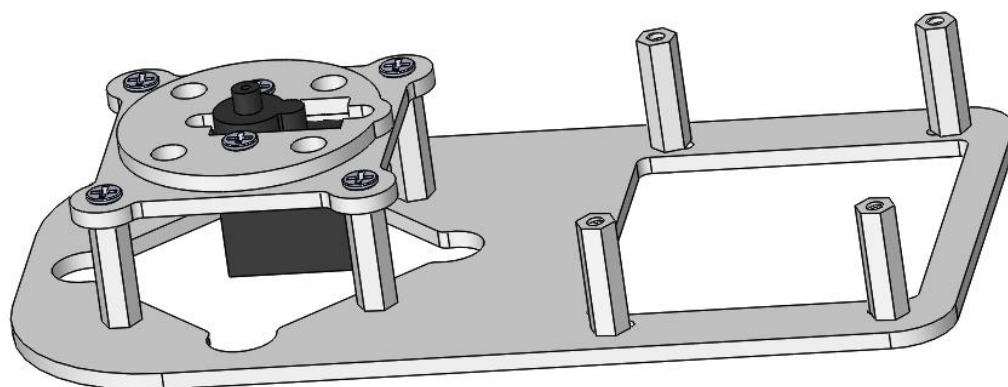
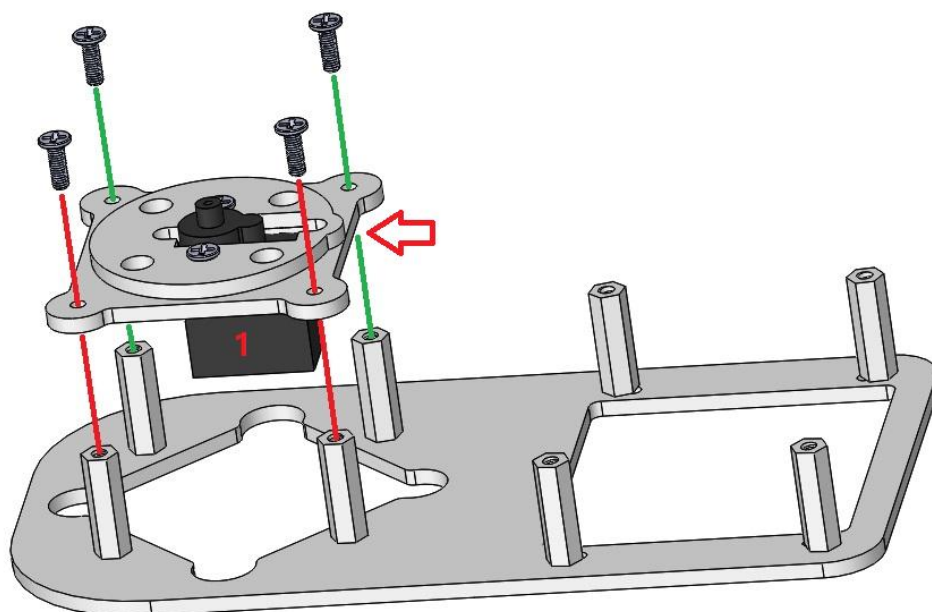
ステップ 7- ベース（スチール製ボールベアリングプレート）の取り付け

ステップ 1 の構造	アクリル板	袋 No.② M3×9mm ネジ 2 本	袋 No.⑤ M3 ナット 2 個
			



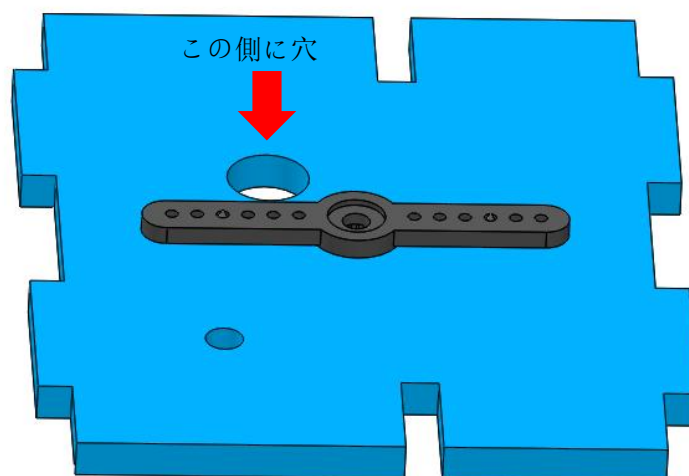
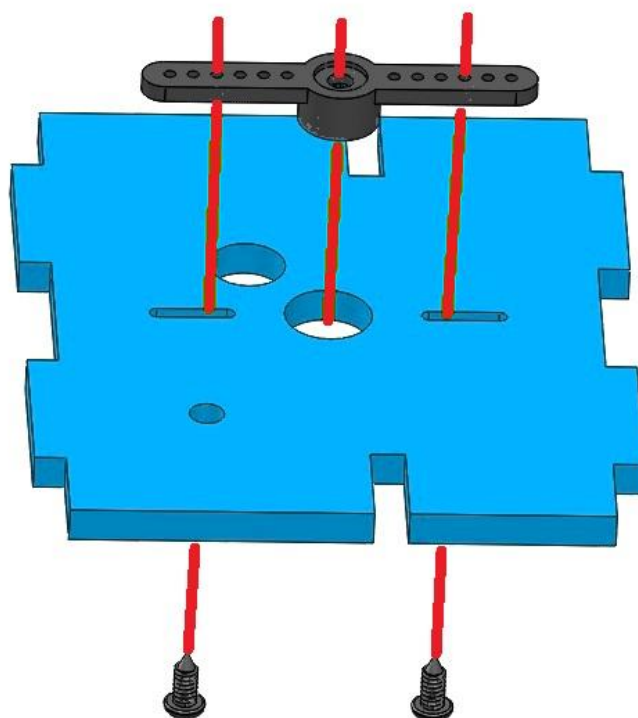
ステップ 8 - ベースの取り付け（サーボ構造）

ステップ 6 の構造	ステップ 7 の構造	袋 No.② M3×9mm ネジ 4 本
		

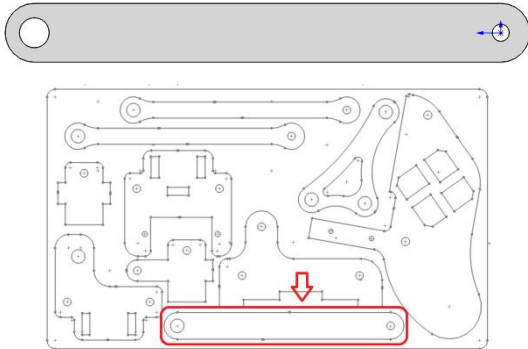
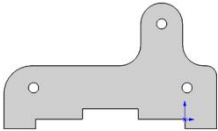


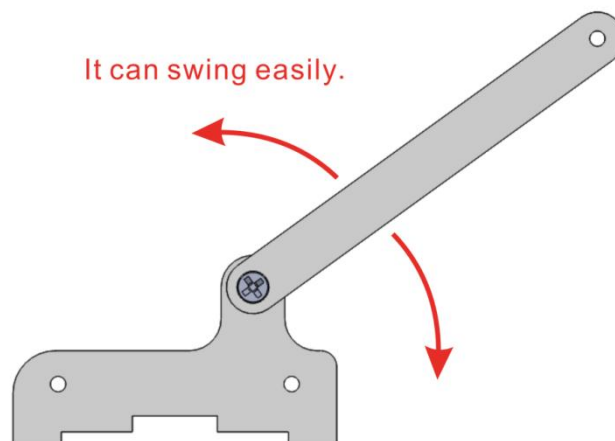
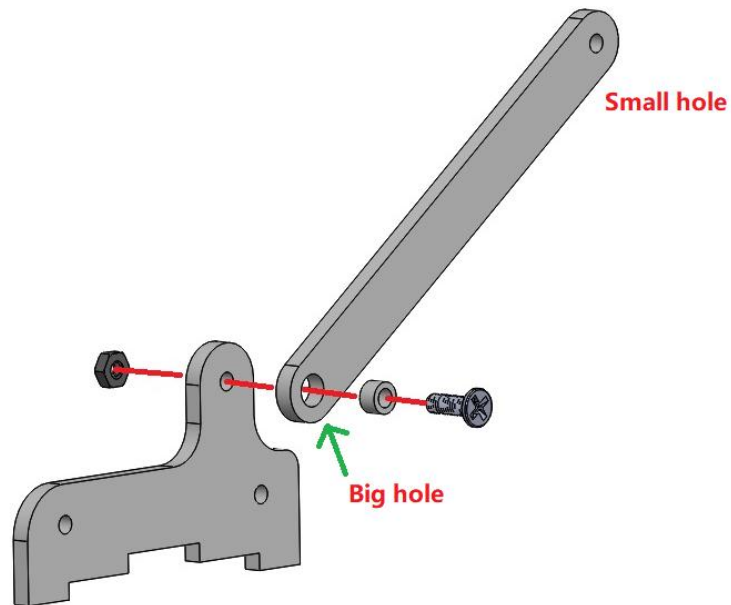
ステップ 9 - サーボホーンを回転ベースに取り付ける

アクリル板	サーボホーン 1 個	袋 No.④ M1.4×5 セルフタッピングネジ 2 本
		

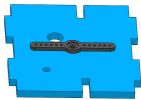
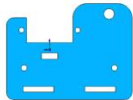
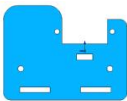


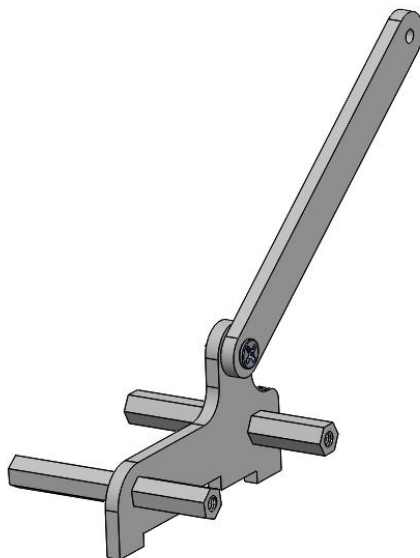
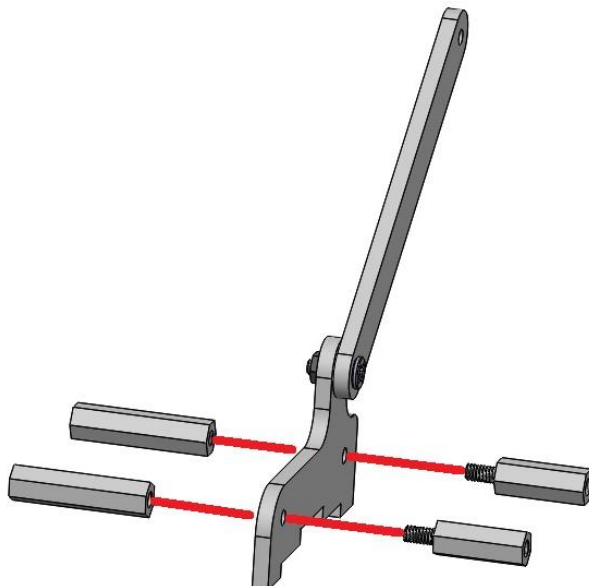
ステップ 10 - アッパーアームリンケージの取り付け

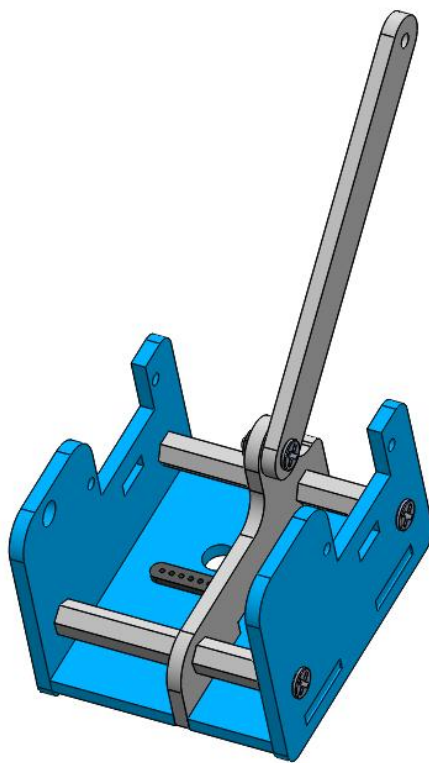
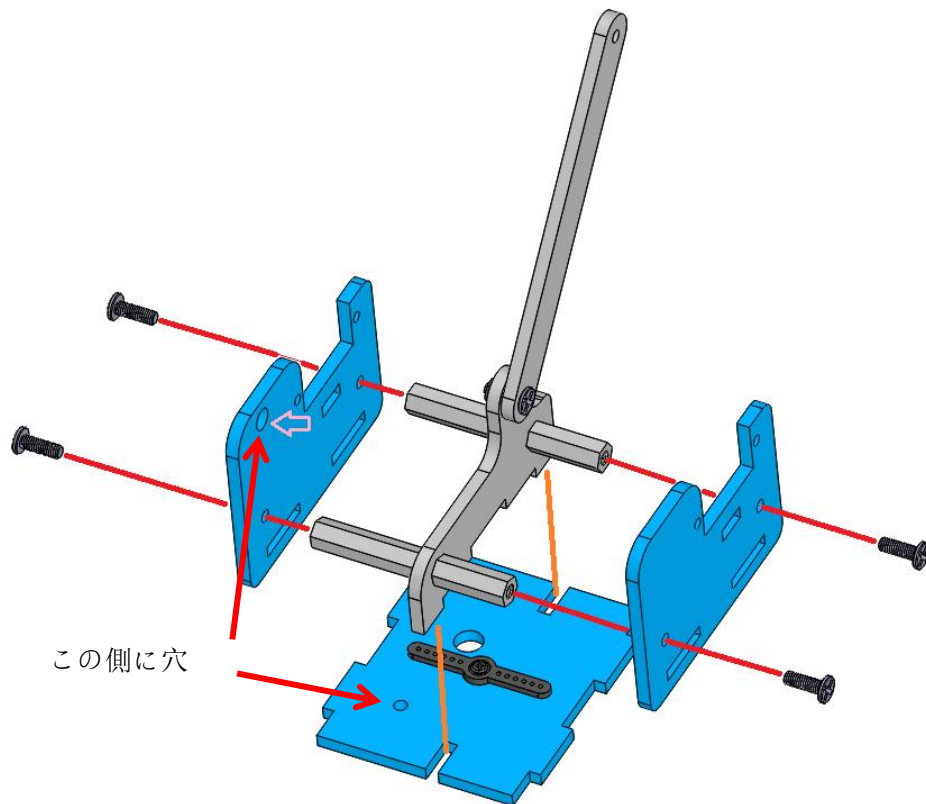
アクリル板		アクリル板
		
袋 No.⑬ 5×3.2×3mm スペーサー 1 個	袋 No.④ M3×9mm ネジ 1 本	袋 No.⑤ M3 ナット 1 個
		



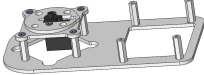
ステップ 11 - 回転ベースの組み立て

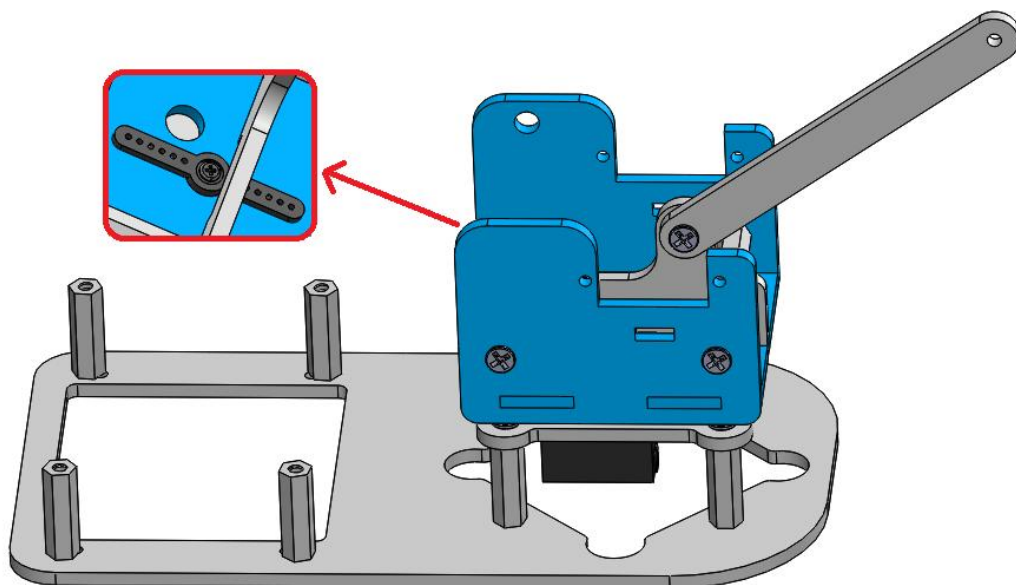
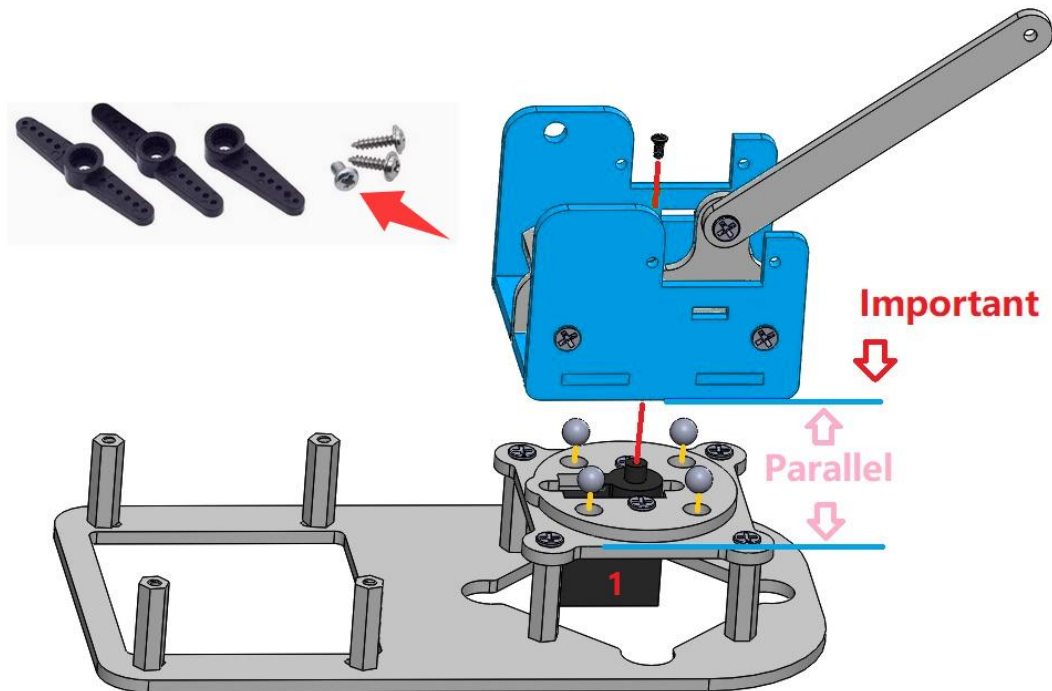
ステップ 9 の構造		ステップ 10 の構造		アクリル板 1 枚
				
アクリル板 1 枚		袋 No.⑧ M3x17+6mm ナイ ロンスタンドオフ 2 個	袋 No.⑨ M3x26mm ナイロ ンスタンドオフ 2 個	袋 No.② M3x9mm ネジ 4 本
				



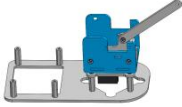


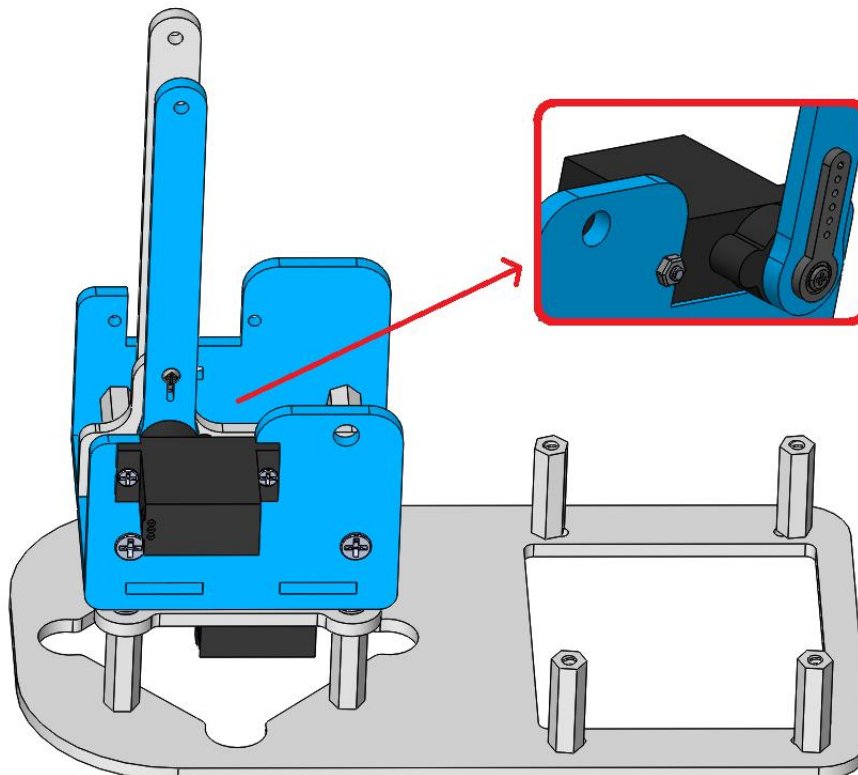
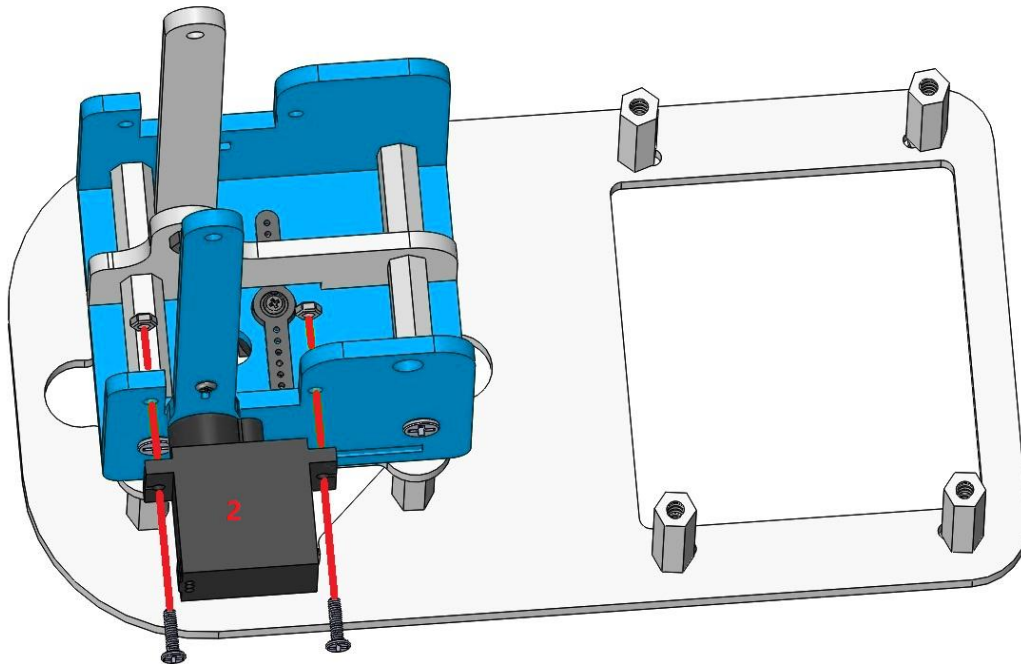
ステップ 12 - 回転ベースをメインフレームに取り付ける

ステップ 8 の構造	ステップ 11 の構造	スチール板 4 枚	M2.5×4mm ネジ 1 本
			

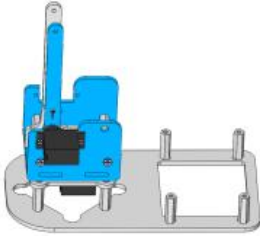
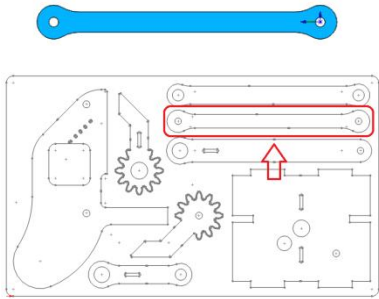
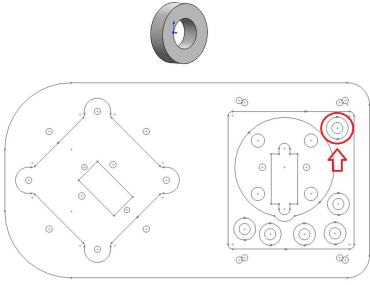


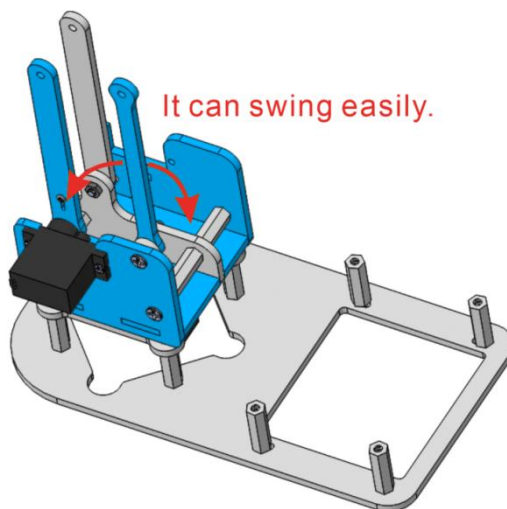
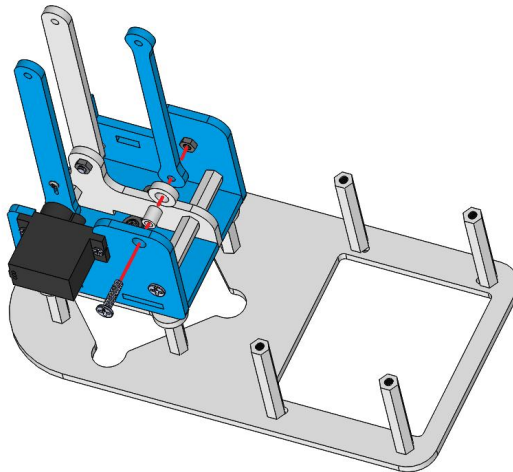
ステップ 13 - ターンテーブルに左サーボ構造を取り付ける

ステップ 2 の構造	ステップ 12 の構造	袋 No.③ M2x8mm ネジ 2 本	袋 No.⑥ M2 ナット 2 個
			



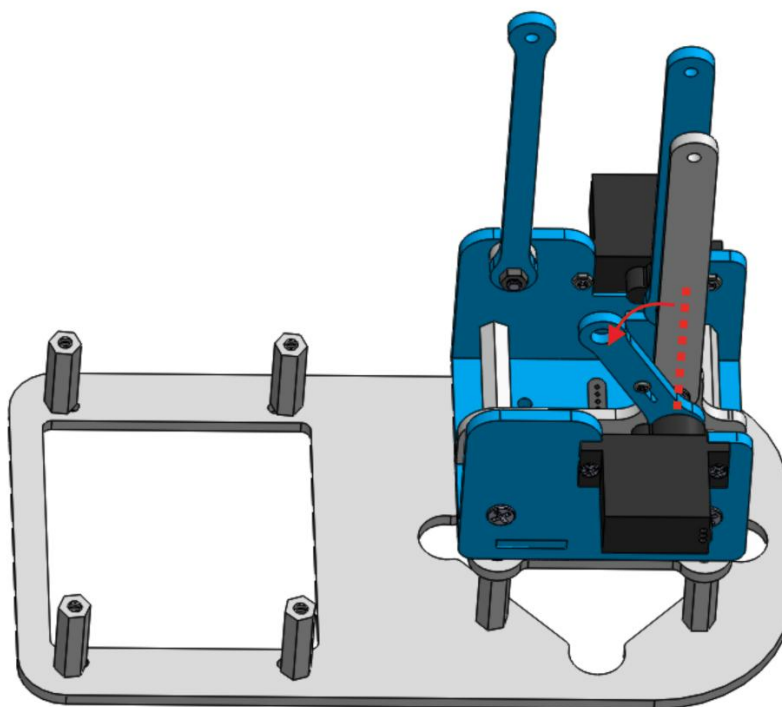
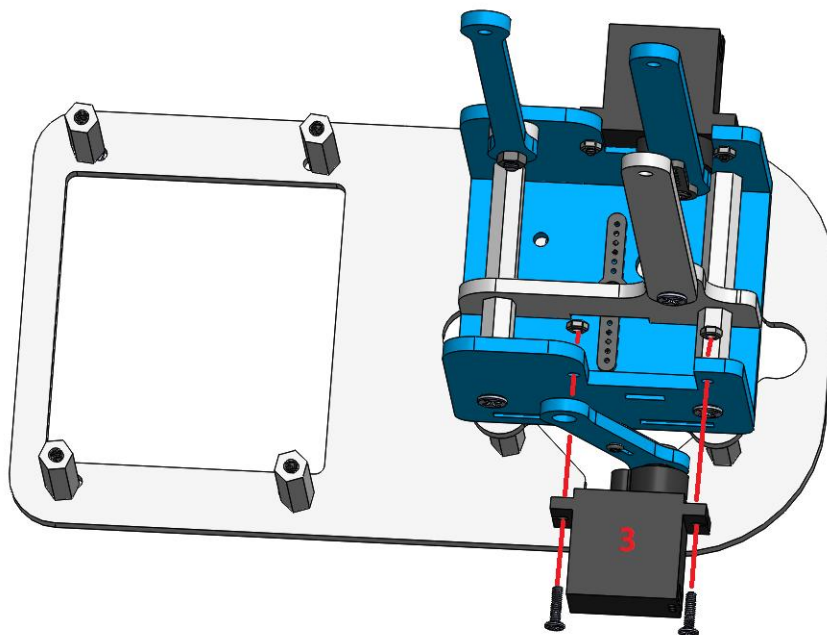
ステップ 14 - 上腕リンクの取り付け（左側）

ステップ 13 の構造	アクリル板 1 枚	アクリル製スペーサー 1 個
		
袋 No.⑭ 5×3.2×6mm スペーサー 1 個	袋 No.① M3×12mm ネジ 1 個	袋 No.⑤ M3 ナット 1 個
		

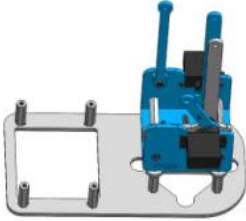
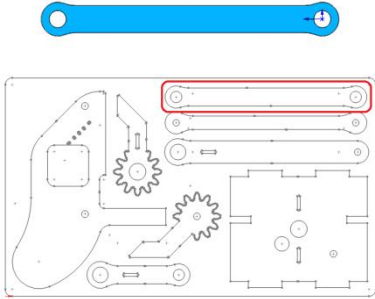
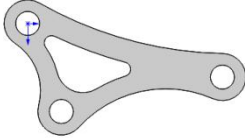


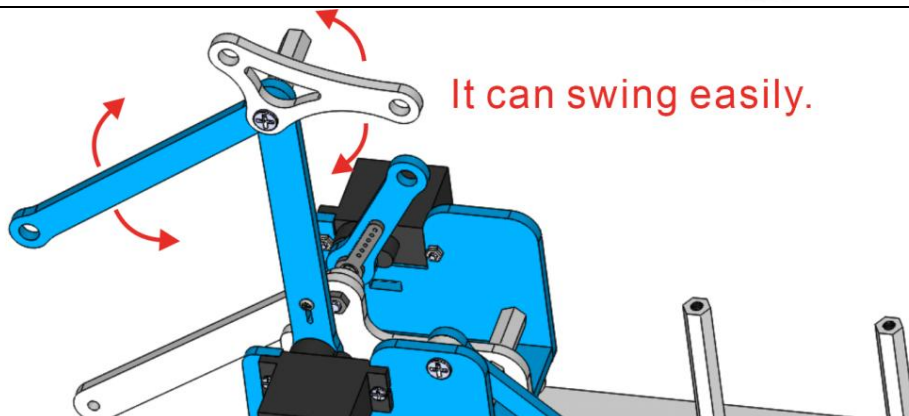
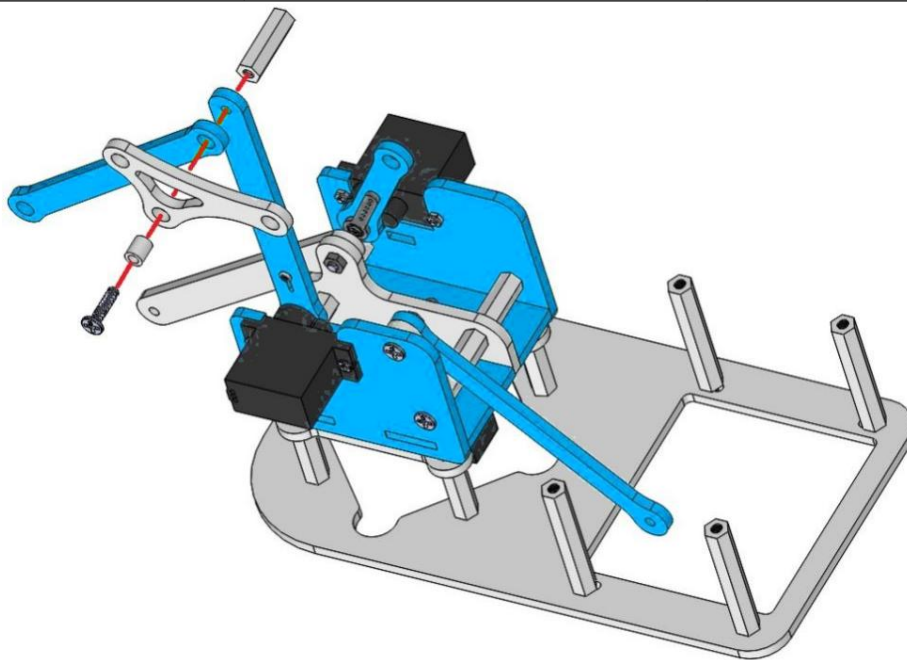
ステップ 15 - ターンテーブルに右サーボ構造を取り付ける

ステップ 14 の構造	ステップ 3 の構造	袋 No.③ M2x8mm ネジ 2 本	袋 No.⑥ M2 ナット 2 個
			

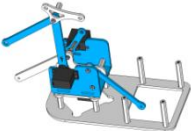


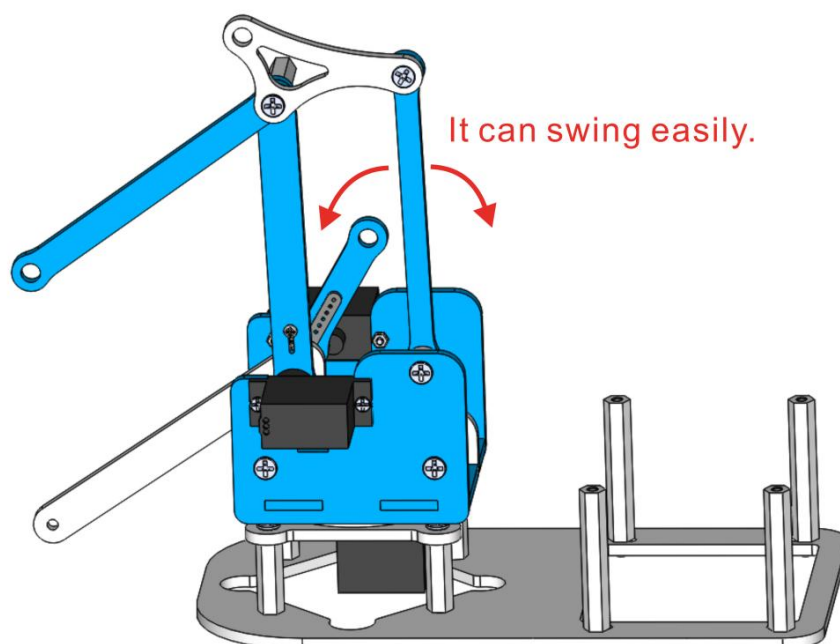
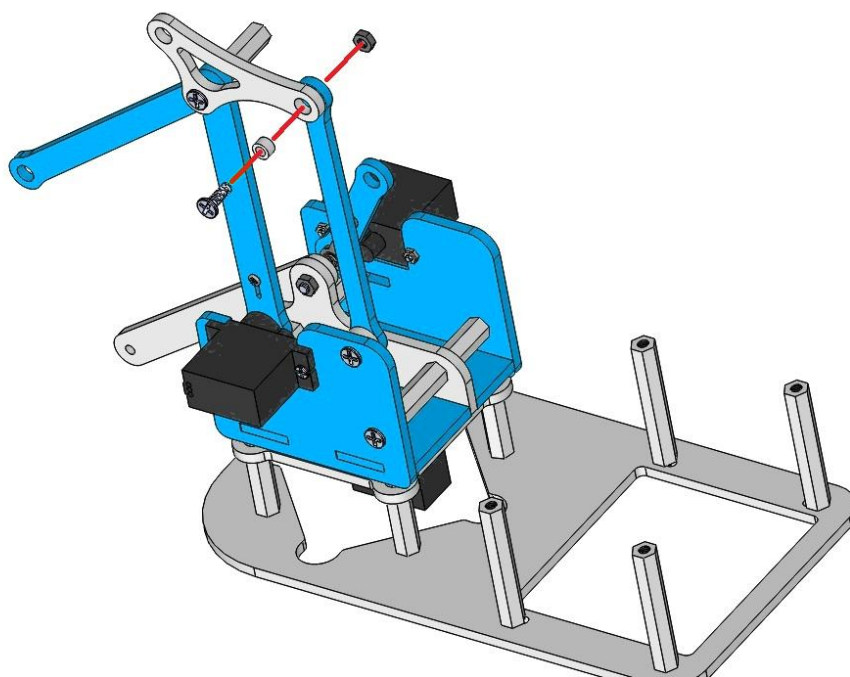
ステップ 16 - プライマリアーム / エルボコネクタの取り付け(1)

ステップ 15 の構造	アクリル板 1 枚	アクリル板 1 枚
		
袋 No.⑭ 5×3.2×6mm スペーサー 1 個	袋 No.⑪ M3×20mm ナイロンスタンドオフ 1 個	袋 No.① M3×12mm ネジ 1 個
		

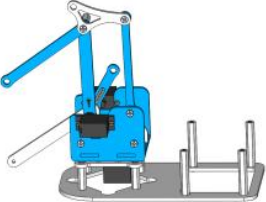
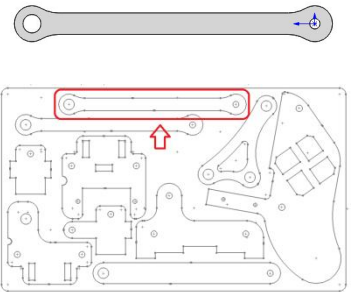
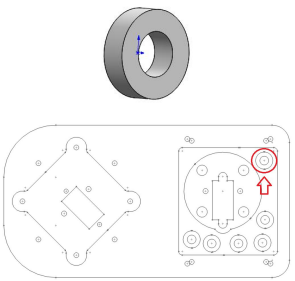
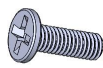


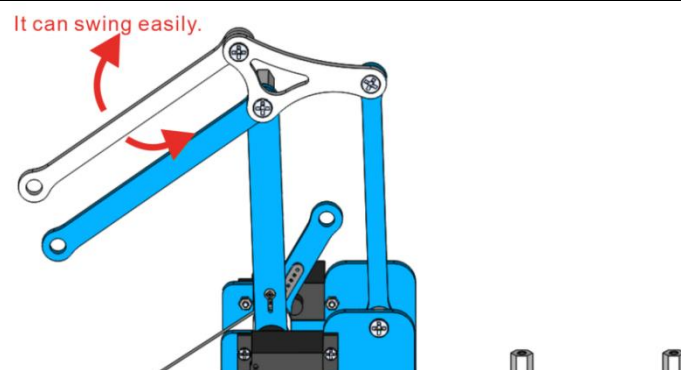
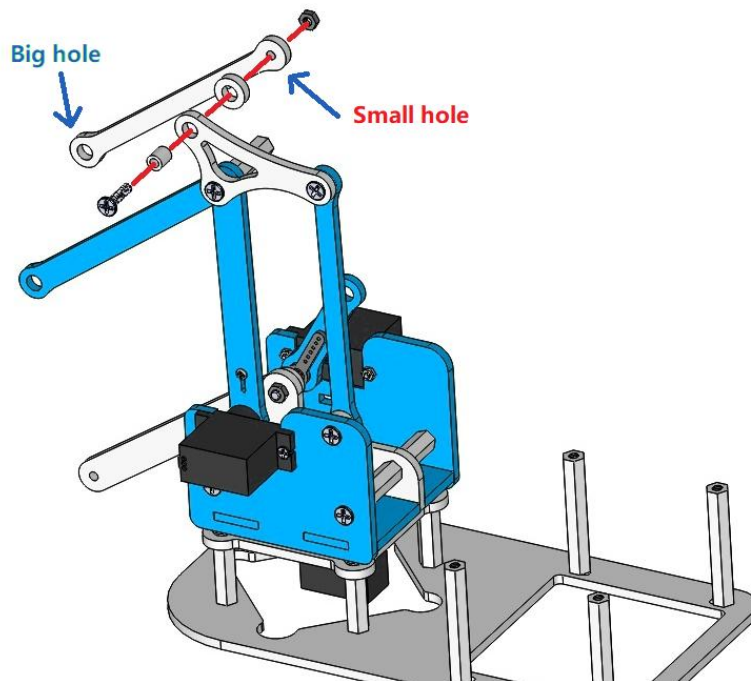
ステップ 17 - プライマリアーム / エルボコネクタ(2)の取り付け

ステップ 16 の構造	袋 No.⑬ 5x3.2x3mm M スペーサー 1 個	袋 No.② M3x9mm ネジ 1 本	袋 No.⑤ M3 ナット 1 個
			

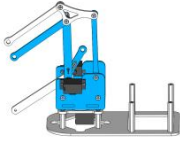


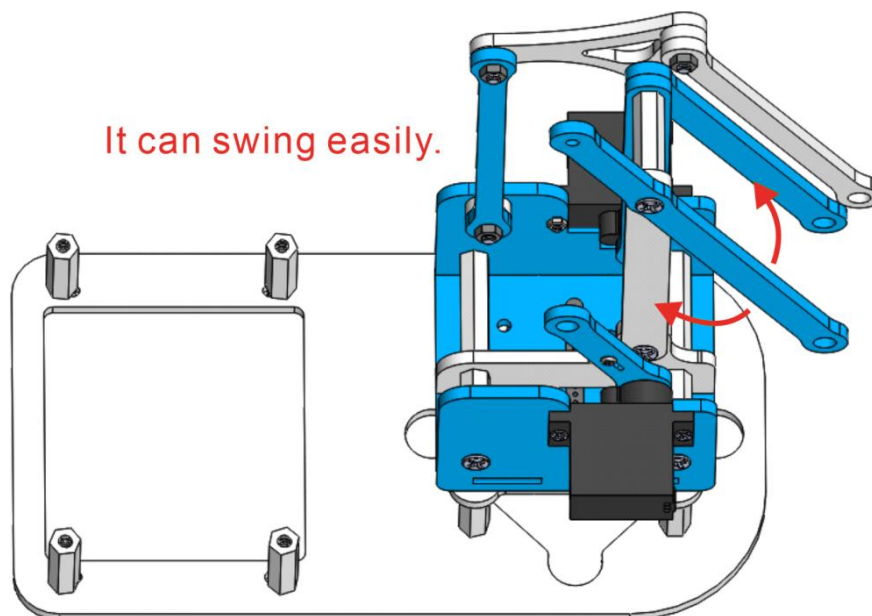
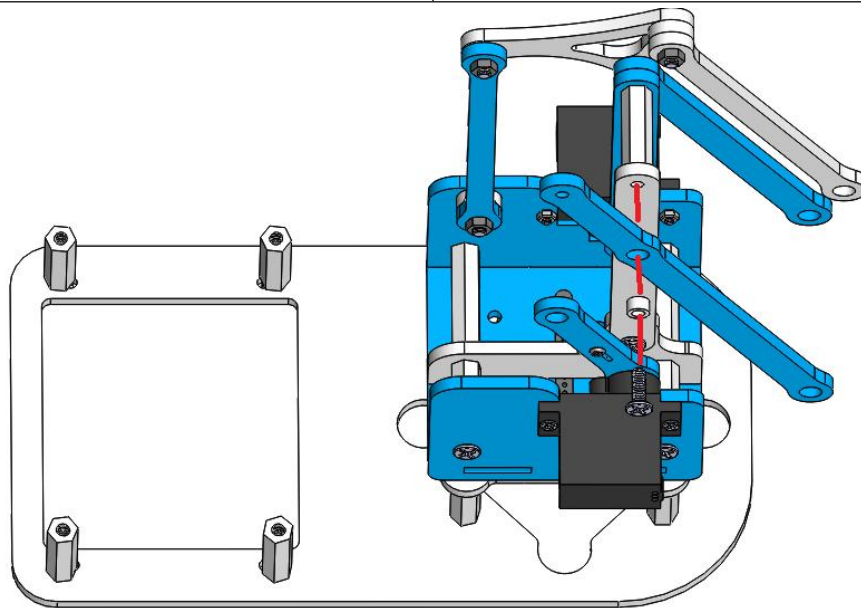
ステップ 18 - 前腕リンケージの取り付け（最初の位置）

ステップ 17 の構造	アクリル板 1 枚	アクリルスペーサー 1 個
		
袋 No.⑭ 5×3.2×6mm スペーサー 1 個	袋 No.① M3×12mm ネジ 1 本	袋番号⑤ M3 ナット 1 個
		

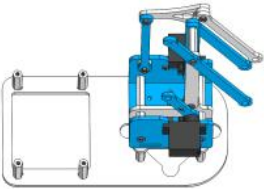
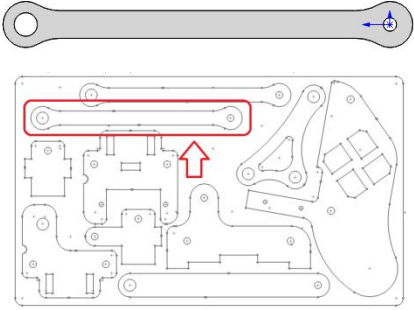


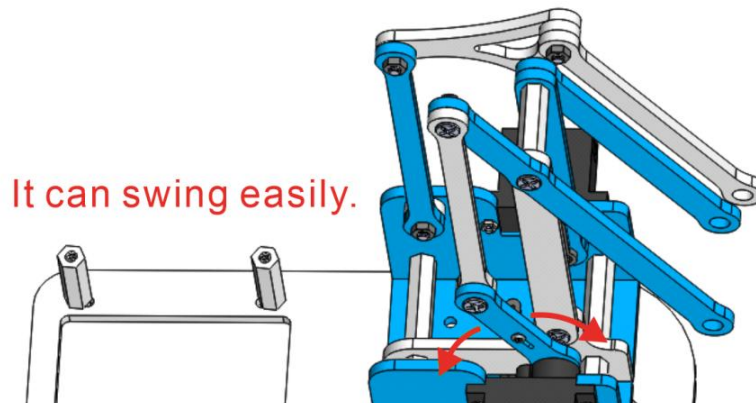
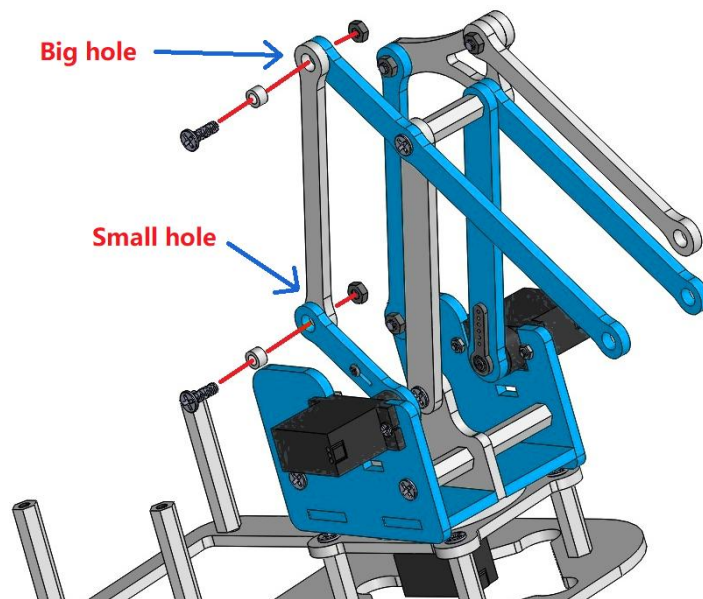
ステップ 19 - 前腕リンケージの取り付け（第 2 位置）

ステップ 18 の構造	アクリル板 1 枚
	
袋 No.⑬ 5×3.2×3mm スペーサー 1 個	袋 No.② M3×9mm ネジ 1 本
	

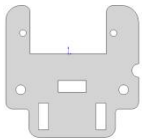


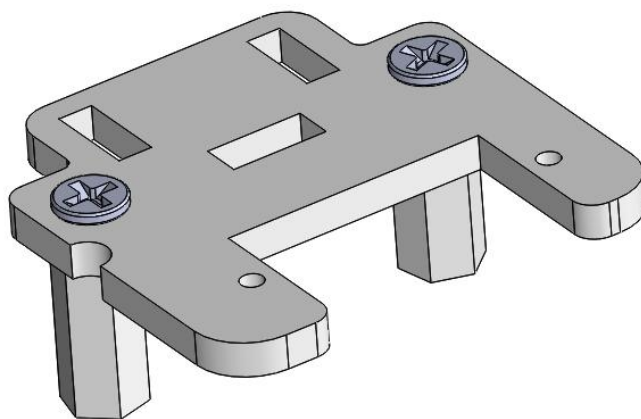
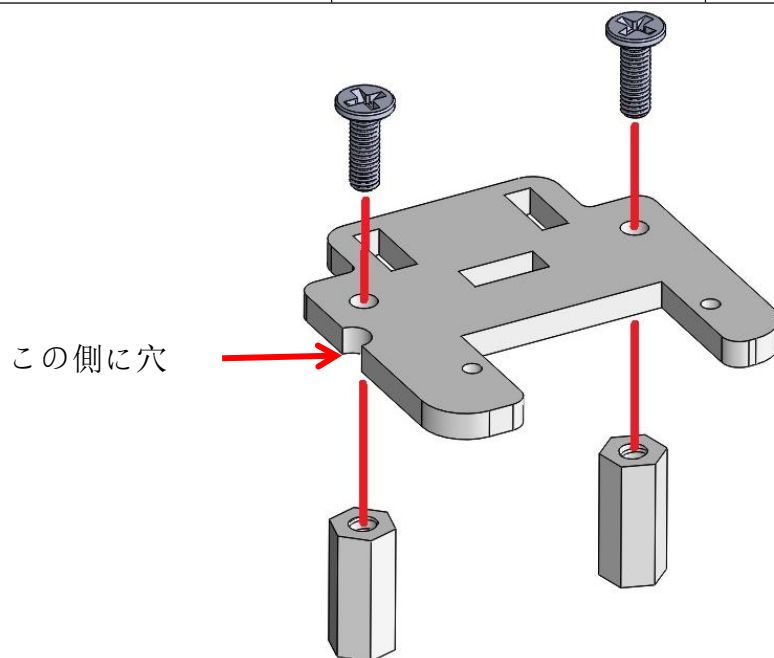
ステップ 20 - 上腕と前腕のリンクを接続する

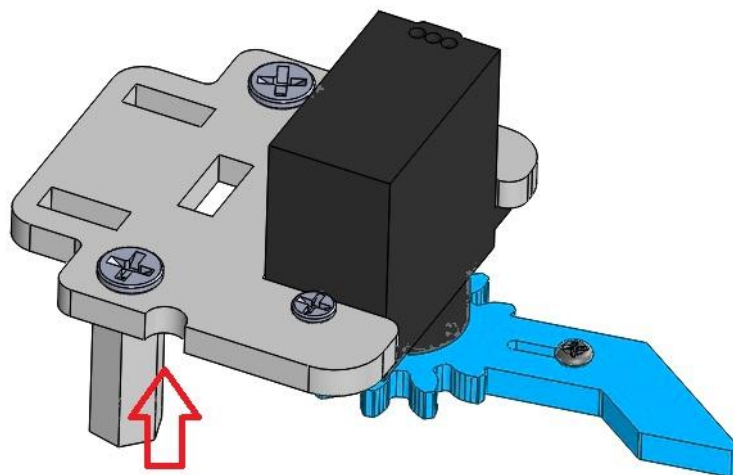
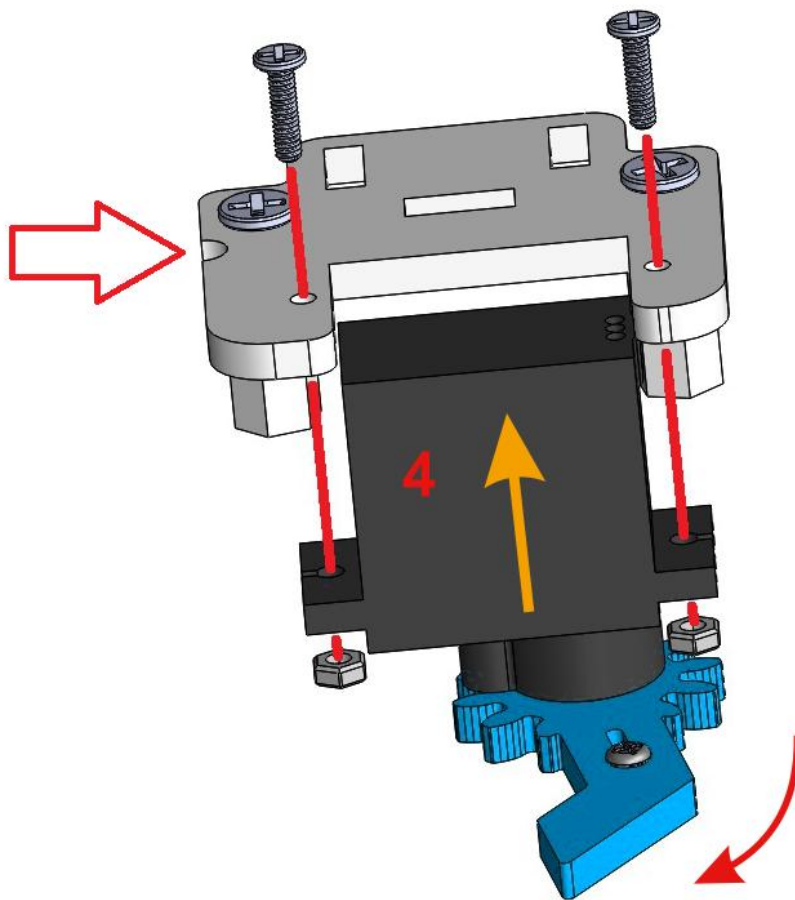
ステップ 19 の構造		アクリル板 1 枚	
			
袋 No.⑬ 5×3.2×3mm スペーサー 2 個		袋 No.② M3×9mm ネジ 2 本	袋 No.⑤ M3 ナット 2 個
			



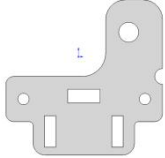
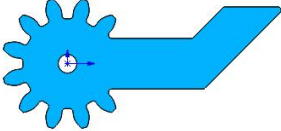
ステップ 21 - 左グリッパージョーを取り付ける

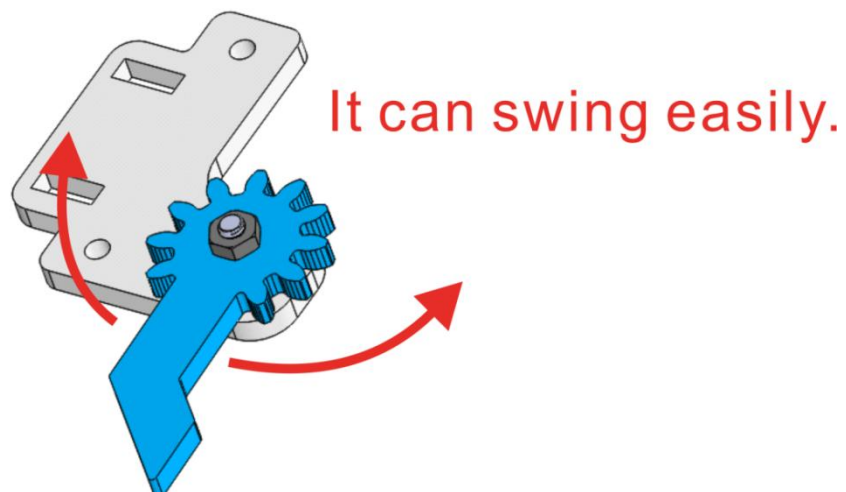
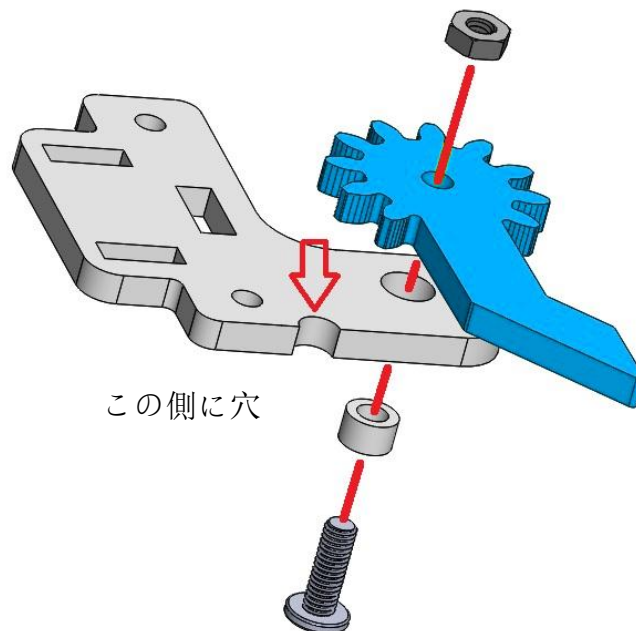
アクリル板 1 枚	袋 No.② M3×9mm ネジ 2 本	袋 No.⑦ M3×14mm ナイロン製ス ペーサー 2 個
		
ステップ 4 の構造	袋 No.③ M2×8mm ネジ 2 本	袋 No.⑥ M2 ナット 2 個
		



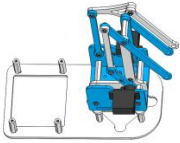
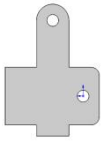
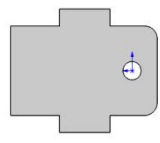


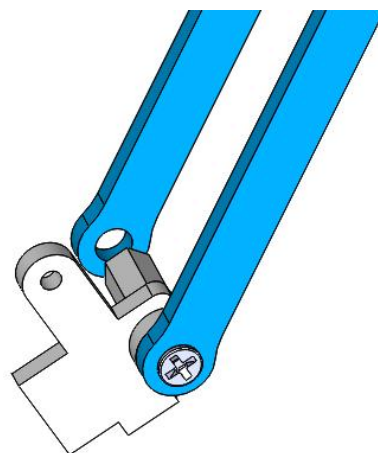
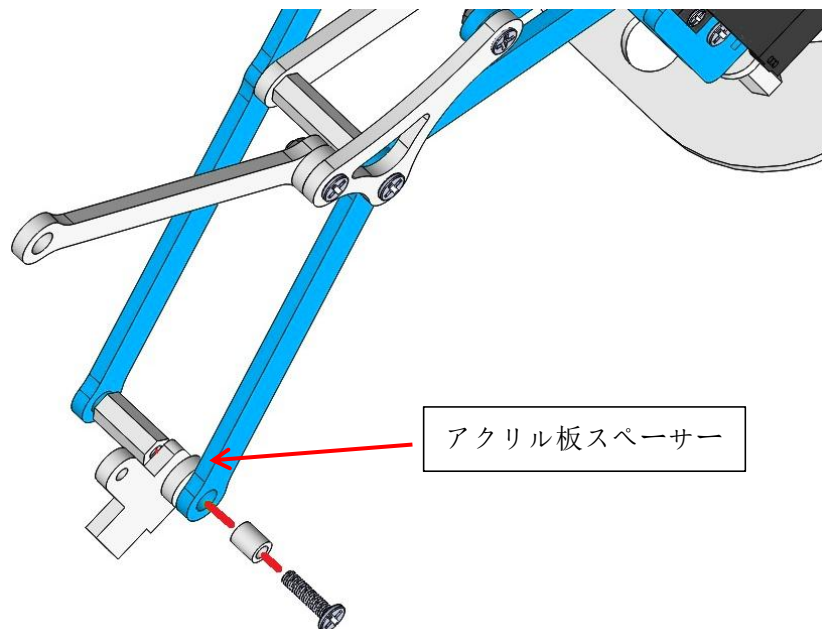
ステップ 22 - 右グリッパージョーを取り付ける

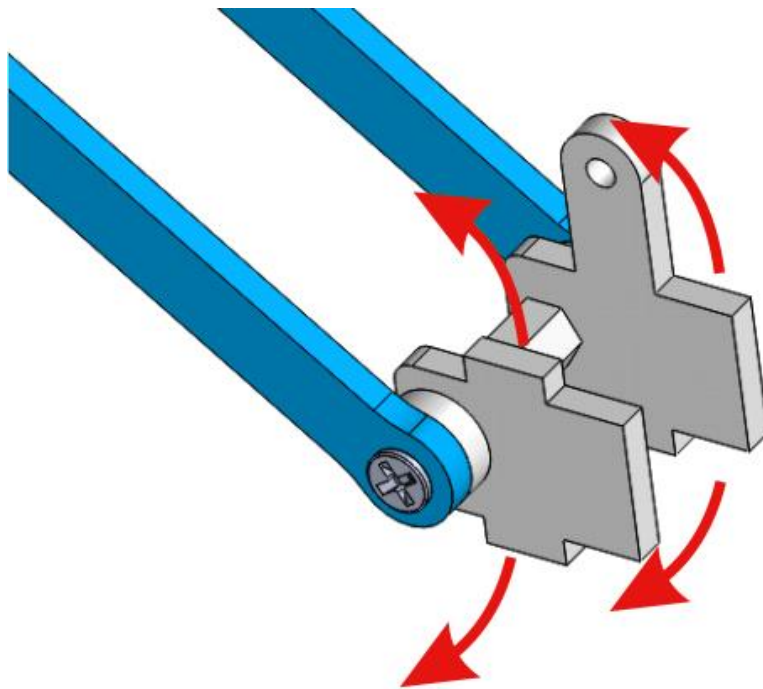
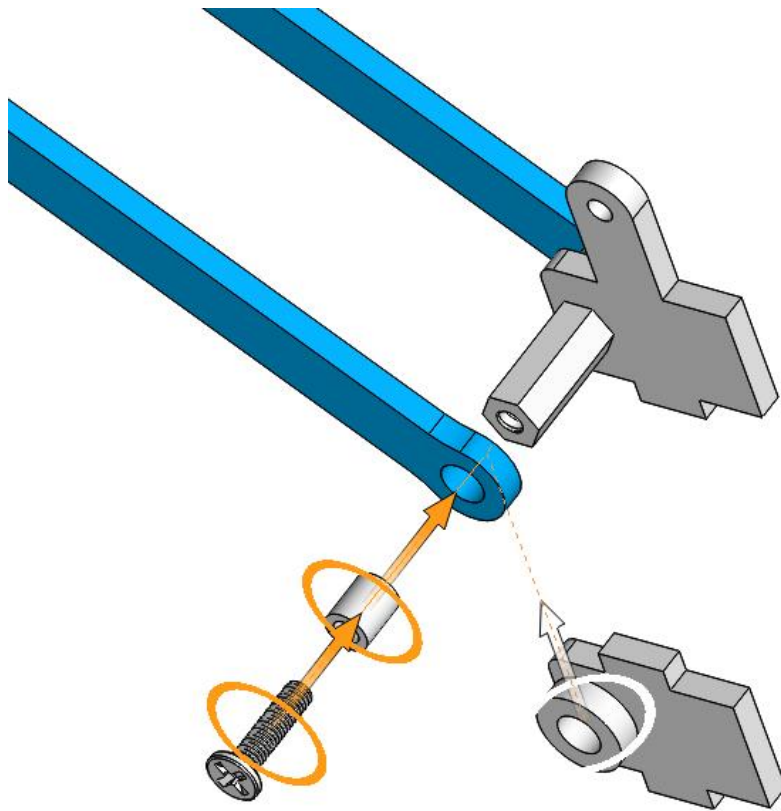
アクリル板 1 枚		アクリル板 1 枚	
			
袋 No.⑬ 5×3.2×3mm スペーサー 1個	袋 No.② M3×9mm ネジ 1個	袋 No.⑤ M3 ナット 1個	
			



ステップ 23 - 手首の構造を取り付ける

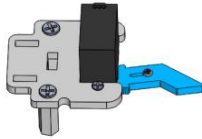
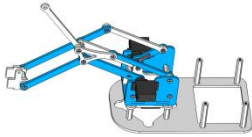
ステップ 20 の構造	アクリル板 1 枚	アクリル板 1 枚	アクリル板スペーサー 2 枚
			
袋 No.① M3×12mm ネジ 2 本	袋 No.⑦ M3×14mm ナイロンスタンドオフ 1 個		袋 No.⑭ 5×3.2×6mm スペーサー 2 個
			

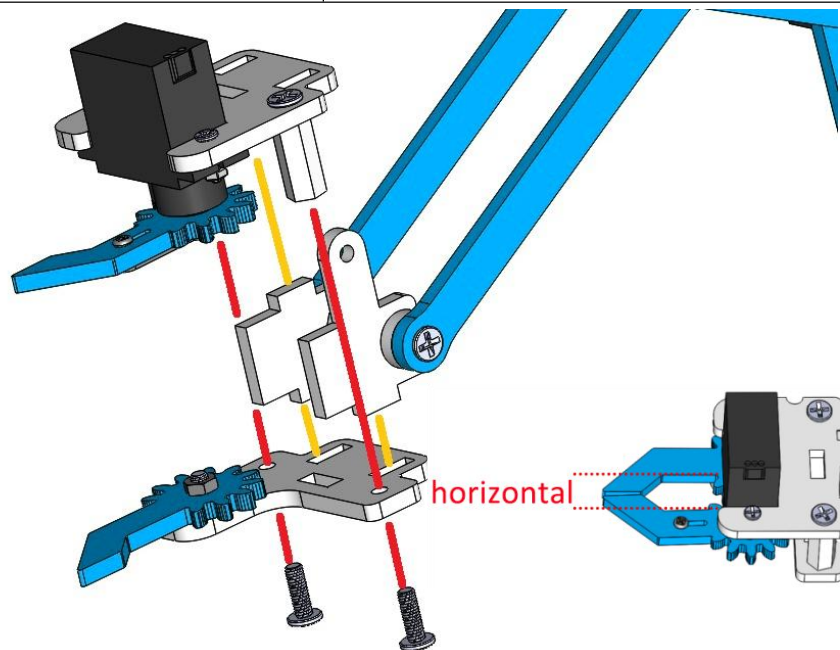




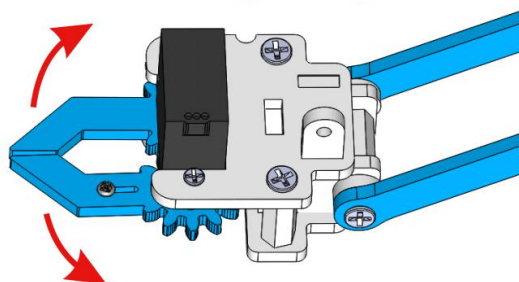
It can swing easily.

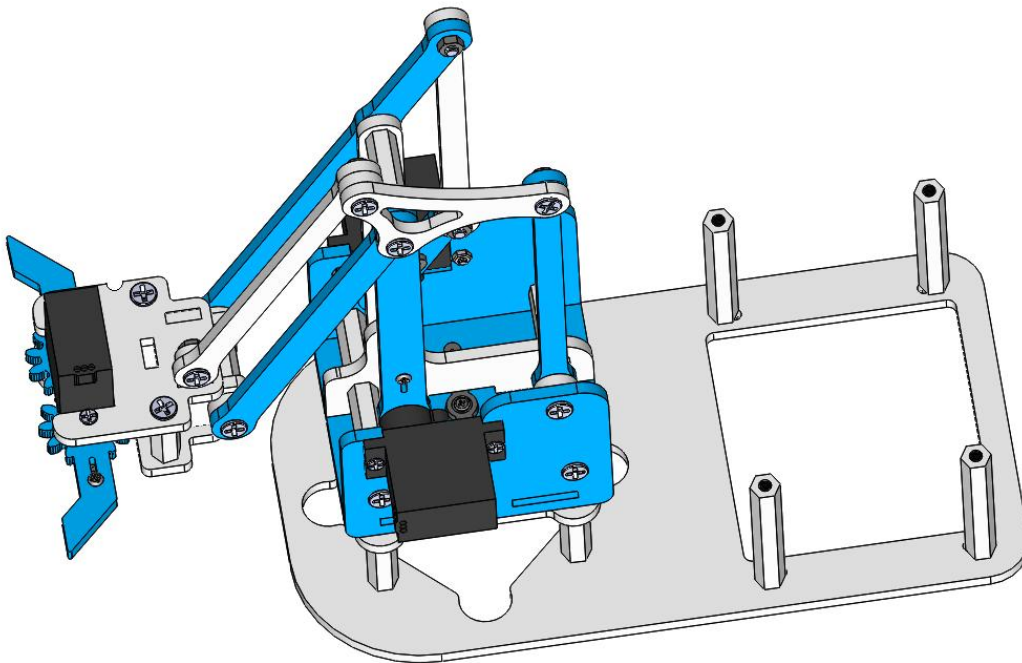
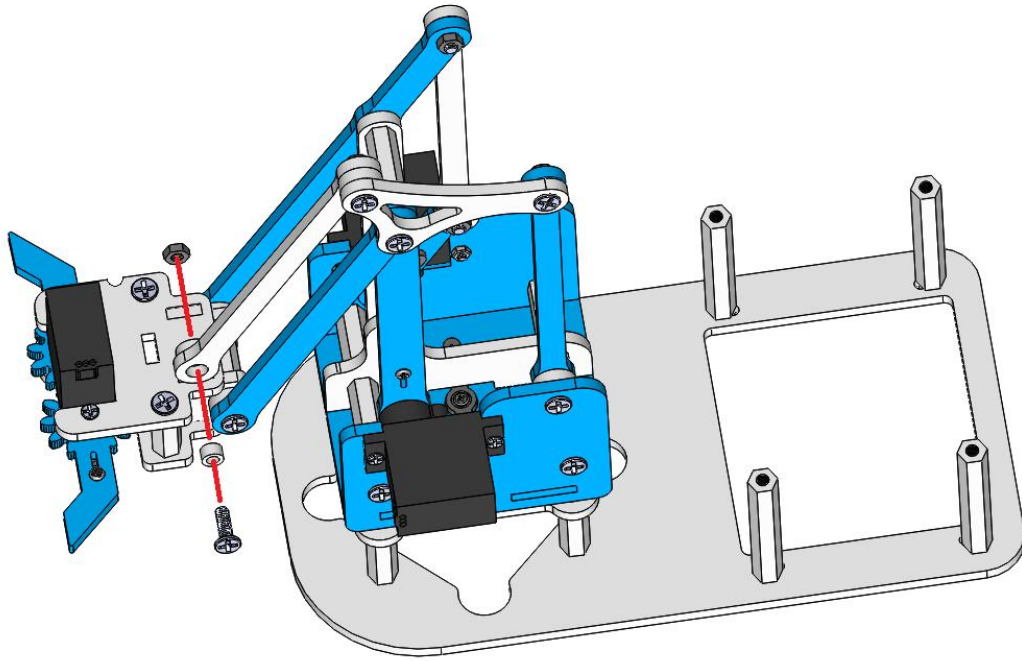
ステップ 24 - グリッパーを手首に取り付ける

ステップ 21 の構造	ステップ 22 の構造
	
ステップ 23 の構造	袋 No.② M3×9mm ネジ 3 本
	
袋 No.⑤ M3 ナット 1 個	袋 No.⑬ 5×3.2×3mm スペーサー 1 個
	

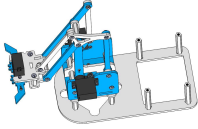


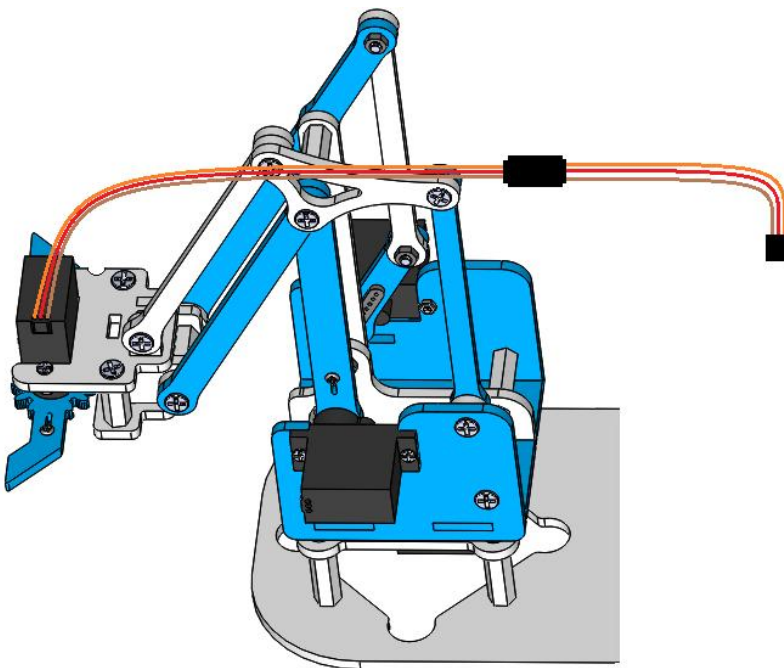
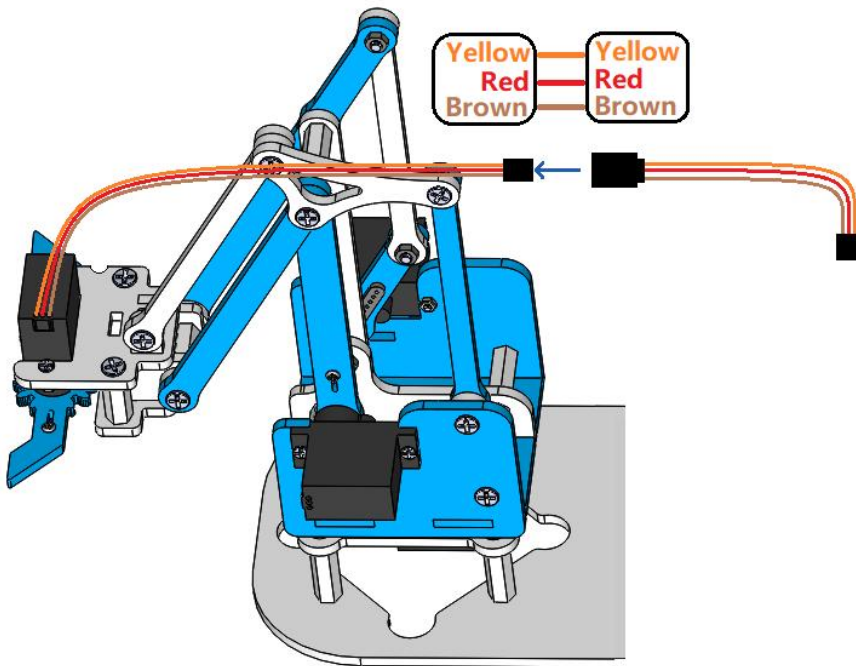
It can swing easily.



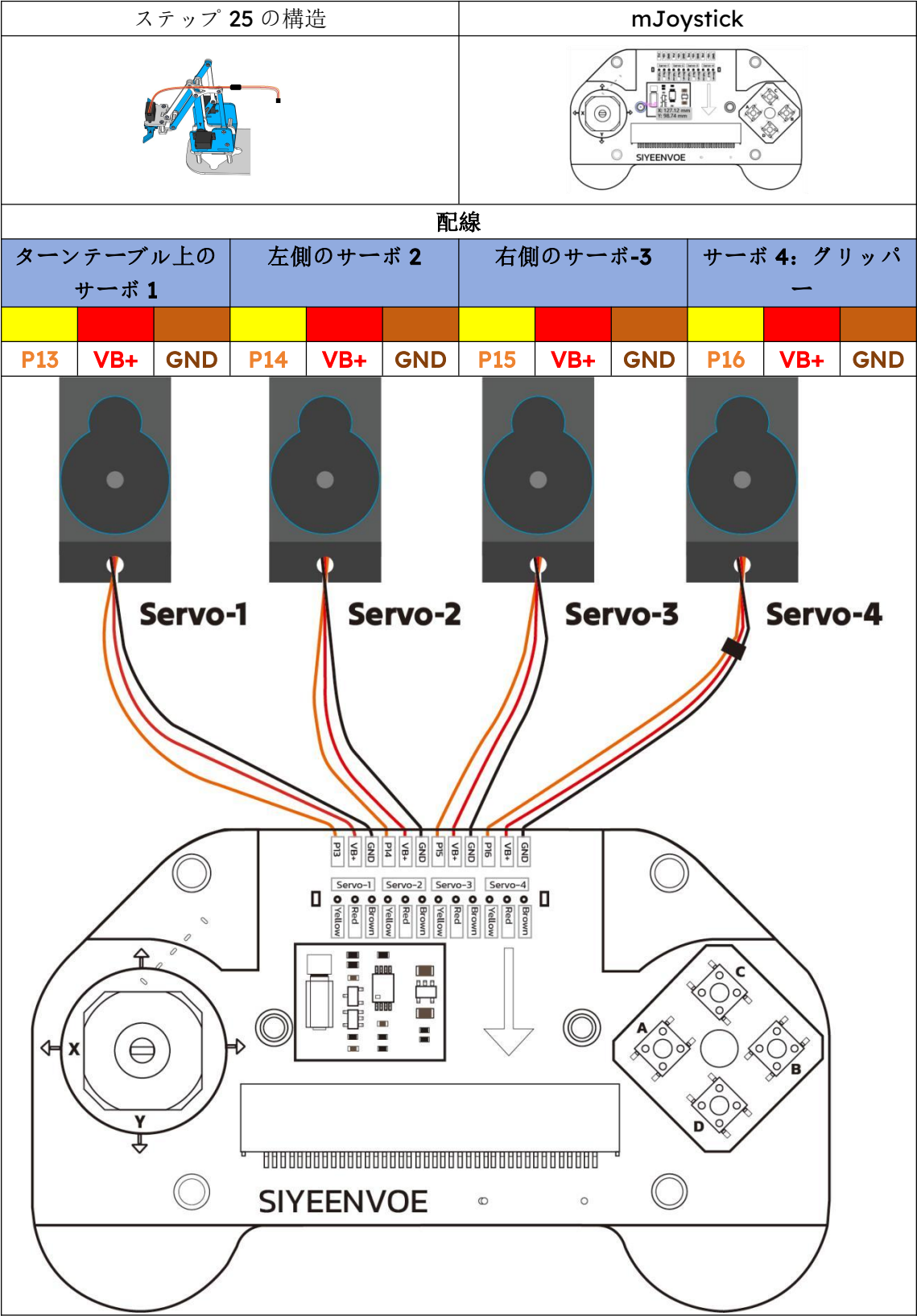


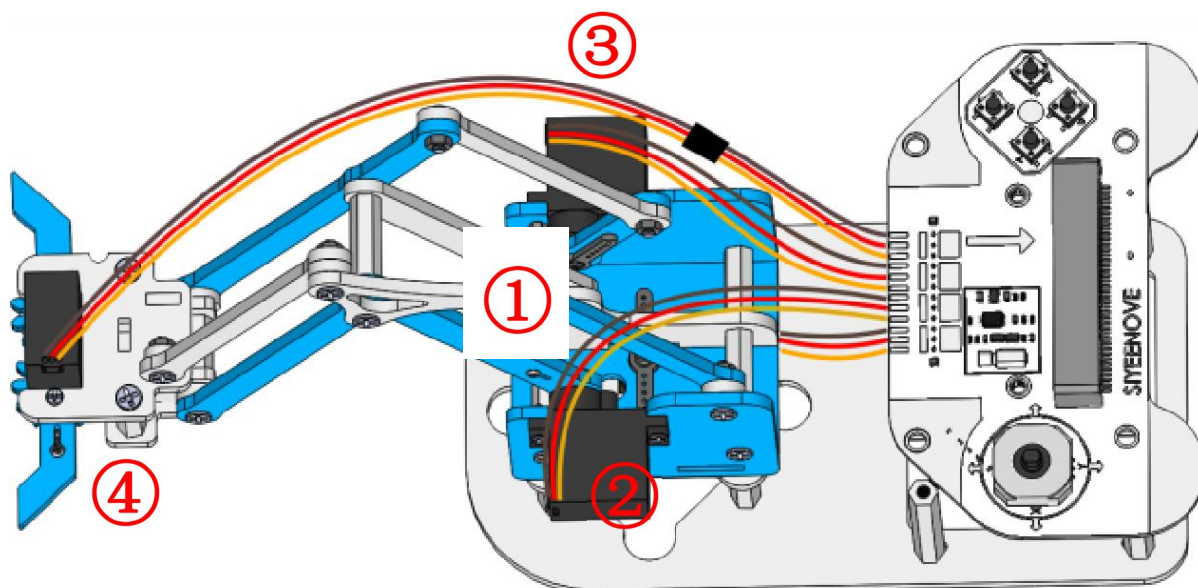
ステップ 25 - 延長ケーブルでサーボ 4 を接続する

ステップ 24 の構造	3P サーボ延長ケーブル 1 本
	

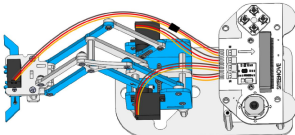
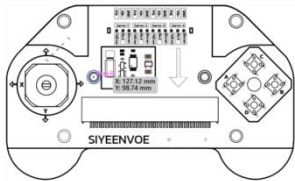
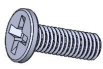
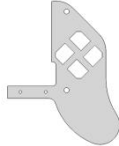


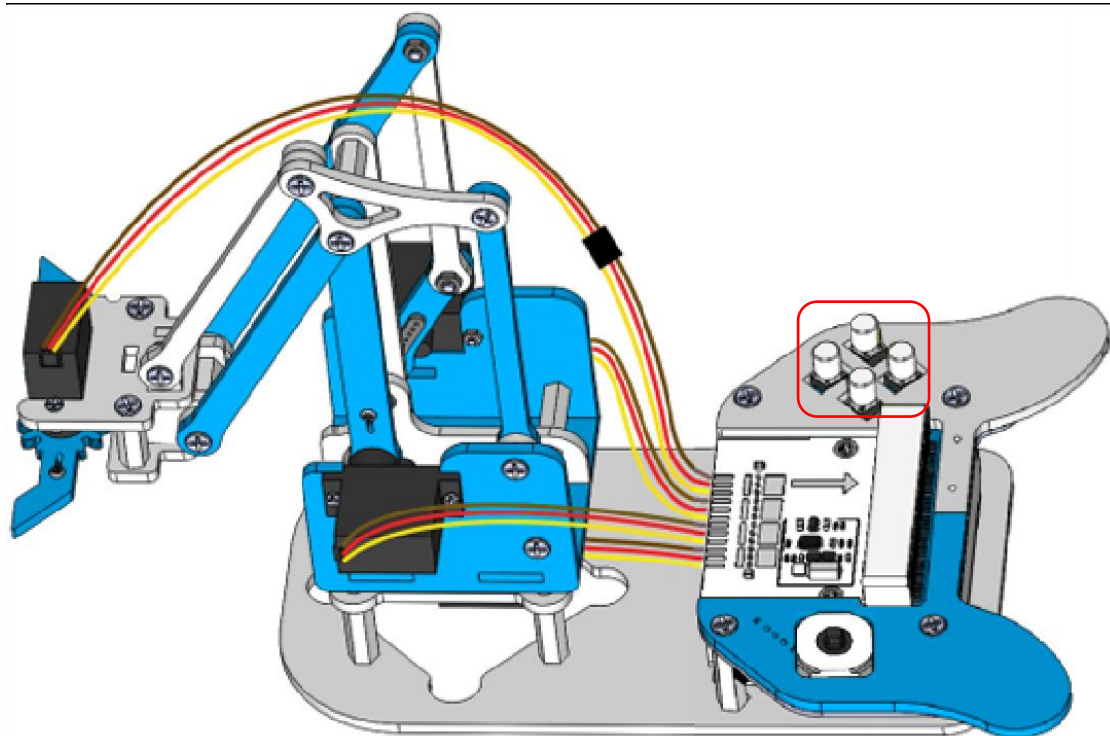
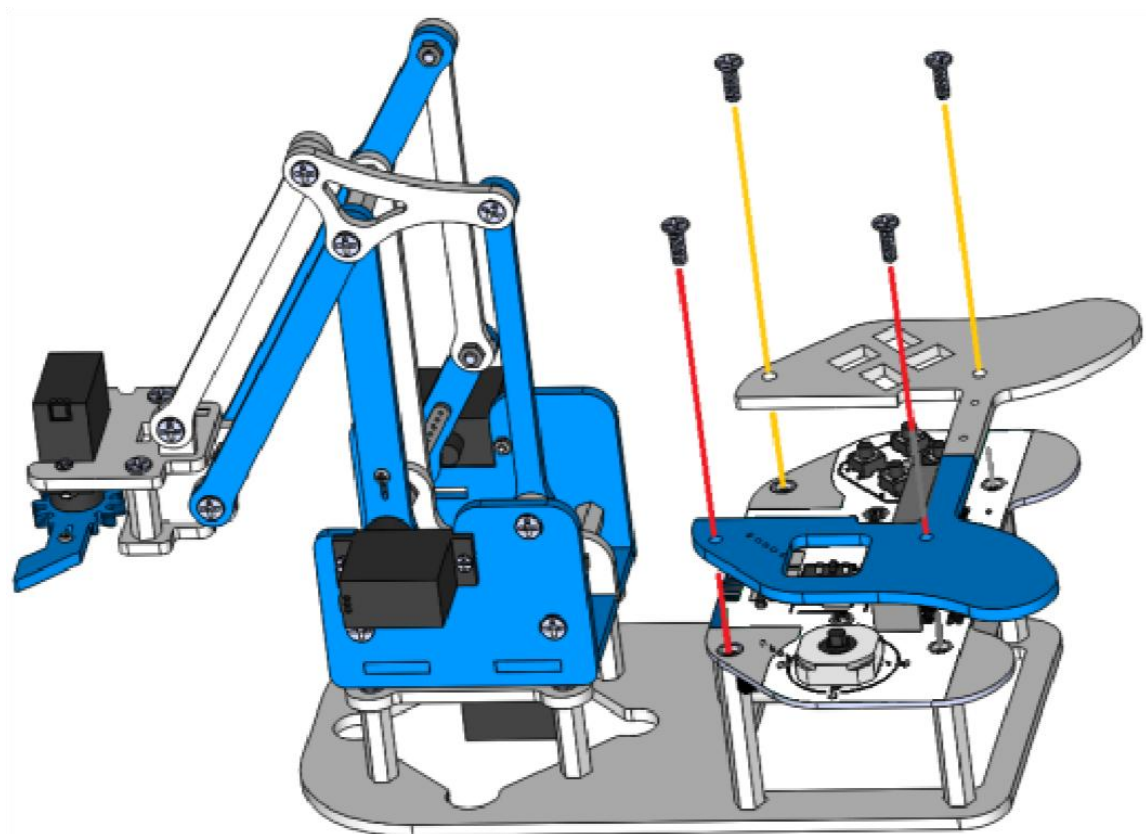
ステップ 26 - サーボをジョイスティックコントローラーに配線する





ステップ 27 - mJoystick をベースフレームに取り付ける

ステップ 26 の構造	mJoystick	袋 No.② M3×9mm ネジ 4 本
		
アクリル板 1 枚	アクリル板 1 枚	袋 No.⑪ ボタンカップ 4 個
		



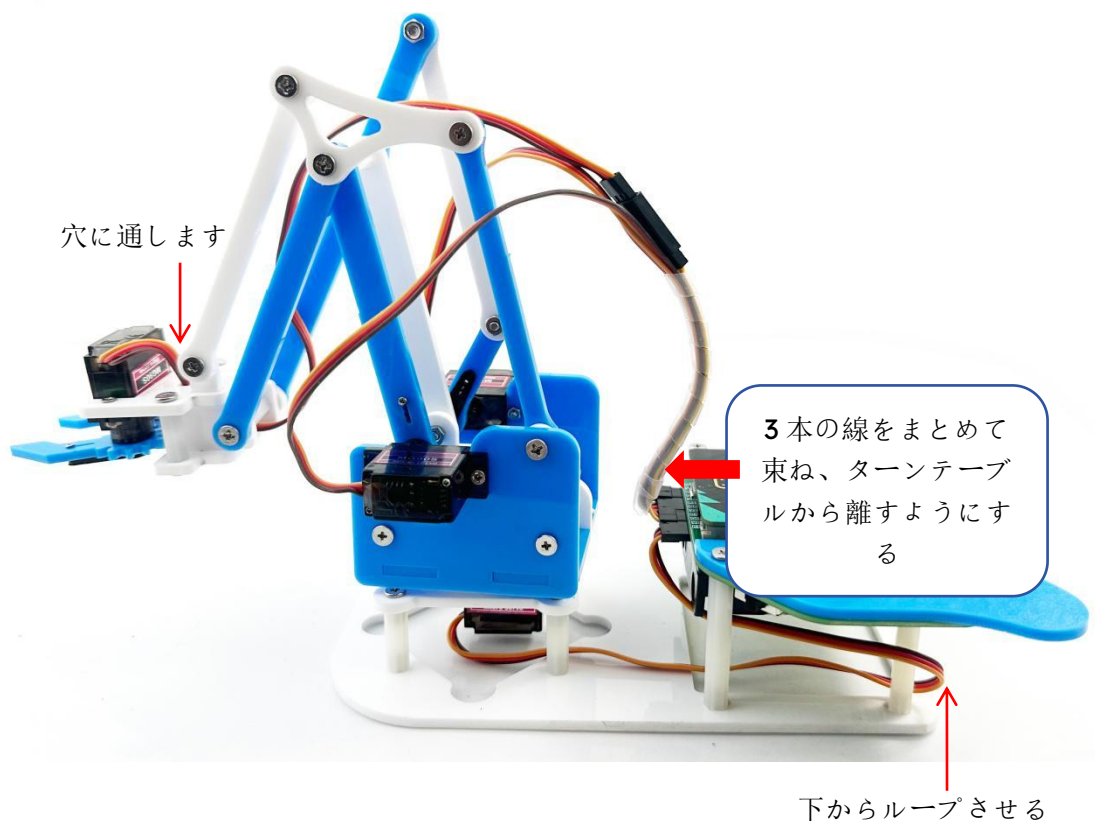
ステップ 28 - ケーブル管理:

▶ スパイラルラップチューブの取り付け:

複数のサーボワイヤーを束ねて、見た目をすっきりさせる

可動部品とのケーブル干渉を防止

長さは調整可能（必要に応じてハサミでカットしてください）



組み立て完了!

残りのアクリル板の用途

組み立て完了後、ジョイスティック用に予備のアクリル板 1 枚と **M3x18mm スタンドオフ 4 個**が残ります。この部品は、ジョイスティックを単独で使用する際に重要な役割を果たします：

主な機能：

バッテリーコンパートメントカバー

単 3 形電池 4 本を確実に固定します

操作中の不意な脱落を防止します

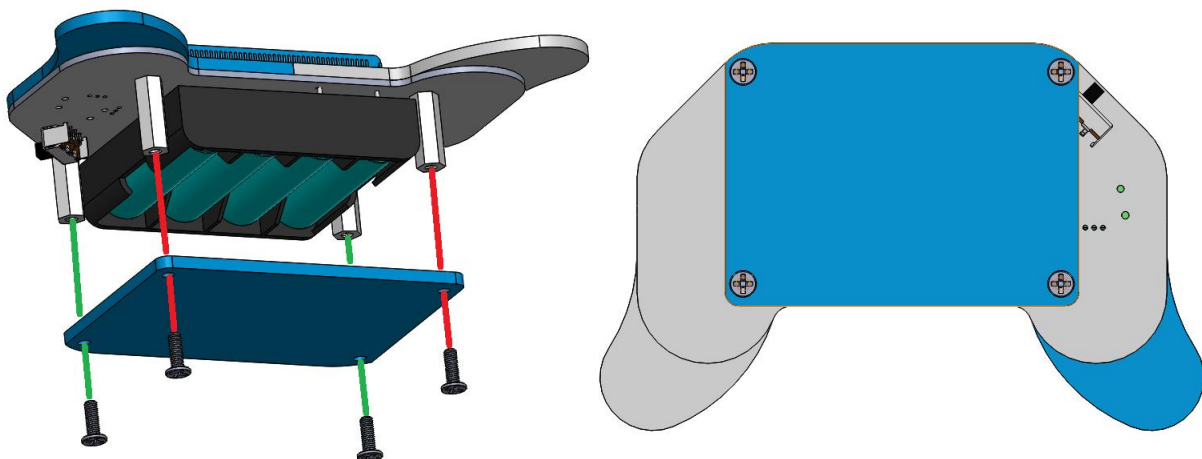
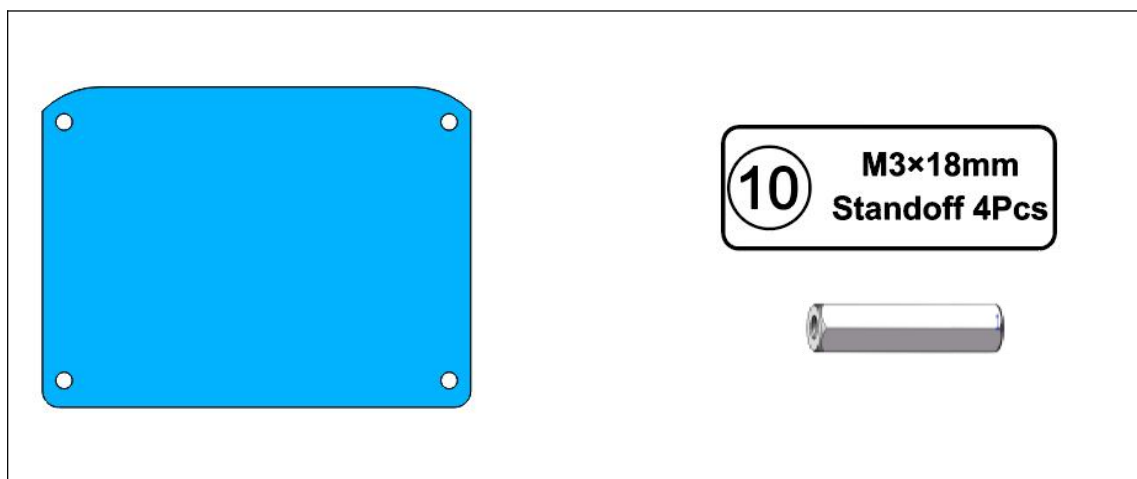
安全バリア

電池端子への直接的な指での接触を遮断

金属物による短絡のリスクを排除

構造補強

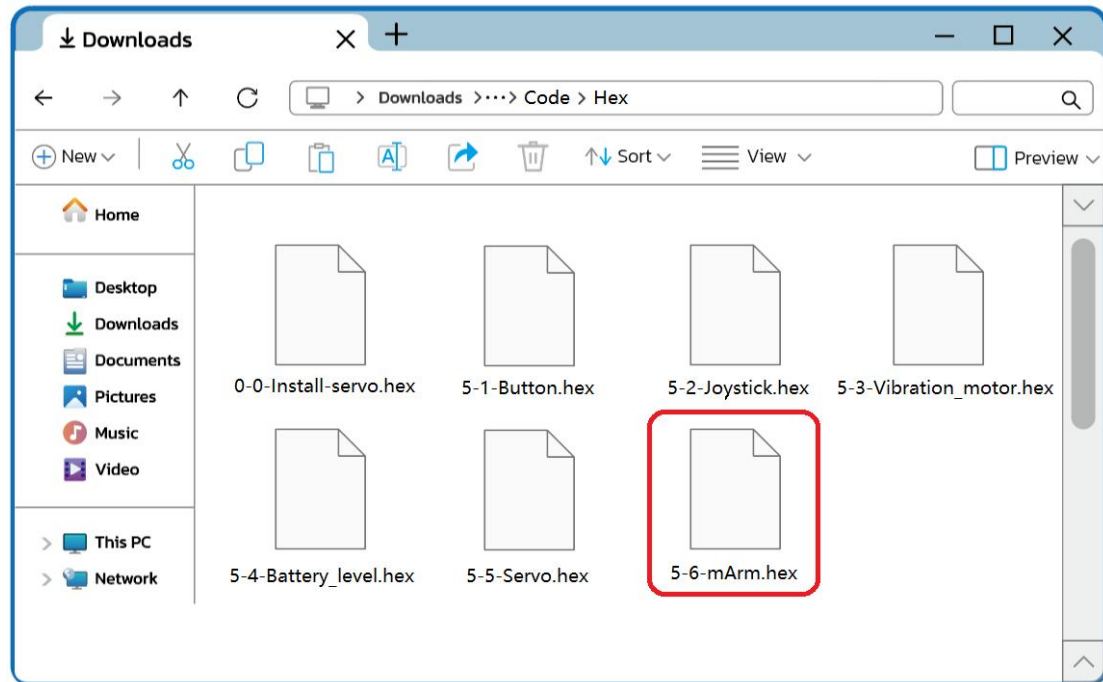
mJoystick のバッテリー収納部に剛性を付与



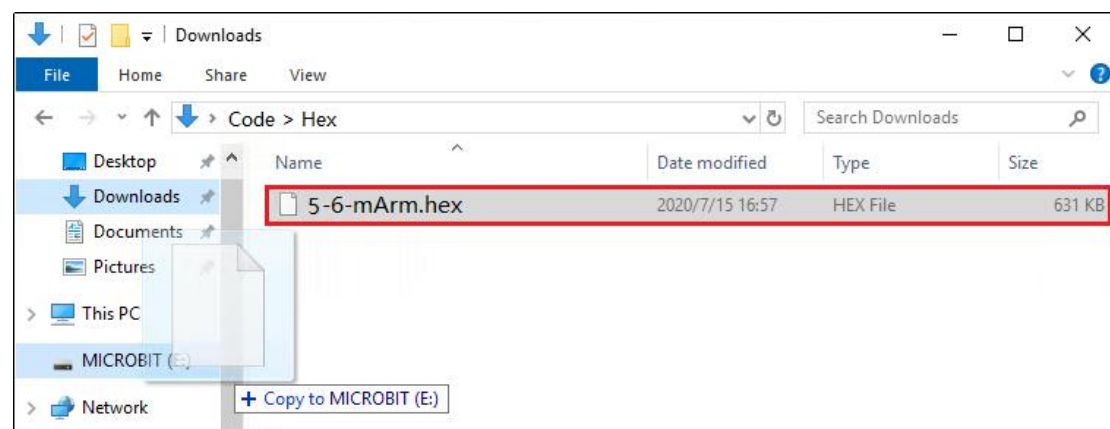
3. ロボットアームの制御

3.1 制御コードのアップロード

ロボットアームの制御コード「**5-6-mArm.hex**」は、ダウンロードしたチュートリアルパッケージの「**Code**」フォルダに保存されています。



HEX ファイルを **MICROBIT** ドライブにドラッグ&ドロップするだけです。



注: これは **micro:bit v2** 専用の **hex** ファイルです。**v1** にドラッグ&ドロップまたは送信すると、通常はサイレントエラーが発生します（ファイルは正常にコピーされますが、プログラムが実行されないか、エラーパターンのみが表示されます）。**micro:bit v1** をご使用の場合は、セクション 4「**micro:bit Python** エディタ クイックスタート」に記載されている以下の手順をお勧めします。

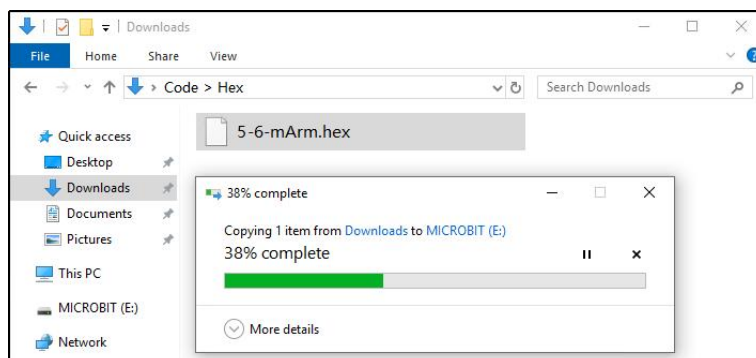
オンラインエディタ（**WebUSB** 経由）を使用して直接フラッシュしてください。デフォルトでは、**v1** と **v2** の両方に対応する汎用 **hex** ファイルが生成されます。

または、**v1** ボードで **Python** エディタを使用して **hex** ファイルを再生成し、それをドラッグ&ドロップしてください。

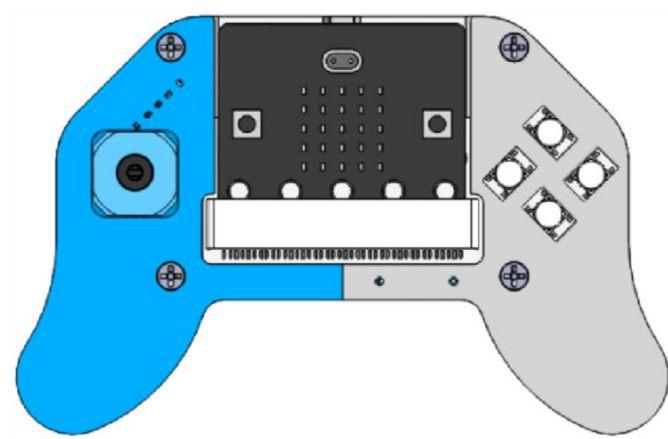
または、**HEX** ファイルを右クリックし、「送る」→「**MICROBIT**」を選択します



アップロード中は **micro:bit** の **LED** が点滅します。完了するまでお待ちください。

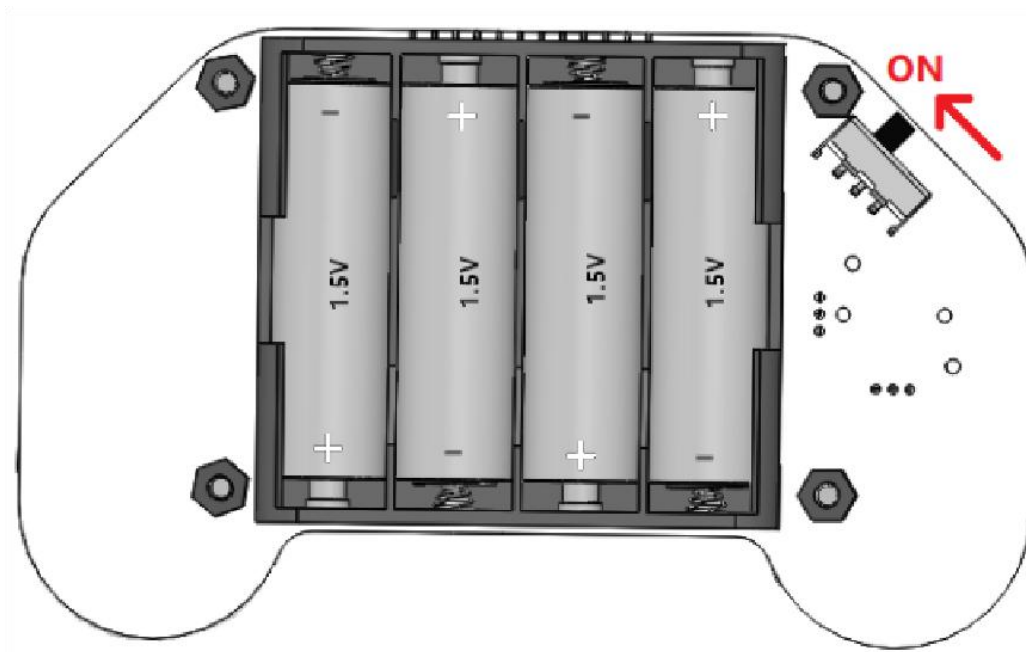


コードのアップロードが完了したら、**micro:bit** ボードをロボットアームのスロットに挿入します。



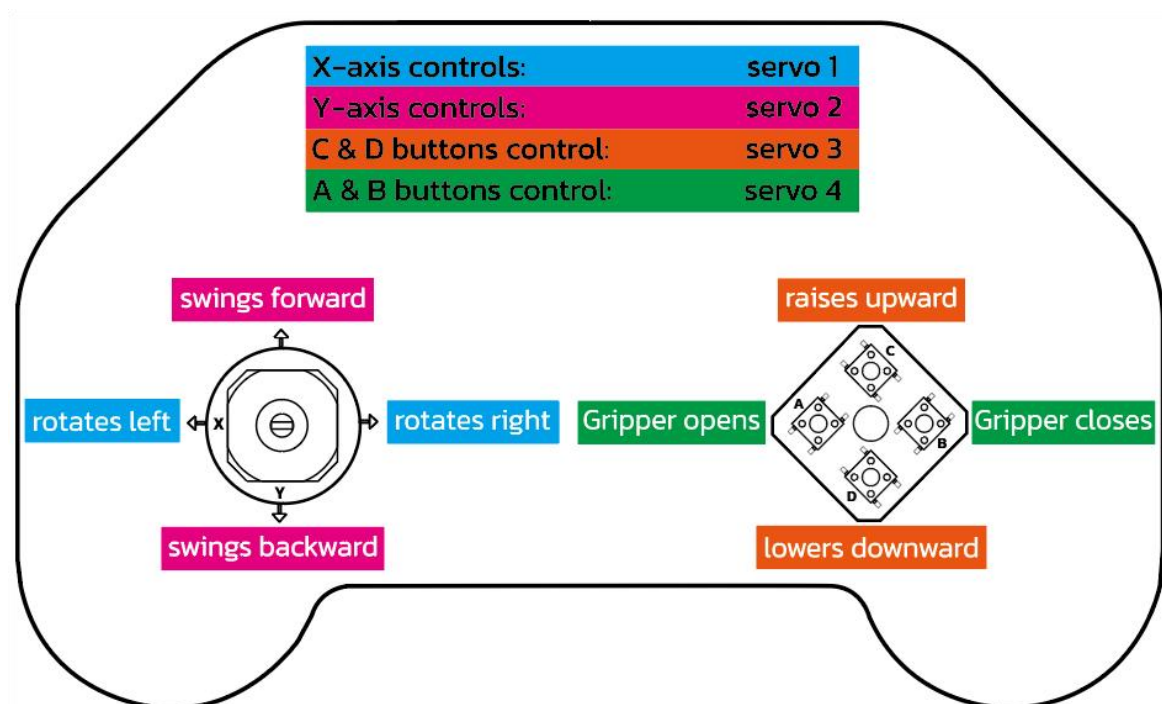
3.2 ロボットアーム制御のクイックスタート

ロボットアームの電源スイッチを入れます。



電源投入後、micro:bit の LED マトリックスは、バッテリー残量（0%～100%）を表示しながら連続して点滅します。バッテリー残量が 30% を下回った場合は、直ちにすべてのバッテリーを交換してください。

- ジョイスティックと4つのボタンを使用して、ロボットアームの動きを制御できます。
- micro:bitのタッチロゴを押すと、コントローラーの振動モーターが作動します。



4. micro:bit Python エディタのクイックスタート

micro:bit Python エディタは、BBC micro:bit での MicroPython プログラミングを始めるのに最適な方法です。無料で、すべてのプラットフォームのブラウザで動作します。

Chrome または Edge ブラウザの使用をお勧めします。WebUSB は、Web ページから直接 micro:bit にアクセスできる、新しく開発中の Web 機能です。また、micro:bit から micro:bit Python エディタへ直接データを受信することも可能です。Google Chrome および Microsoft Edge ブラウザで動作します。

お使いの micro:bit における WebUSB 対応

Chrome または Microsoft Edge の最新版を使用していない場合は、以下のバージョン以降であることを確認してください：

Android、Chrome OS、Linux、macOS、および Windows 10 用の Chrome（バージョン 79 以降）ブラウザ。

Android、Chrome OS、Linux、macOS、および Windows 10 向けの Microsoft Edge（バージョン 79 以降）。

最新の Google Chrome をダウンロードするリンク：

<https://www.google.com/chrome/>

最新の Microsoft Edge をダウンロードするリンク：

<https://www.microsoft.com/en-us/edge/download>

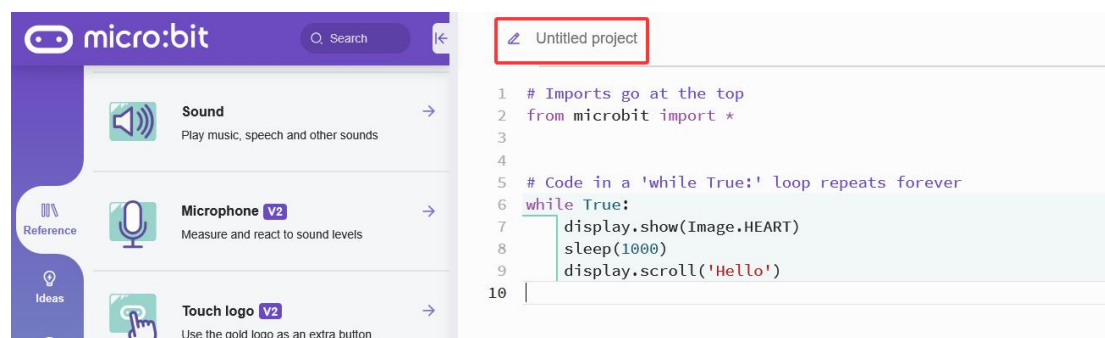
4.1 新しいプロジェクトを作成する

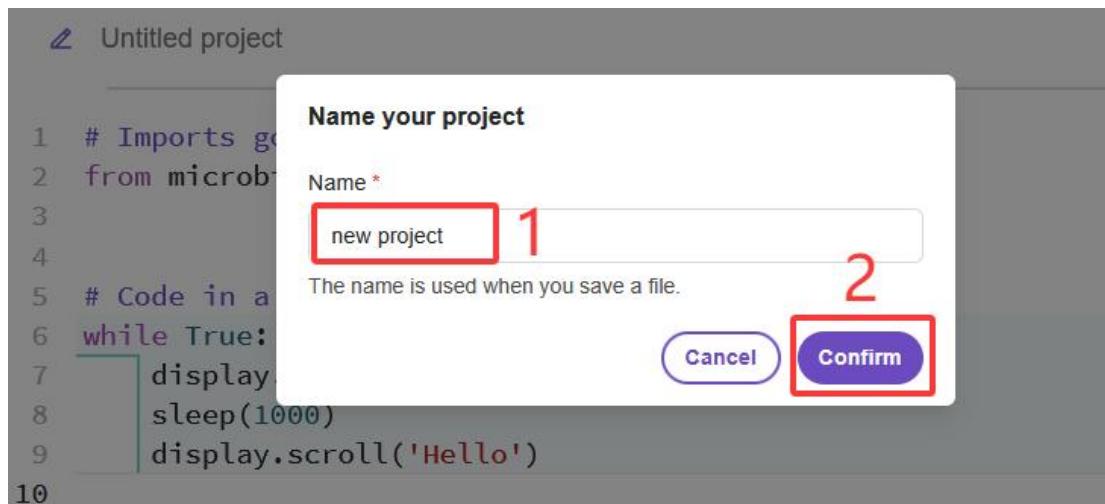
ブラウザで micro:bit Python エディタを開く：

<https://python.microbit.org/v/3> .

エディターが読み込まれると、簡単なスターターテンプレートが含まれたコード編集エリアが表示されます。エディター上部の「Untitled project」をクリックし、プロジェクトにわかりやすい名前（例: Joystick_Test）を入力してください。

既存のテンプレートを編集することも、独自の Python コードで完全に置き換えることもできます。



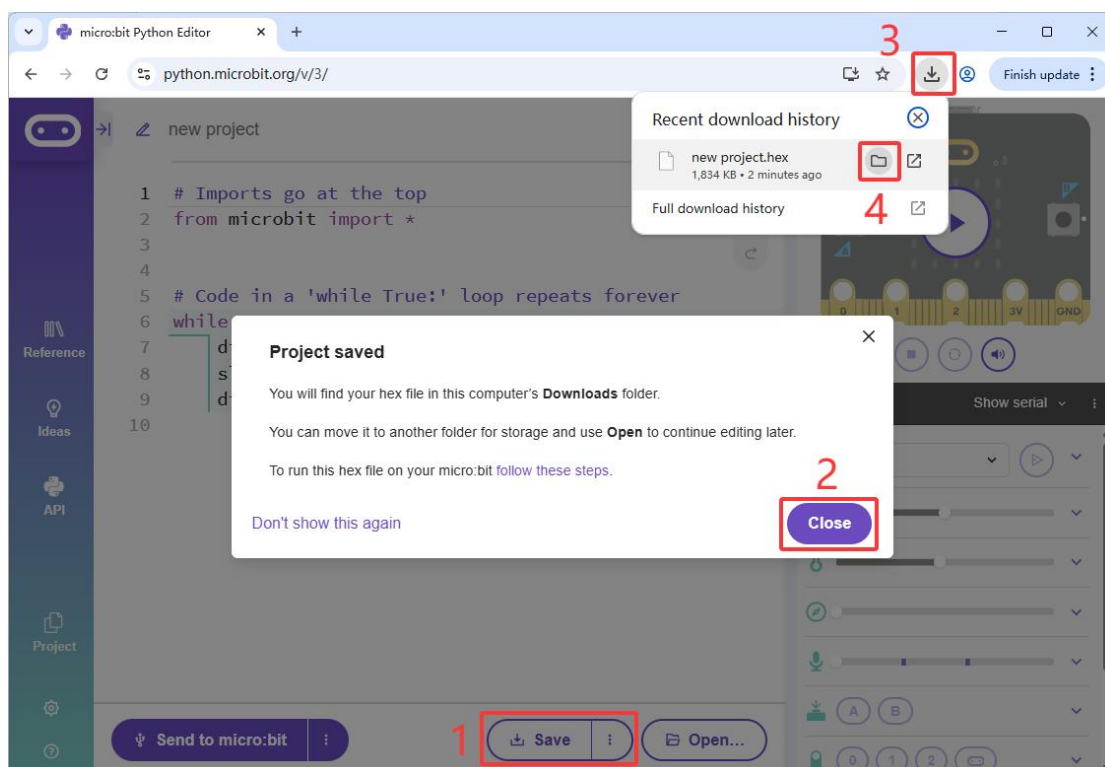


注: micro:bit Python エディタはシングルプロジェクトモデルで動作するため、一度に編集できるプロジェクトは **1** つだけです。マルチタブブラウジング、複数ファイルの並列編集、または組み込みのプロジェクト切り替え機能はサポートされていません。

4.2 プロジェクトの保存

1. プロジェクトを hex ファイルとして保存

「**保存**」ボタンをクリックすると、エディタは自動的に作業内容をブラウザのローカルストレージに保存します。ダウンロードされた **hex** ファイルは、ブラウザのデフォルトのダウンロードフォルダにあります。



これが、**micro:bit** が認識して実行できる最終的なファイル形式です。

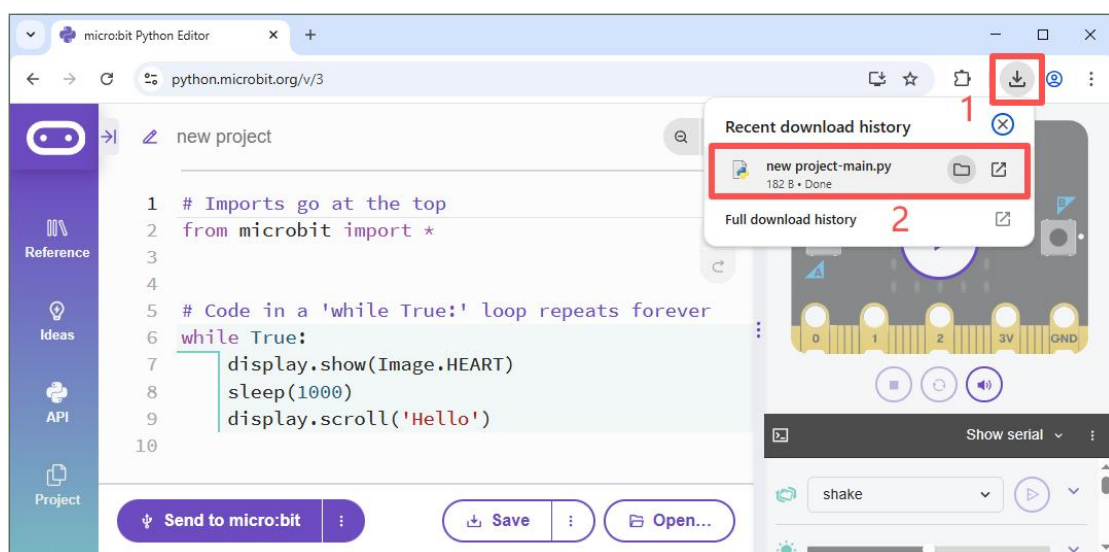
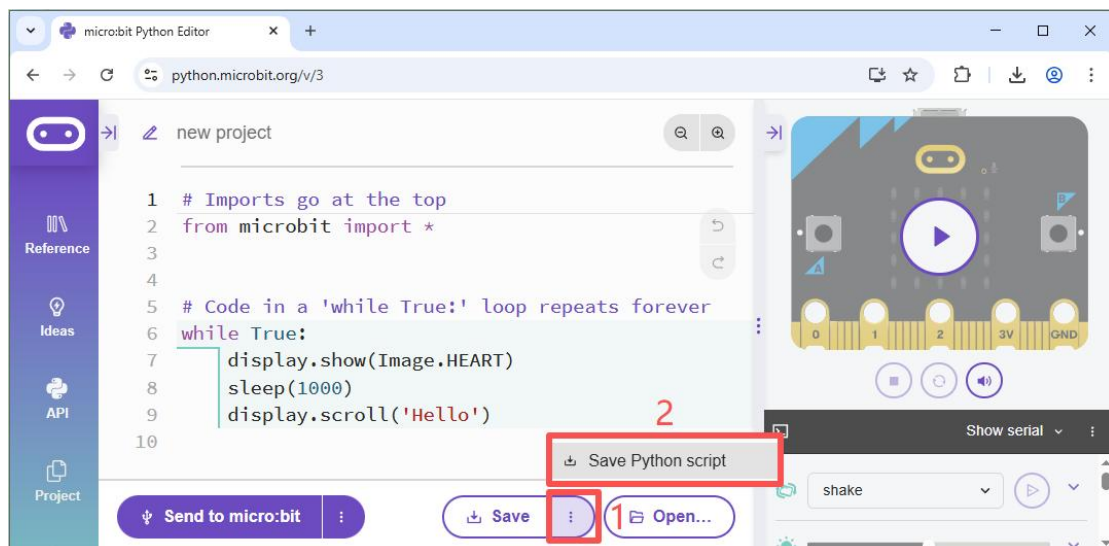
機能:

micro:bit へのアップロード: **.hex** ファイルを **micro:bit** の **USB** ドライブにドラッグ & ドロップ（または送信）すると、**micro:bit** がメモリに読み込んで実行を開始します。

プロジェクトのバックアップと共有: **.hex** ファイルを保存することで、すぐに実行可能なプロジェクトのバージョンをバックアップしたり、ソースコードにアクセスすることなく直接使用できる形で他の人と共有したりできます。

2. プロジェクトを.py ファイルとして保存

「保存」ボタンの横にある **3** つの点をクリックし、次に「**Python スクリプトを保存**」ボタンをクリックすると、**.py** ファイルがコンピュータのダウンロードフォルダに保存されます。



これが記述された **Python** コードのテキストです。

機能:

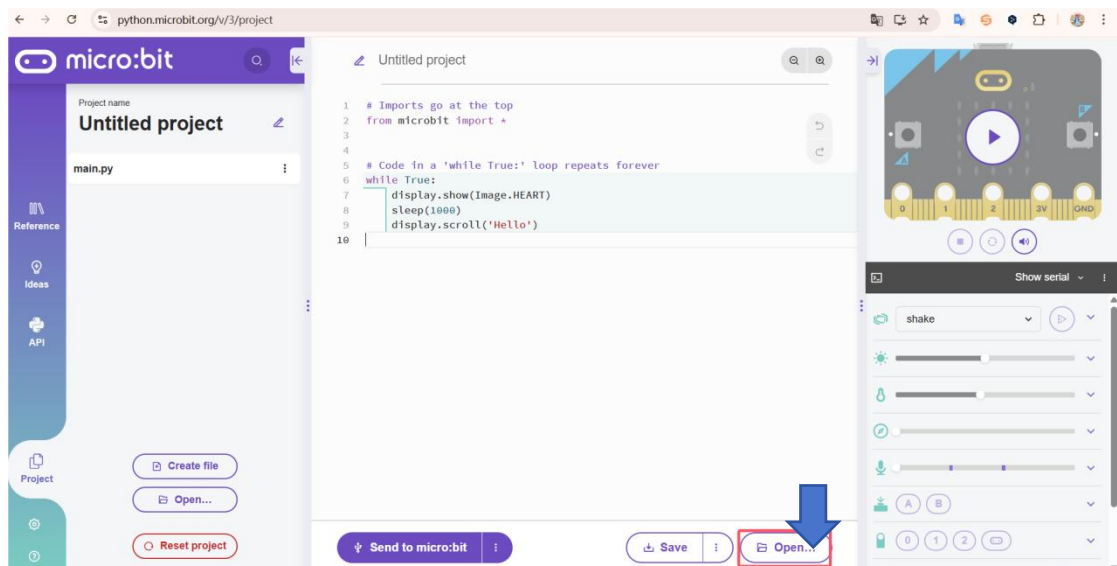
編集と作成: micro:bit Python エディタまたは任意のテキストエディタで、プログラムの作成、変更、デバッグを行ってください。

注意: micro:bit は **.hex** ファイルを実行しますが、開発は **.py** ファイルで行います。

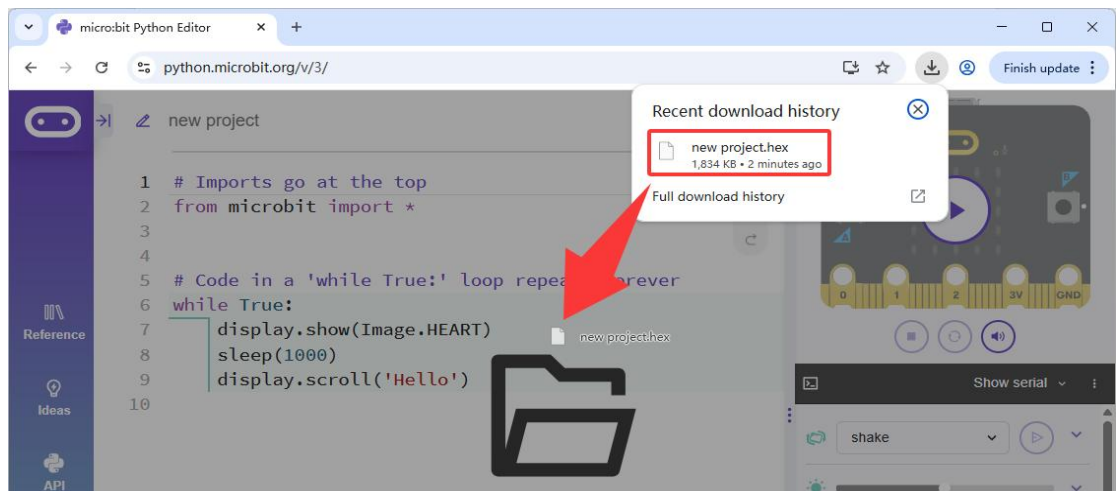
4.3 ファイルのインポート

プログラム（以前に保存したものや、教師や第三者から提供されたものなど）を開くには、「開く」ボタンを選択します。

そこから、開きたい **.hex** ファイルまたは **.py** ファイルを選択し、「開く」をクリックします。



あるいは、**.hex** ファイルや **.py** ファイルをエディタにドラッグ&ドロップすることもできます。



ファイルをインポートする際:

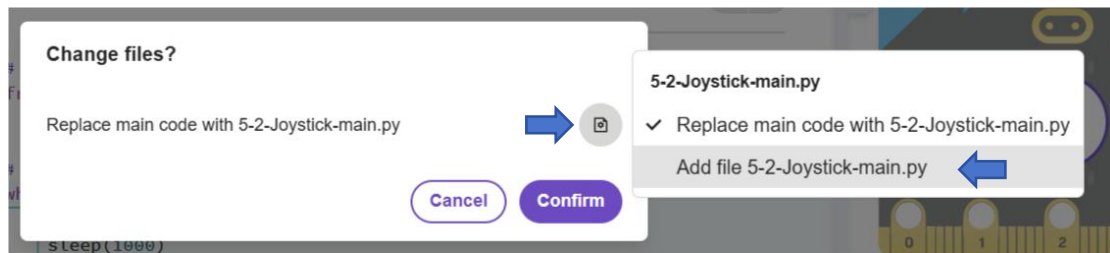
新しい **.hex** ファイルを開くと、警告なしに現在のプロジェクトが即座に上書きされ、この操作は元に戻せません。コードが失われないよう、必ず事前に作業内容を保存してください。

新しい **.py** ファイルを開くと、「ファイルを変更しますか？」というプロンプトが表示されます。その際、以下のいずれかを選択できます：

「現在のコードを上書きしますか？」と確認が表示されます。

「キャンセル」を選択して現在のプロジェクトを維持する、または

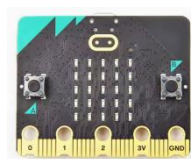
「追加」を選択して、新しいファイルを既存のコードに追加します。



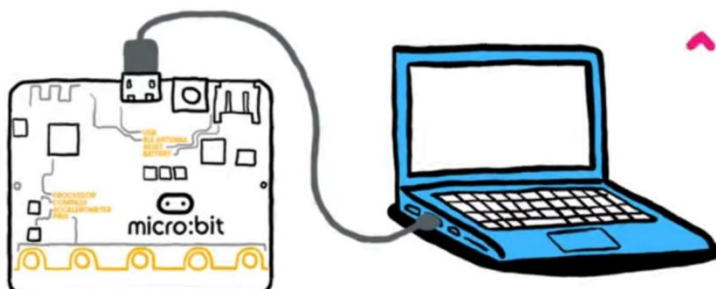
注： micro:bit Python エディタはシングルプロジェクトモデルを採用しており、一度に開いてアクティブにできるプロジェクトは **1** つだけです。マルチタブ閲覧、複数ファイルの並列編集、または組み込みのプロジェクト切り替え機能はサポートしていません。新しいプロジェクトを読み込むと、現在のプロジェクトが置き換えられます。この合理化された集中型のアプローチにより、初心者向けのインターフェースが簡素化されます。生徒が現在のタスクにのみ集中できるようにすることで認知的負荷を軽減し、複数のプロジェクト間の切り替えによる混乱を避け、「一度に一つのこと」という教育哲学に沿った設計となっています。

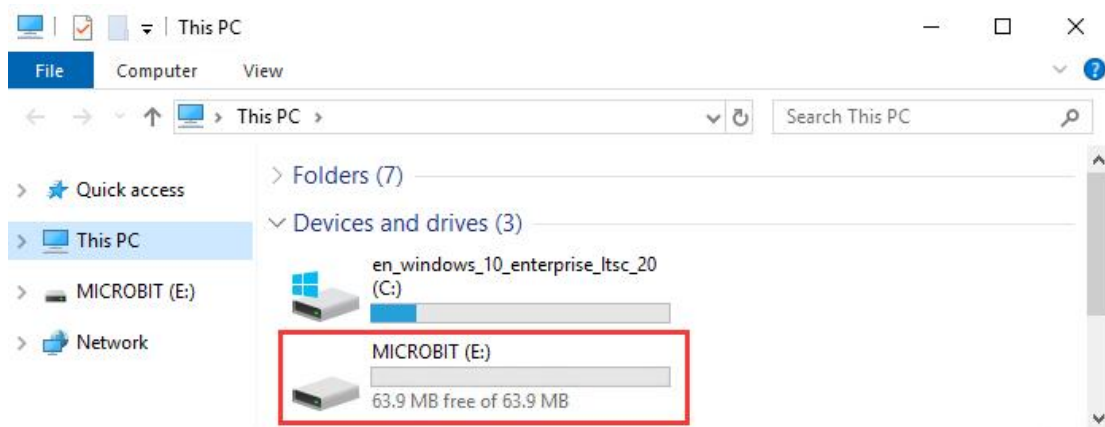
4.4 コードのアップロード

必要なもの：

PC	Micro:bit v2.x.x	Micro USB ケーブル
		

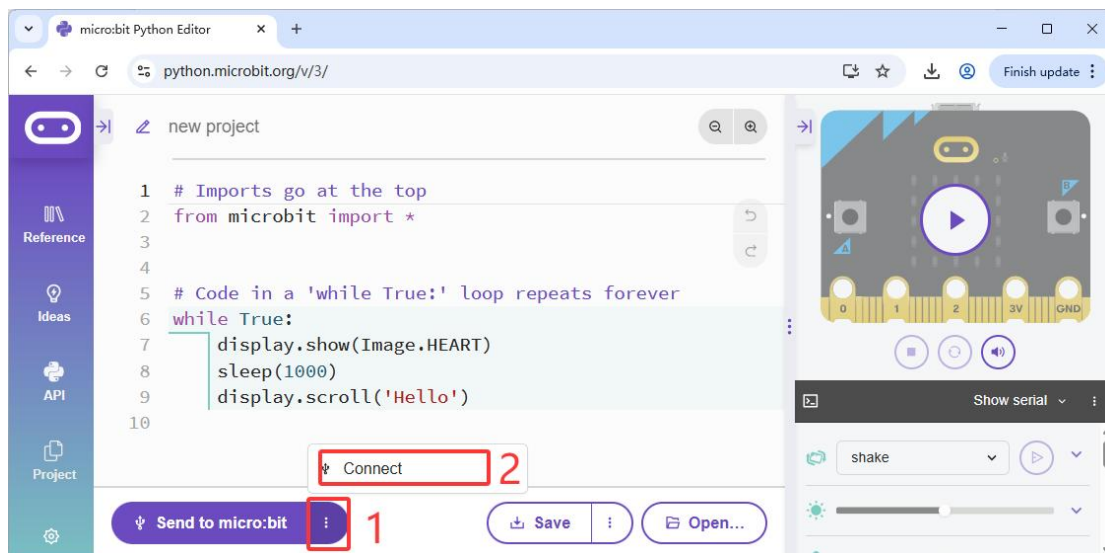
Micro USB ケーブルを使用して、micro:bit ボードを PC に接続します。PC 上に「MICROBIT」という新しい USB ドライブが表示されます：



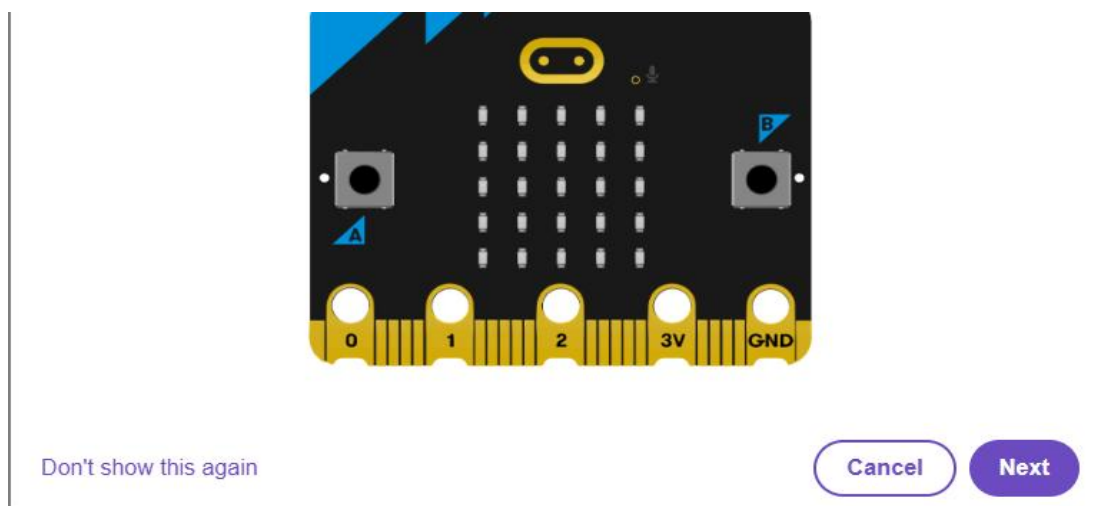


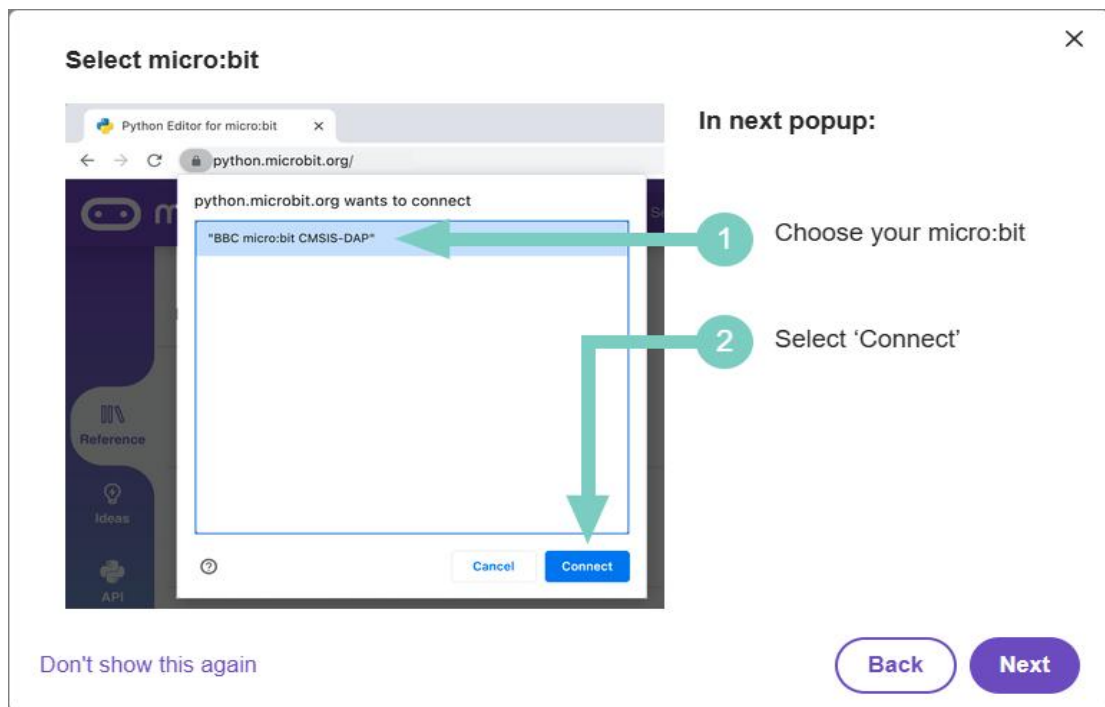
ダイレクトフラッシュ（WebUSB）：Google ChromeやMicrosoft Edgeなどの対応ブラウザを使用している場合、WebUSBを使用してmicro:bitをエディタに直接接続し、ファイルをドラッグ&ドロップすることなくプログラムを書き込むことができます。

「Send to micro:bit」ボタンの横にある3つのドットをクリックし、次に「Connect」ボタンをクリックします：

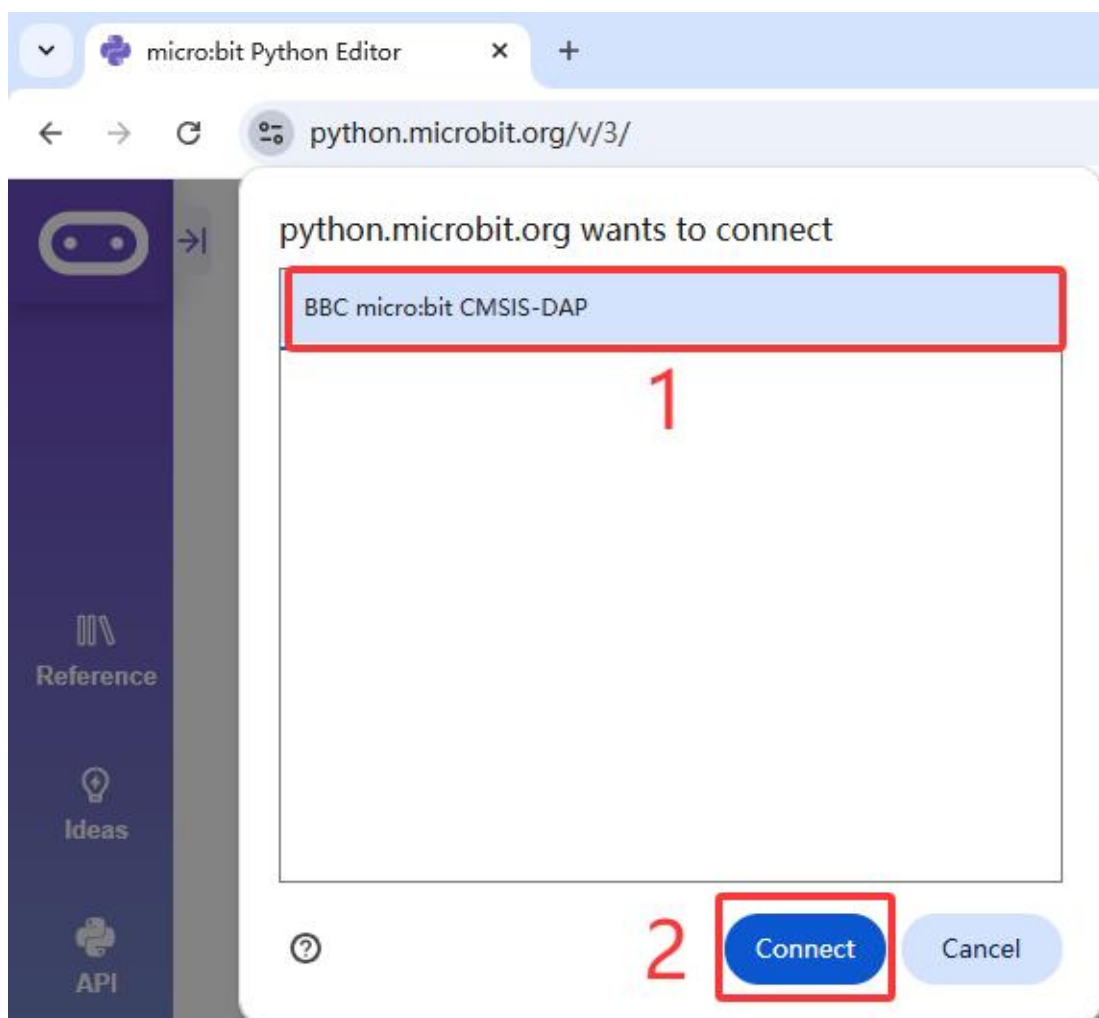


「次へ」をクリックします：

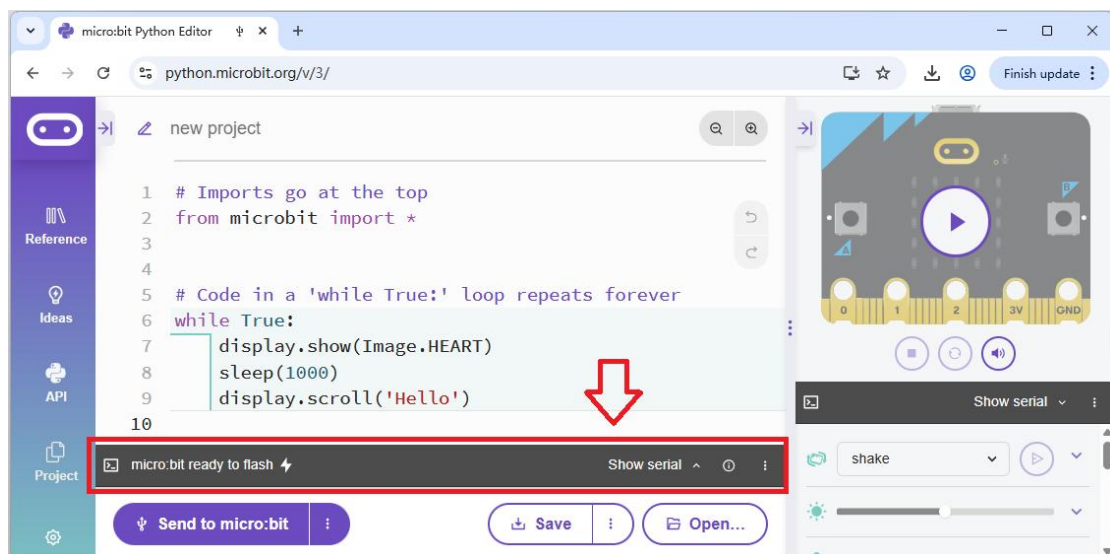




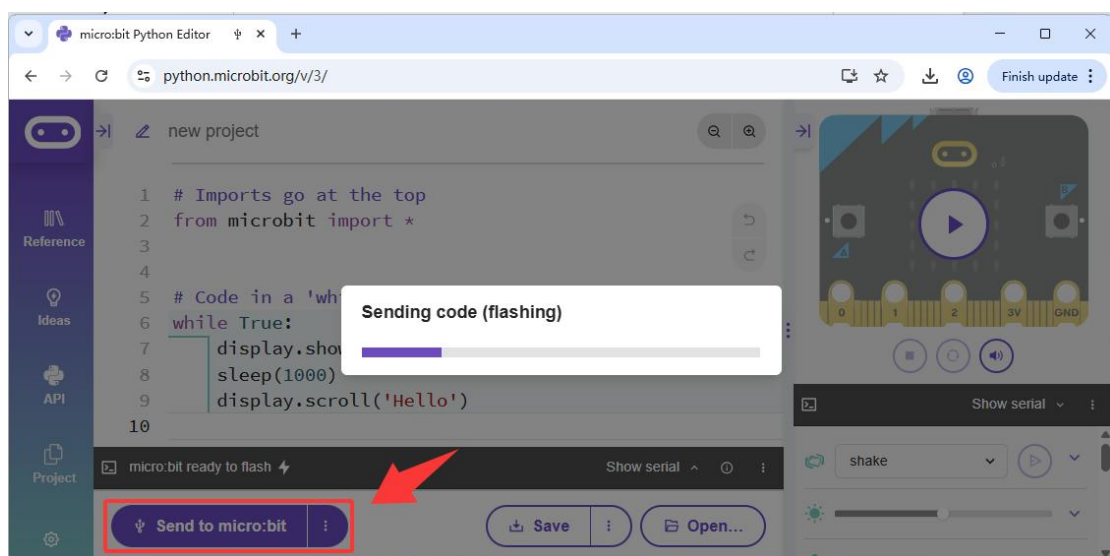
micro:bit を選択し、
「接続」をクリックします。



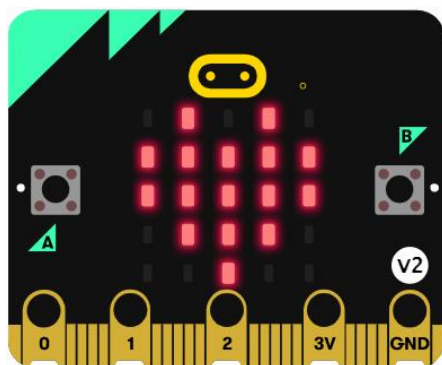
micro:bitの書き込み準備完了画面が表示されたら、micro:bitがPythonエディタに正常に接続されたことを示しています。



これで「**micro:bitに送信**」をクリックして、Pythonエディタからmicro:bitにコードをアップロードできます：



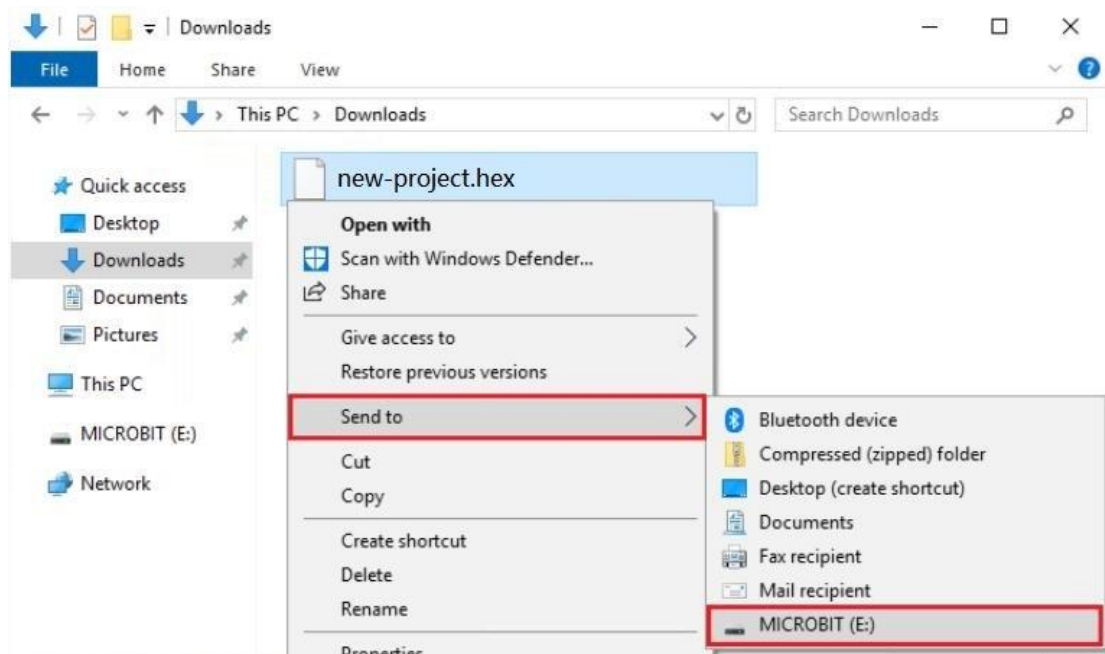
コードのアップロードが完了すると、micro:bitのLEDマトリックスにハートのアイコンが表示され、続いて「Hello」の文字がスクロール表示されます。



コードをアップロードする 2つの代替方法

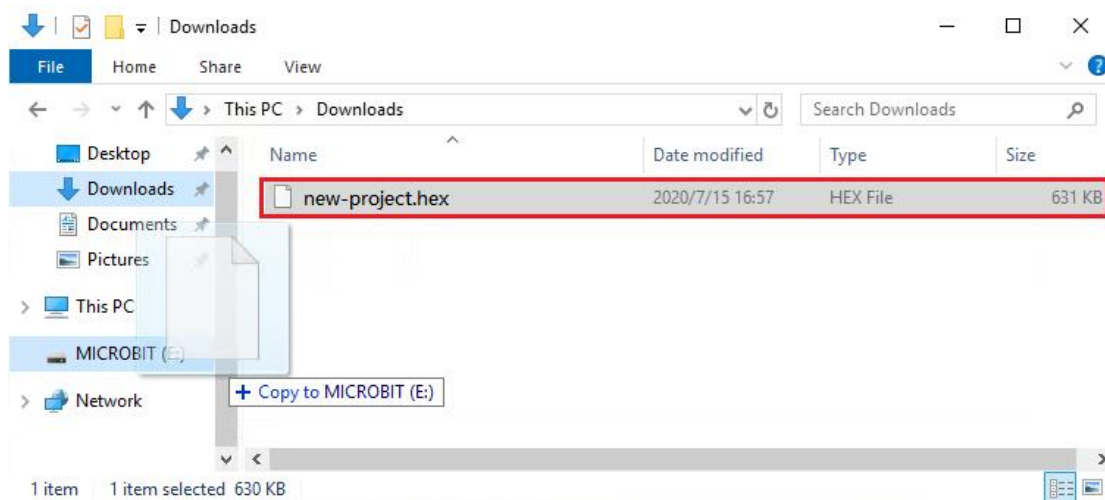
1. 右クリックで送信

「.hex」ファイルを選択し、右クリックして、micro:bit ドライブを「送る」で選択します:

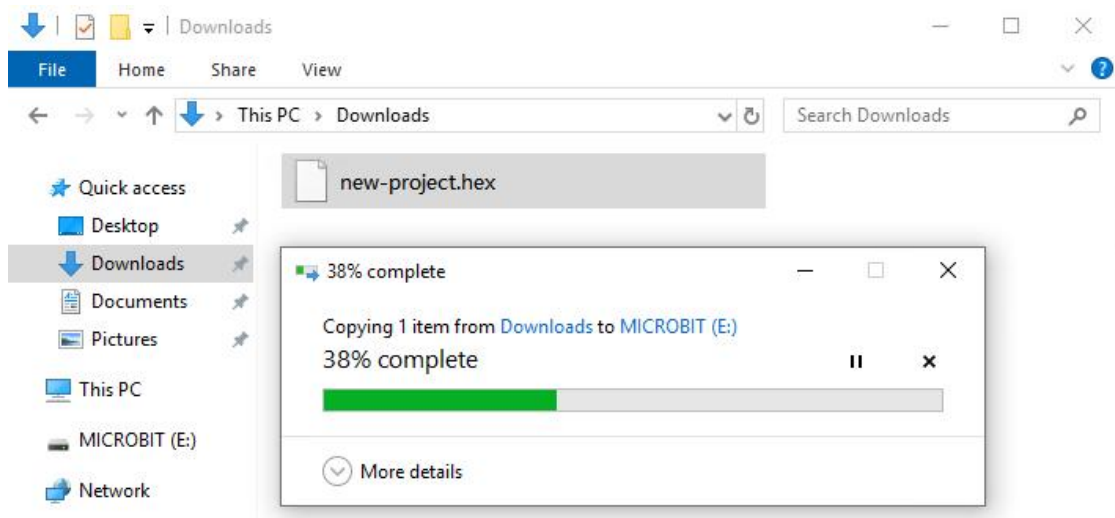


2. ドラッグ&ドロップ

または、「.hex」ファイルを micro:bit ドライブに直接ドラッグ&ドロップします:



コードがアップロードされていることを示す以下の画面が表示されます:



4.5 micro:bit Python エディタの基本構文を学ぶ

micro:bit プラットフォームでは、Python エディタの公式リファレンスドキュメントが提供されています。

ユーザーガイド:

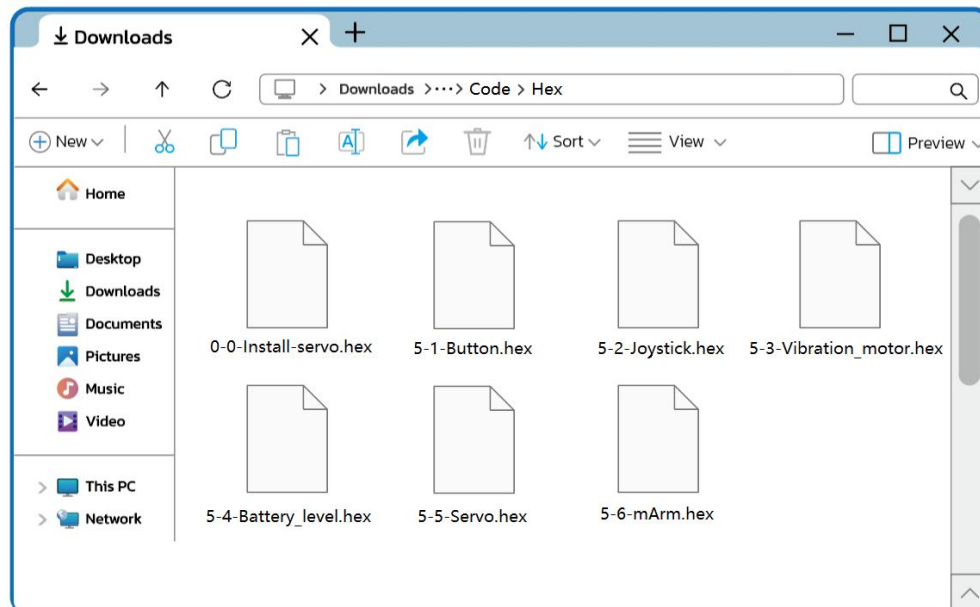
<https://microbit.org/get-started/user-guide/python-editor/>

MicroPython ドキュメント:

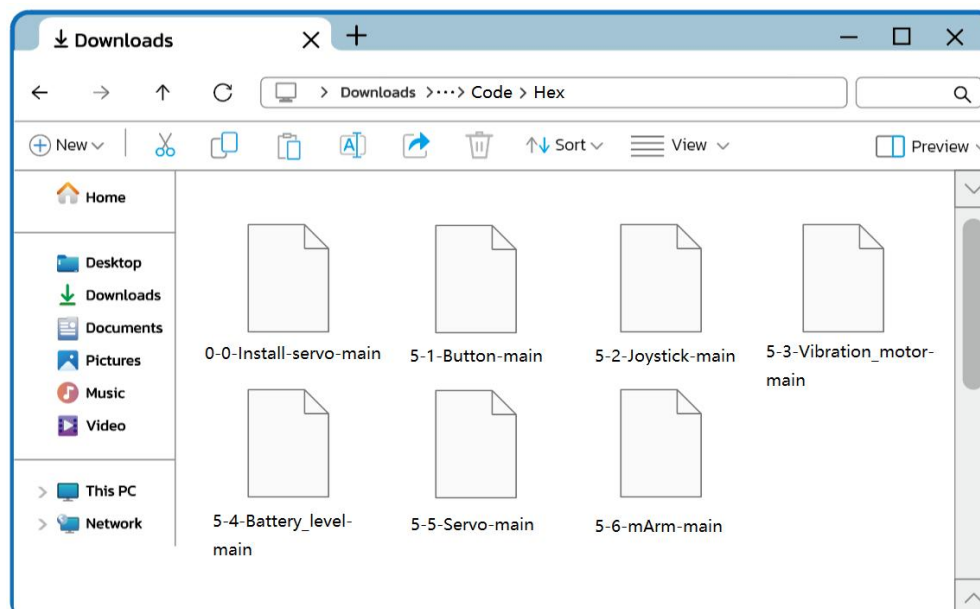
<https://microbit-micropython.readthedocs.io/en/v2-docs/>

5. 基本的なサンプルプロジェクト

すべての基本的なサンプル **Hex** コードは、「**Micropython > Code > Hex**」フォルダに保存されています:



チュートリアルの基本的な **Python** サンプルコードは、「**Micropython > Code > Python**」フォルダに保存されています:



セクション 4.3: ファイルのインポートをお読みください。チュートリアルパッケージからエディタにコードをインポートできます。

警告: micro:bitに新しいプログラムを書き込むと、既存のコードは上書きされます。以前のプログラムは失われます。新しいコードの書き込み後にロボットアームの制御を取り戻すには、必ず本章で提供されている最終プログラムを書き込んでください。

5.1 ボタン

目標

ジョイスティックの **A**、**B**、**C**、**D** ボタンの値を読み取ります。

コード

```
# Imports go at the top
from microbit import *

# Define the pins of the A, B, C, and D button
button_a = pin5
button_b = pin11
button_c = pin12
button_d = pin8

# I/O port with pull resistor
button_a.set_pull(button_a.PULL_UP)
button_b.set_pull(button_b.PULL_UP)
button_c.set_pull(button_c.PULL_UP)
button_d.set_pull(button_d.PULL_UP)

# Read button value function
def readButtonValue(button):
    # Return button value
    return button.read_digital()

# Code in a 'while True:' loop repeats forever
while True:
    # Determine whether button_a is pressed
    if readButtonValue(button_a) == 0:
        # If button_a is pressed, dot matrix displays character 'A'
        display.show('A')

    # Determine whether button_b is pressed
    if readButtonValue(button_b) == 0:
        # If button_b is pressed, dot matrix displays character 'B'
        display.show('B')
```

```

# Determine whether button_c is pressed
if readButtonValue(button_c) == 0:
    # If button_c is pressed, dot matrix displays character 'C'
    display.show('C')

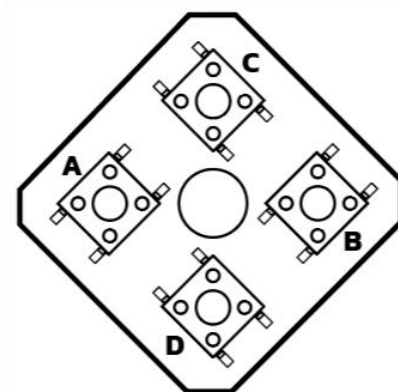
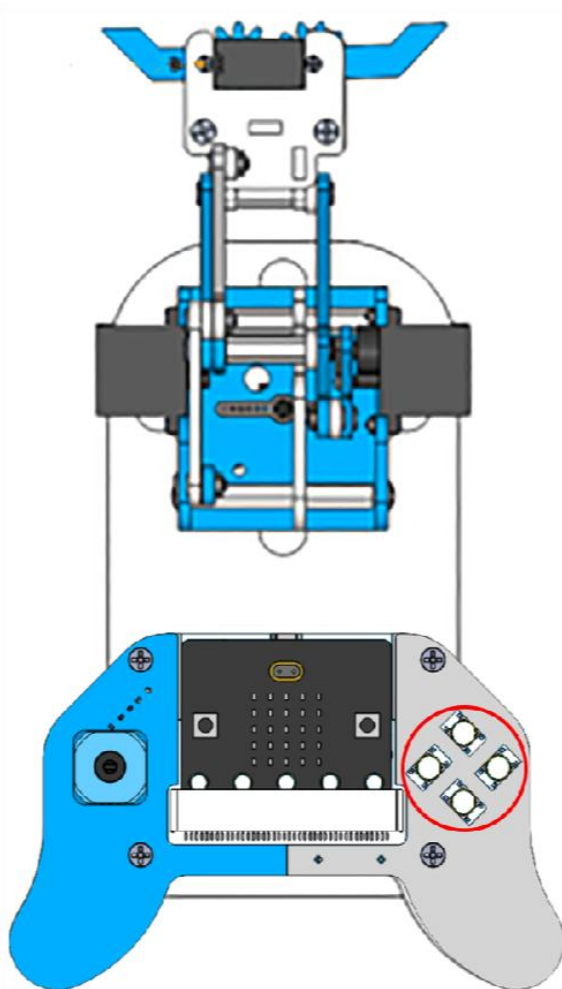
# Determine whether button_d is pressed
if readButtonValue(button_d) == 0:
    # If button_d is pressed, dot matrix displays character 'D'
    display.show('D')

# delay 100 millisecond
sleep(100)

```

結果

電源投入後、**micro:bit** のマトリクスディスプレイにハートの形が表示されます。ボタン **A**、**B**、**C**、または **D** がそれぞれ押されると、**micro:bit** の LED マトリクスに「**A**」、「**B**」、「**C**」、または「**D**」が表示されます。



5.2 のジョイスティック

目標

ジョイスティックのアナログ値をリアルタイムで読み取り、ジョイスティックの傾き方向に基づいて、**micro:bit** の **LED** マトリックスに対応する方向を示す文字を表示します。

コード

```
# Imports go at the top
from microbit import *

# Define the pins of the joystick
x_axis = pin1
y_axis = pin0

# Read the joystick value function
def readJoystickValue(axis):
    # Map the values from 0 to 1023 to -100 to +100,
    # and then return the mapped value
    return scale(axis.read_analog(), from_=(0, 1023), to=(-100, 100))

# Code in a 'while True:' loop repeats forever
while True:
    # Determine whether the value of the X-axis is greater than 80
    if readJoystickValue(x_axis) > 80:
        # Dot matrix displays character 'L'
        display.show('L')

    # Determine whether the value of the X-axis is less than -80
    if readJoystickValue(x_axis) < -80:
        # Dot matrix displays character 'R'
        display.show('R')

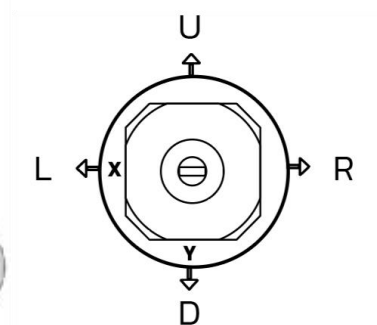
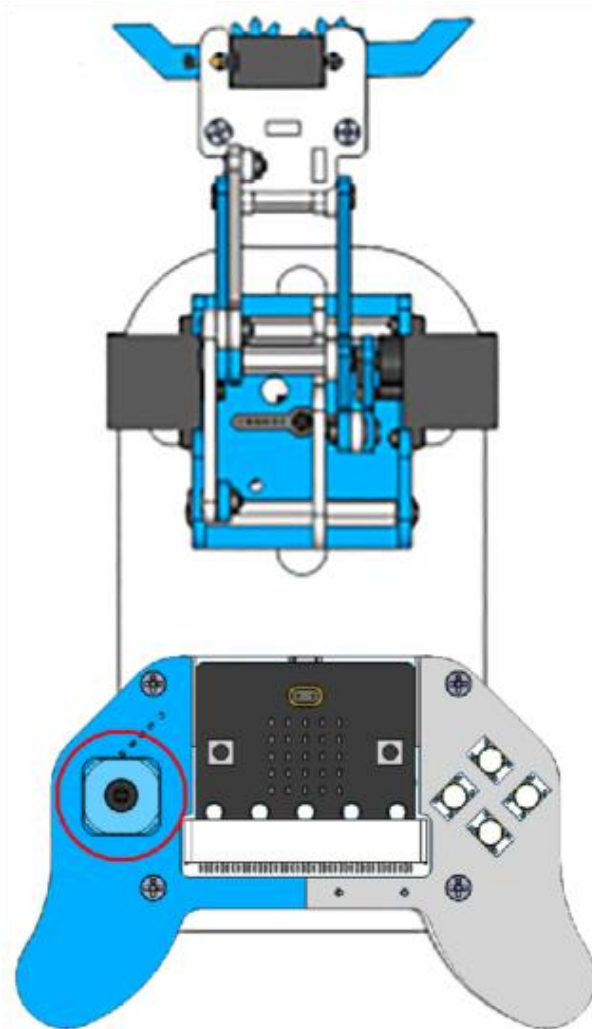
    # Determine whether the value of the Y-axis is greater than 80
    if readJoystickValue(y_axis) > 80:
        # Dot matrix displays character 'U'
        display.show('U')

    # Determine whether the value of the Y-axis is less than -80
    if readJoystickValue(y_axis) < -80:
        # Dot matrix displays character 'D'
        display.show('D')

    # delay 100 millisecond
    sleep(100)
```

結果

micro:bit の電源を入れると、ドットマトリクスにハートの形が表示されます。ジョイスティックを左、右、上、または下に押すと、micro:bit の LED マトリクスにそれぞれ「L」、「R」、「U」、または「D」が表示されます。



5.3 振動モーター

目標

ジョイスティックの振動モーターを作動させる。

コード

```
# Imports go at the top
from microbit import *

# Define the vibration motor state variables
ON = 1
OFF = 0

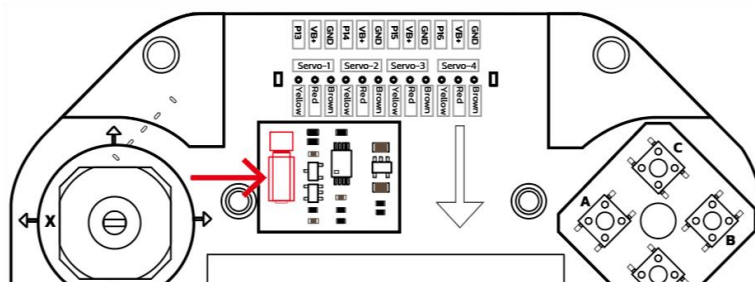
# Control the vibration motor function
def setVibrationMotor(OnOrOff):
    # Determine whether the value of the variable OnOrOff is equal to 1
    if OnOrOff == ON:
        # Turn on the vibration motor
        pin2.write_digital(1)
    # Determine whether the value of the variable OnOrOff is equal to 0
    if OnOrOff == OFF:
        # Turn off the vibration motor
        pin2.write_digital(0)

# Code in a 'while True:' loop repeats forever
while True:
    # Turn on the vibration motor
    setVibrationMotor(ON)
    sleep(1000) # delay 1000 millisecond

    # Turn off the vibration motor
    setVibrationMotor(OFF)
    sleep(1000) # delay 1000 millisecond
```

結果

ジョイスティックの振動モーターが、1秒ごとに1秒間振動します。



5.4 バッテリー電圧

目標

ジョイスティックのバッテリー残量を読み取ります。

コード

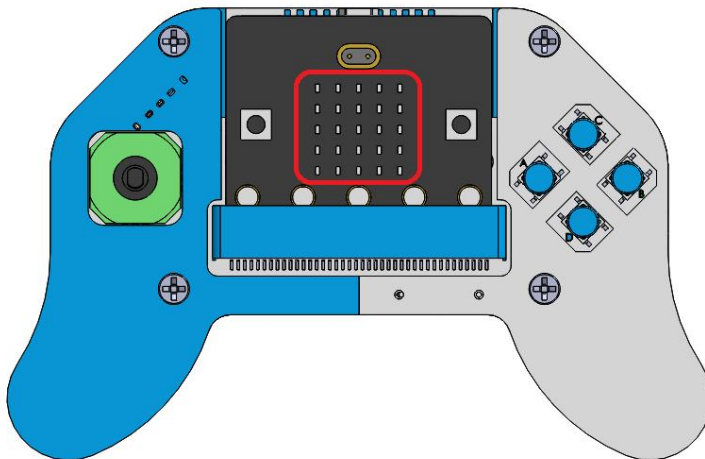
```
# Imports go at the top
from microbit import *

# Read the battery level
# 4 AA batteries
def readBatteryLevel():
    # Read the battery voltage value
    batLevel = pin2.read_analog()
    if batLevel > 310: # 310=6V/6/0.0032226, 100%
        batLevel = 310
    if batLevel < 232: # 232=4.5V/6/0.0032226, 0%
        batLevel = 232
    # Map 232-310 values to 0-100
    batLevel = scale(batLevel, from_=(232, 310), to=(0, 100))
    return batLevel # return battery level

# Code in a 'while True:' loop repeats forever
while True:
    # Dot matrix display of battery level
    display.show(readBatteryLevel())
    sleep(1000) # delay 1000 milliseconds
```

結果

Micro:bit の LED マトリックスに、0～100 の範囲でバッテリー残量値がそれぞれ表示されます



注

Micro:bit では、バッテリー電圧の読み取りと振動モーターの制御は同じ **P2** ピンを共有しています。バッテリー電圧の読み取りと振動モーターの制御は、頻繁に切り替えないでください。、バッテリー電圧の読み取り精度に影響が出る可能性があります。振動モーターを制御した後、電圧を連続して複数回読み取り、最後の値を採用することを推奨します。

5.5 サーボ

目標

サーボモーターの回転を制御する。

コード

```
# Imports go at the top
from microbit import *

# Servo class
class Servo:
    # The constructor of a class
    def __init__(self, pin):
        self.pin = pin
        # Servo parameters
        self.min_pulse = 500    # Pulse width at 0 degrees (microseconds)
        self.max_pulse = 2400   # Pulse width at 180 degrees (microseconds)
        self.period = 20000     # 20ms period (50Hz)
        self.pin.set_analog_period(20) # Set period to 20ms

    # Set the servo Angle function
    def set_angle(self, angle):
        # Limit angle range
        angle = max(0, min(180, angle))

        # Calculate pulse width
        pulse_width = self.min_pulse + (angle / 180) * (self.max_pulse - self.min_pulse)

        # Convert to analog value (0-1023)
        analog_value = int((pulse_width / self.period) * 1023)

        # Set PWM
        self.pin.write_analog(analog_value)
```

```

# Set the servo Angle function using pulse width
def set_pulse(self, pulse_width):
    # Limit the maximum pulse value
    pulse_width = max(self.min_pulse, min(self.max_pulse, pulse_width))

    # Calculate pulse
    analog_value = int((pulse_width / self.period) * 1023)

    # Set PWM
    self.pin.write_analog(analog_value)

# Show happy face
display.show(Image.HAPPY)
# delay 100 milliseconds
sleep(100)

# Create 4 servo objects and define the pins to use
servo1 = Servo(pin13) # servo1 uses pin13
servo2 = Servo(pin14) # servo2 uses pin14
servo3 = Servo(pin15) # servo3 uses pin15
servo4 = Servo(pin16) # servo4 uses pin16

# Initialize the angles of the 4 servos
servo1.set_angle(90) # Set the Angle of servo1
sleep(500)           # delay 500 milliseconds
servo2.set_angle(90) # Set the Angle of servo2
sleep(500)           # delay 500 milliseconds
servo3.set_angle(60) # Set the Angle of servo3
sleep(500)           # delay 500 milliseconds
servo4.set_angle(90) # Set the Angle of servo4
sleep(500)           # delay 500 milliseconds

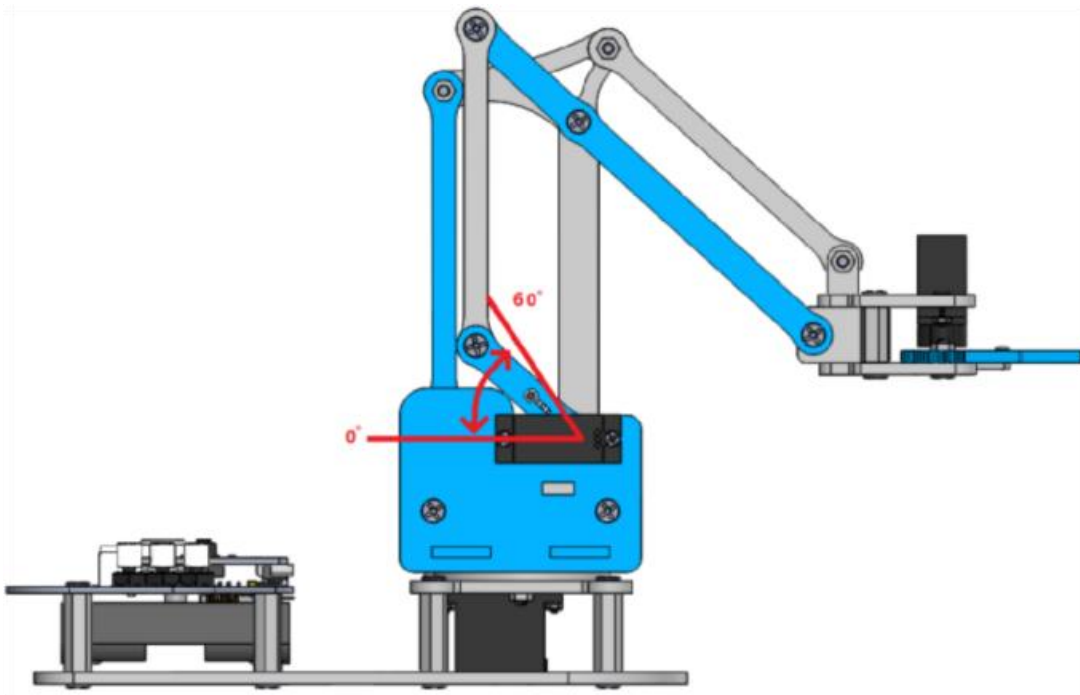
# Let servo 3 swing between 0 and 60 degrees.
while True:
    # The servo swings from 60 degrees to 0 degrees
    for i in range(60):
        # Set the Angle of servo3
        servo3.set_angle(60-i)
        # delay 20 milliseconds
        sleep(20)

    # The servo swings from 0 degrees to 60 degrees
    for i in range(60):
        # Set the Angle of servo3

```

```
servo3.set_angle(i)
# delay 20 milliseconds
sleep(20)
```

結果



サーボ **3** のスイングアームが **120 度** から **180 度** の範囲で揺れ、ロボットはロボットアームの上げ下げ動作を繰り返し行います。

注:

ジョイスティックには単 **3** 形電池 **4** 本が装着されており、電源スイッチがオンになっている必要があります！

5.6 ロボットアームの制御

目標

このセクションでは、「**3.2 ロボットアームの制御**」で使用されたコードについて詳しく説明します。他のプログラムをアップロードした後、このロボットアームを再度使用したい場合は、このコードをアップロードしてください。

コード

```
# Imports go at the top
from microbit import *

# Define the pins of the A, B, C, and D button
button_a = pin5
button_b = pin11
button_c = pin12
button_d = pin8

# I/O port with pull resistor
button_a.set_pull(button_a.PULL_UP)
button_b.set_pull(button_b.PULL_UP)
button_c.set_pull(button_c.PULL_UP)
button_d.set_pull(button_d.PULL_UP)

# Read key value function
def readButtonValue(button):
    return button.read_digital()

# Define the pins of the joystick
x_axis = pin1
y_axis = pin0

# Read the joystick value function
def readJoystickValue(axis):
    # Map 0-1023 values to -100 - +100
    return scale(axis.read_analog(), from_=(0, 1023), to=(-100, 100))

# Define the vibration motor state variables
ON = 1
OFF = 0

# Control the vibration motor function
def setVibrationMotor(OnOrOff):
    if OnOrOff == ON:    # Turn on the vibration motor
```

```

        pin2.write_digital(1)
    if OnOrOff == OFF: # Turn off the vibration motor
        pin2.write_digital(0)

# Read the battery level.
# 4 AA batteries
def readBatteryLevel():
    batLevel = pin2.read_analog()
    if batLevel > 310: # 310=6V/6/0.0032226, 100%
        batLevel = 310
    if batLevel < 232: # 232=4.5V/6/0.0032226, 0%
        batLevel = 232
    # Map 232-310 values to 0-100
    batLevel = scale(batLevel, from_=(232, 310), to=(0, 100))
    return batLevel

# Servo class
class Servo:
    def __init__(self, pin):
        self.pin = pin
        # Servo parameters
        self.min_pulse = 500 # Pulse width at 0 degrees (microseconds)
        self.max_pulse = 2400 # Pulse width at 180 degrees (microseconds)
        self.period = 20000 # 20ms period (50Hz)
        self.pin.set_analog_period(20) # Set period to 20ms

    def set_angle(self, angle):
        # Limit angle range
        angle = max(0, min(180, angle))

        # Calculate pulse width
        pulse_width = self.min_pulse + (angle / 180) * (self.max_pulse - self.min_pulse)

        # Convert to analog value (0-1023)
        analog_value = int((pulse_width / self.period) * 1023)

        # Set PWM
        self.pin.write_analog(analog_value)

    def set_pulse(self, pulse_width):
        # Set pulse width directly (microseconds)
        pulse_width = max(self.min_pulse, min(self.max_pulse, pulse_width))
        analog_value = int((pulse_width / self.period) * 1023)
        self.pin.write_analog(analog_value)

```

```

# Show happy face
display.show(Image.HAPPY)
sleep(100) # delay 100 milliseconds

# Define the Angle variables of the 4 servos
servo1_degree = 90
servo2_degree = 90
servo3_degree = 60
servo4_degree = 90

# Create 4 servo objects and define the pins to use
servo1 = Servo(pin13)
servo2 = Servo(pin14)
servo3 = Servo(pin15)
servo4 = Servo(pin16)

# Initialize the angles of the 4 servos
servo1.set_angle(servo1_degree) # Set the Angle of servo1
sleep(500) # delay 500 milliseconds
servo2.set_angle(servo2_degree)
sleep(500)
servo3.set_angle(servo3_degree)
sleep(500)
servo4.set_angle(servo4_degree)
sleep(500)

# Reads the current time when the program is running
last_time = running_time()
interval = 5000 # 5000ms
last_ticker = running_time()
ticker = 40000 # 40000ms

# An infinite loop statement.
while True:
    # Reads the current time when the program is running
    current_time = running_time()
    if current_time - last_time >= interval:
        # Display battery power value
        display.show(readBatteryLevel())
        last_time = current_time

    # Prevent the servo of the claws from overheating
    current_ticker = running_time()

```



```

if current_ticker - last_ticker >= ticker:
    # Turn off the pulse of the servo
    servo4.set_pulse(0)

# Read the value of the joystick X-axis and drive the Angle
# of the servo1 according to the value.
if readJoystickValue(x_axis) > 60 and servo1_degree < 180:
    servo1_degree = servo1_degree + 1 # Variable increments by 1
    servo1.set_angle(servo1_degree) # Set the Angle of servo1
    sleep(15)
if readJoystickValue(x_axis) < -60 and servo1_degree > 0:
    servo1_degree = servo1_degree - 1 # The variable decrements by 1
    servo1.set_angle(servo1_degree)
    sleep(15)

# Read the value of the joystick Y-axis and drive the Angle
# of the servo2 according to the value.
if readJoystickValue(y_axis) > 60 and servo2_degree < 180:
    servo2_degree = servo2_degree + 1
    servo2.set_angle(servo2_degree)
    sleep(15)
if readJoystickValue(y_axis) < -60 and servo2_degree > 0:
    servo2_degree = servo2_degree - 1
    servo2.set_angle(servo2_degree)
    sleep(15)

# Read the value of the button_a and drive the Angle
# of the servo4 according to the value.
if readButtonValue(button_a) == 0 and servo4_degree > 5:
    servo4_degree = servo4_degree - 1
    servo4.set_angle(servo4_degree)
    sleep(15)

# Read the value of the button_b and drive the Angle
# of the servo4 according to the value.
if readButtonValue(button_b) == 0 and servo4_degree < 180:
    servo4_degree = servo4_degree + 1
    servo4.set_angle(servo4_degree)
    sleep(15)

# Read the value of the button_c and drive the Angle
# of the servo3 according to the value.
if readButtonValue(button_c) == 0 and servo3_degree > 0:
    servo3_degree = servo3_degree - 1

```

```

servo3.set_angle(servo3_degree)
sleep(15)

# Read the value of the button_d and drive the Angle
# of the servo3 according to the value.
if readButtonValue(button_d) == 0 and servo3_degree < 120:
    servo3_degree = servo3_degree + 1
    servo3.set_angle(servo3_degree)
    sleep(15)

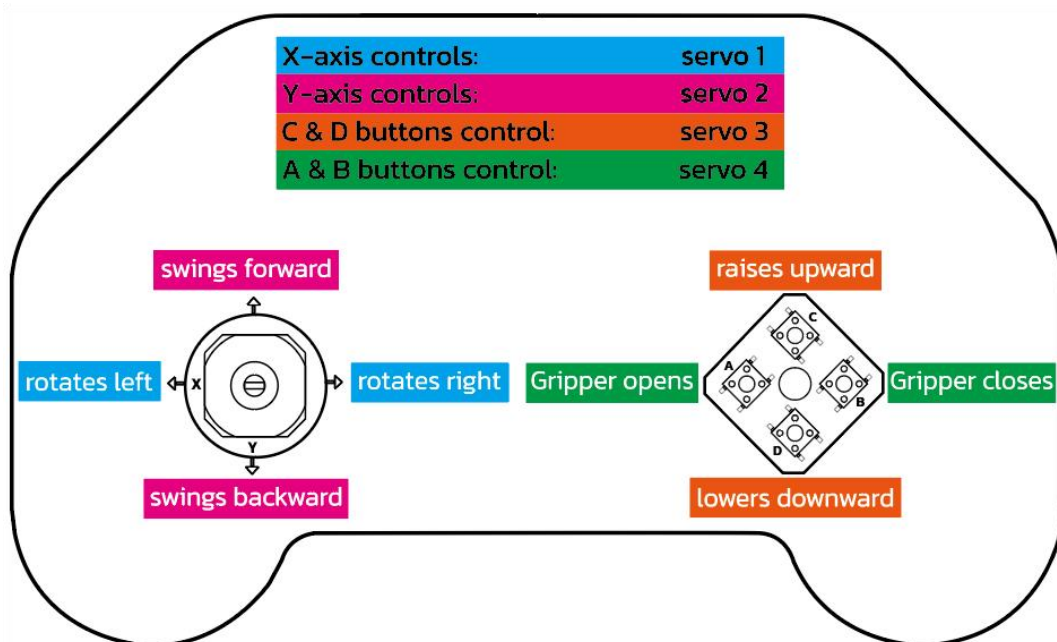
# Executed when the micro:bit v2 logo is touched.
# Attention! Not compatible with microbit v1.
if pin_logo.is_touched():
    setVibrationMotor(ON)
    sleep(200)
    setVibrationMotor(OFF)

```

結果

電源投入後、micro:bit の LED マトリックスはバッテリー残量（0%～100%）を表示し続けます。バッテリー残量が 30%を下回った場合は、直ちにすべての電池を交換してください。

- ジョイスティックと4つのボタンを使用して、ロボットアームの動きを制御できます。
- micro:bitのタッチアイコンを押すと、コントローラーの振動モーターが作動します。



注意！ micro:bit v1 はロゴタッチ機能に対応していません。micro:bit v1 でこのプログラムを使用する場合は、以下のコードを削除する必要があります：

```
# Executed when the micro:bit v2 logo is touched.  
# Attention! Not compatible with microbit v1.  
if pin_logo.is_touched():  
    setVibrationMotor(ON)  
    sleep(200)  
    setVibrationMotor(OFF)
```

6.QA

6.1 micro:bit のコードアップロードに失敗しました

- ☛ データ転送対応の USB ケーブルをご使用ください（充電専用ケーブルでは動作しません）。
- ☛ USB ケーブルやポートに物理的な損傷や接続の問題がないか確認してください。

6.2 ロボットアームが動作しません。

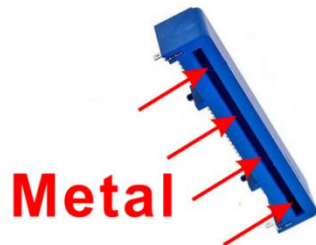
- ☛ 単 3 形電池 4 本が正しく挿入されているか、電源スイッチが ON の位置にあることを確認してください。
- ☛ バッテリーの電圧レベルを確認してください（推奨最低電圧: 5.5V）。
- ☛ チュートリアルガイドに従って、サーボの初期化シーケンスが完了していることを確認してください。

6.3 micro:bit ドライバーに異常な動作が見られる。

- ☛ 「MICROBIT」ではなく「MAINTENANCE」と表示されている場合は、誤ってファームウェアリカバリモードに入ってしまったことを示しています。解決策:
 - 公式ツールを使用してファームウェアを再書き込みしてください:
<https://microbit.org/get-started/user-guide/firmware/>
 - 電源のオン/オフ時にリセットボタンを押さないようにし、再発を防止してください。

6.4 ボタンまたはジョイスティックが反応しない

- micro:bit が mJoystick スロットに完全に差し込まれていることを確認してください。
- micro:bit のエッジコネクタが汚れていないか確認してください。
- mJoystick スロットが汚れていないか確認してください。



6.5 micro:bit v1 使用時のエラー

micro:bit v1 で第 5.6 章のサンプルコード「5-6-mArm.hex」を使用する際にエラーが発生する場合は、そのコードが micro:bit v2 でのみ利用可能な「`pin_logo.is_touched()`」ステートメントを使用しているためです。

- micro:bit v1にはロゴタッチ機能がないため、エラーが発生します。「`pin_logo.is_touched()`」ステートメントを削除し、.hexファイルを再生成してから、micro:bit v1ボードにアップロードしてください。

6.6 その他の問題

- 組み立てが正しいか確認してください。
- 使用している電池が仕様を満たしているか確認してください。

7.お問い合わせ

上記の解決策が見つからない場合は、サポートチームまでご連絡ください。

迅速な対応のため、以下の情報をご用意ください：

ご注文番号。

製品モデル（例：ロボットアームキット **M1R0000**）およびソフトウェアバージョン（例：**MicroPython v3.X.X**）。

お問い合わせ内容または問題の詳細な説明。

これまでに試された手順。

関連するエラーメッセージのスクリーンショット、写真、またはコードの抜粋。

その他、ご質問はありますか？

皆様からのご意見をお待ちしております。改善の参考にさせていただきます。お気軽にお問い合わせください：

チュートリアルのエラーやフィードバック：ドキュメントの改善にご協力ください。

製品のアイデアやご提案：素晴らしいアイデアをお待ちしています。

パートナーシップおよびコラボレーション：ご協力にご興味をお持ちですか？ぜひご相談ください。

割引・キャンペーン：教育機関向け価格やまとめ買い価格についてお問い合わせください。

その他のお問い合わせ：技術的な内容以外のご質問はこちらへ。

サポート窓口



support@siyeenove.com



<https://siyeenove.com>